

05-A1

二叉树

树：术语

他是一位最古老的神，古老就是一种荣誉
他的古老有一个凭证，就是他没有父母

越到你的高度上——那是我的深度！
藏在你的纯洁里——那是我的天真！

邓俊辉

deng@tsinghua.edu.cn

动机

❖ 【应用】层次结构的表示

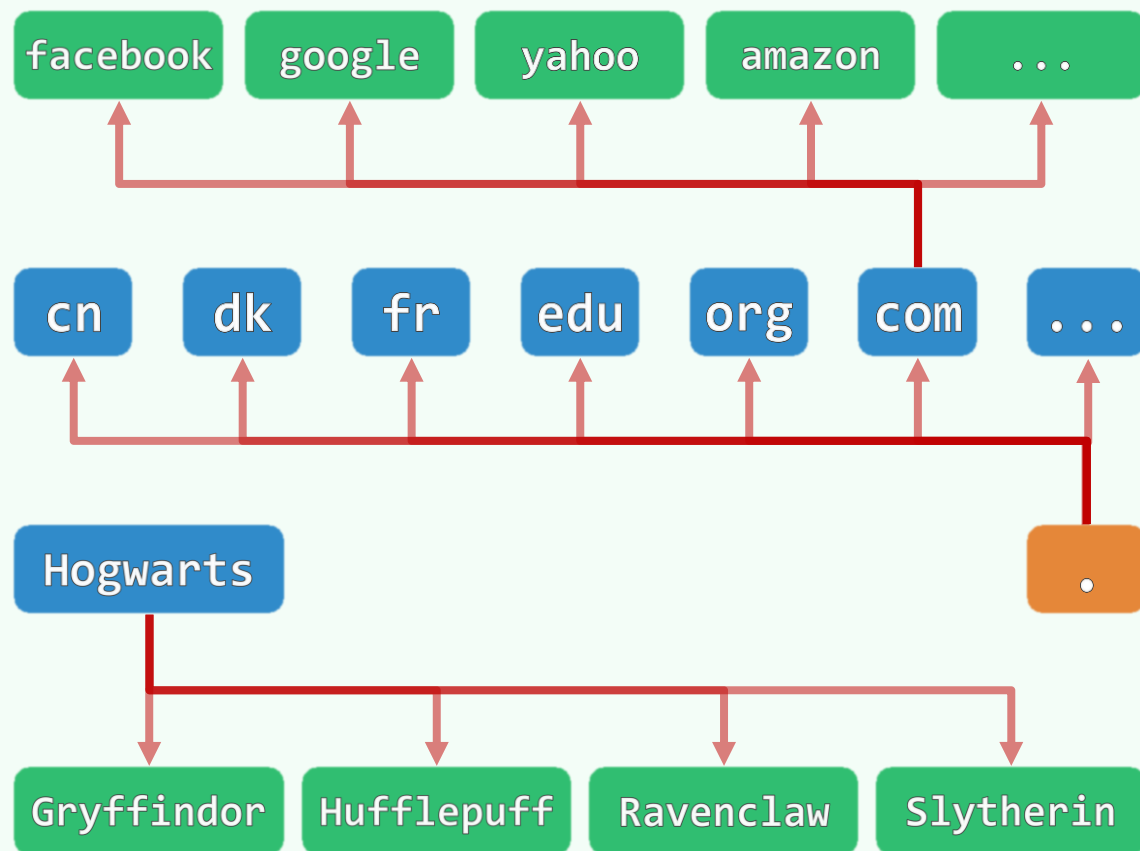
- 表达式
- 文件系统
- URL ...

❖ 【数据结构】综合性

- 兼具Vector和List的优点
- 兼顾高效的**查找**、**插入**、**删除**

❖ 【半线性】

- 不再是简单的线性结构，但
- 在确定某种**次序**之后，具有线性特征



有根有序树

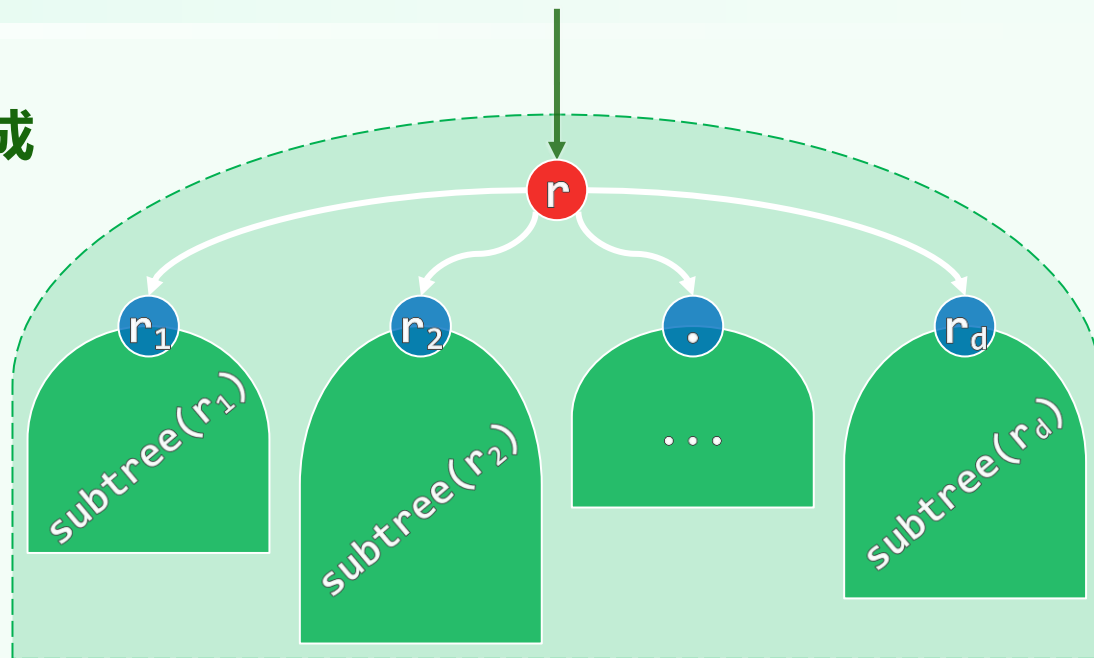
❖ 树也是图，由**顶点** (vertex) 和**联边** (edge) 构成

顶点对应的数据结构为**节点**，不妨混称

❖ 指定某一节点为**根**后，即是有根树

tree rooted at r = tree(r)

❖ 任给若干棵有根**子树** (subtree) $\{r_1, r_2, \dots, r_d\}$



引入新节点 r ，并向 r_i 分别发出一条**有向边**，便可得到一棵（更大、更高的）有根树

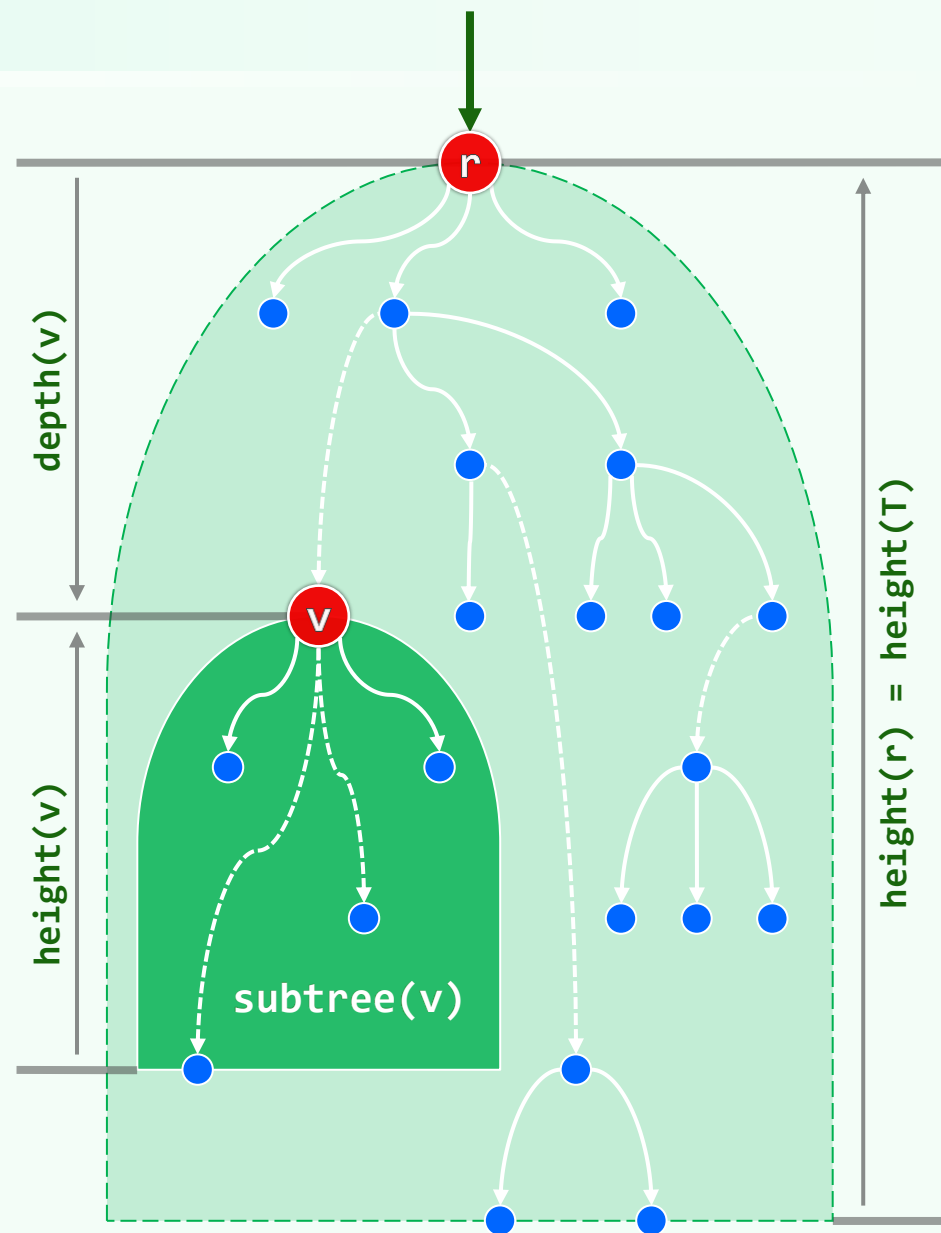
❖ r 和 r_i 互称**父** (parent)、**子** (child)；边数 d 为 r 的**度** (degree)

零度节点称**叶子** (leaf)，其余为**内部节点** (internal node)

❖ **兄弟** (sibling) 之间若依 $\{r_1, r_2, \dots, r_d\}$ 排序，则是**有序树** (ordered tree)

深度 + 层次

- ❖ 从任一**祖先** (ancestor) r 出发, 沿边逐代下溯
可以抵达各级**后代** (descendant) v
- ❖ 沿途各边串接成一条**路径**, **边数**即路径的**长度**
- ❖ 若 r 为全树之根, 则路径长度即 v 的**深度** (depth)
- ❖ 依照深度, 可将节点划分为 $h + 1$ 类; 根自成一类
- ❖ 所有节点的**最大深度**, 即树的**高度** (height)
特别地, **空树**的高度为 **-1**
- ❖ 节点的**高度** = 后代的最大深度 - 自己的深度
- ❖ $\forall v, \text{depth}(v) + \text{height}(v) \leq \text{height}(T) = \text{height}(r)$



二叉树

树：ADT及实现

然而现在他有了一个儿子，这是他的亲骨血，他所最亲爱的人，他可以好好地教养他，把他的抱负拿来在儿子的身上实现。儿子的幸福就是他自己的幸福

木晦於根，春榮華敷；人晦於身，神明內腴

邓俊辉

deng@tsinghua.edu.cn

Tree ADT

操作接口	功能
<code>size() empty()</code>	报告树的规模 判空
<code>root()</code>	树根节点
<code>x.parent()</code>	x的父节点
<code>x.firstChild()</code>	x的长子
<code>x.nextSibling()</code>	x的（按齿序的下一个）弟弟
<code>x.insert(i,e)</code>	将e作为x的第i个孩子插入
<code>x.remove(i)</code>	删除x的第i个孩子（及其后代）
<code>traverse(visit())</code>	遍历全树，统一按visit()处理所有节点

父节点

rank	parent	data
0	2	H
1	7	E
2	9	F
3	4	B
4	4	R
5	2	K
6	7	D
7	4	A
8	2	G
9	4	C

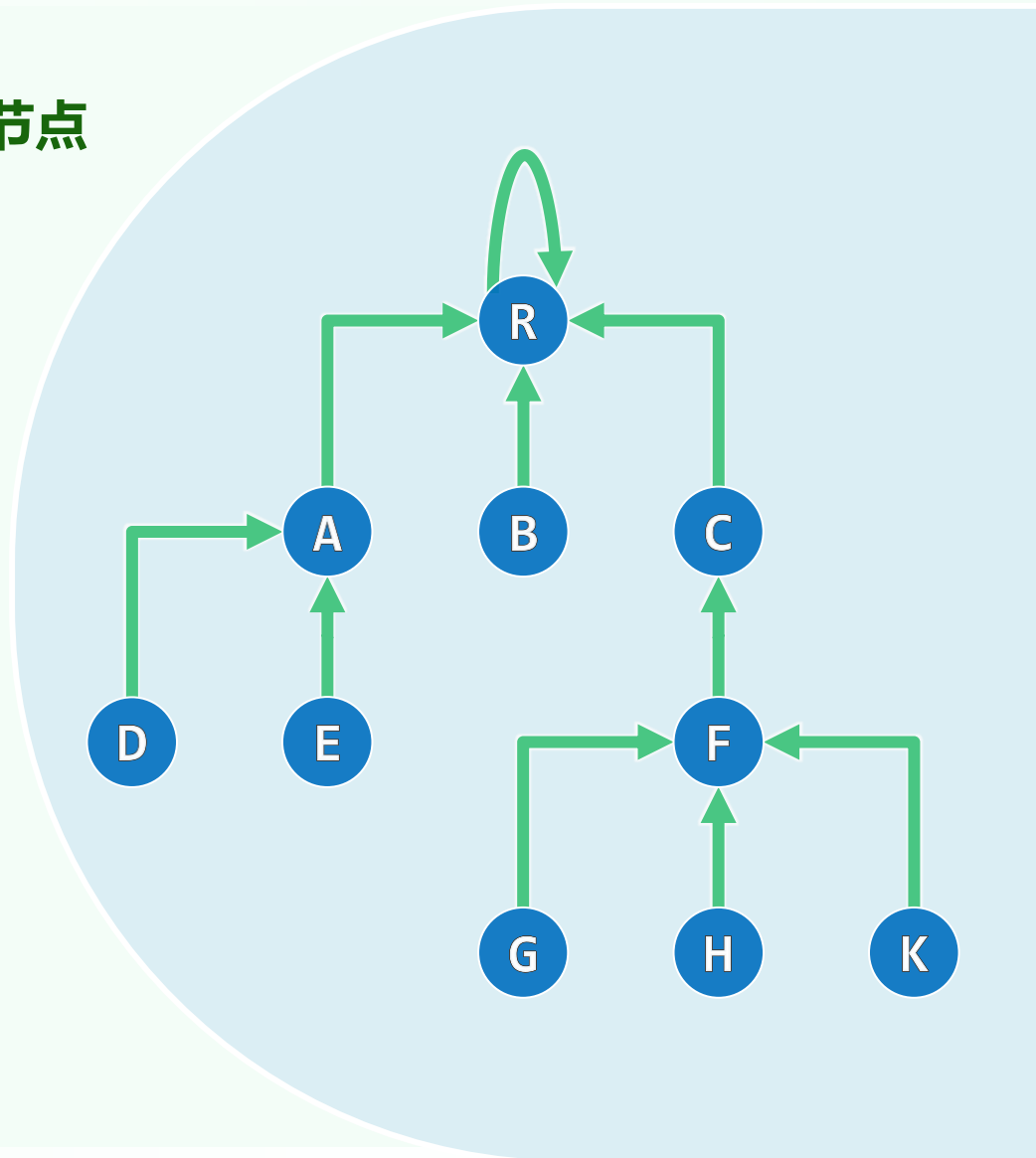
❖ 除根外，任一节点有且仅有一个父节点

❖ 节点组织为一个序列，各自记录：

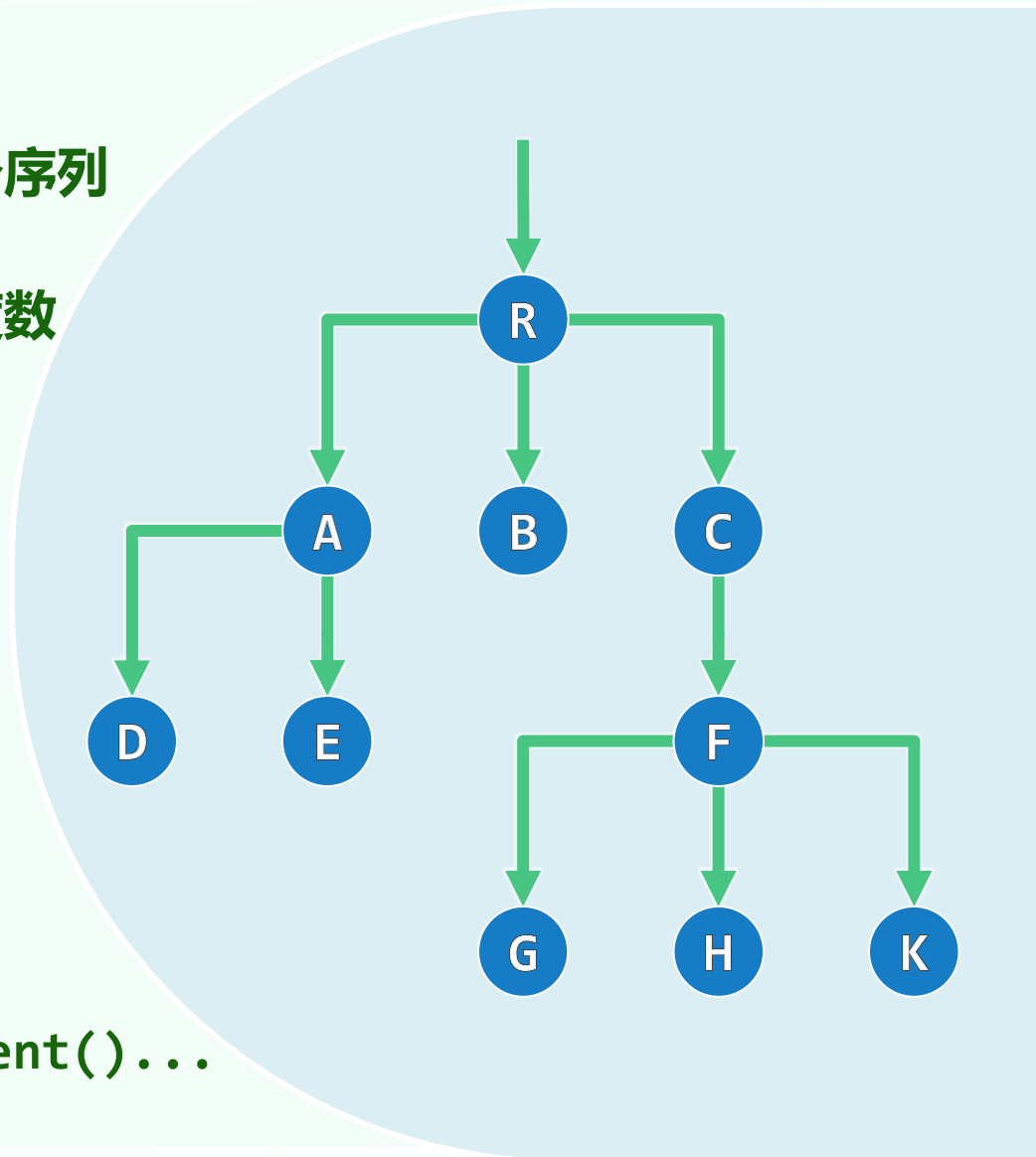
data 本身信息

parent 父节点的秩或位置

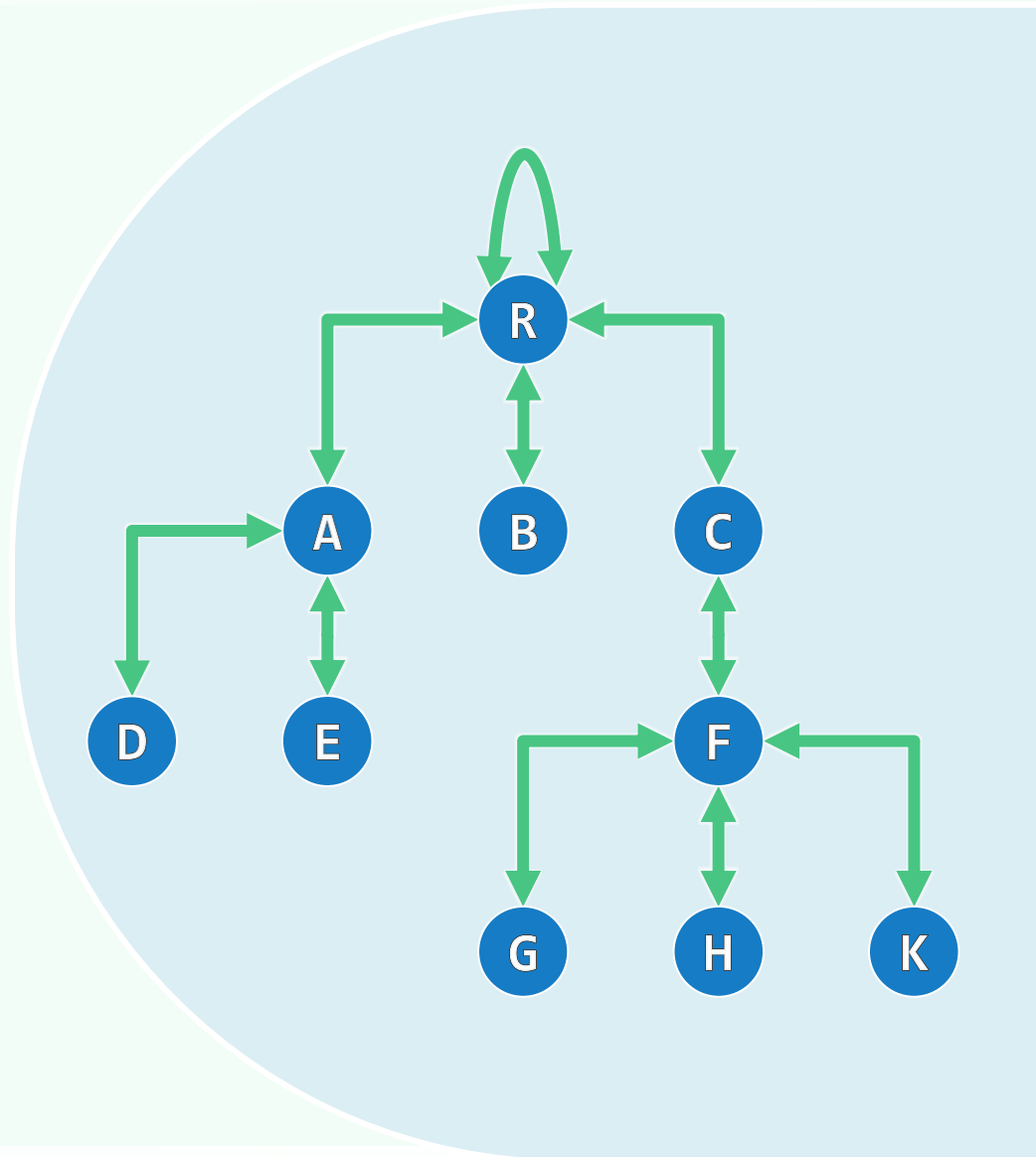
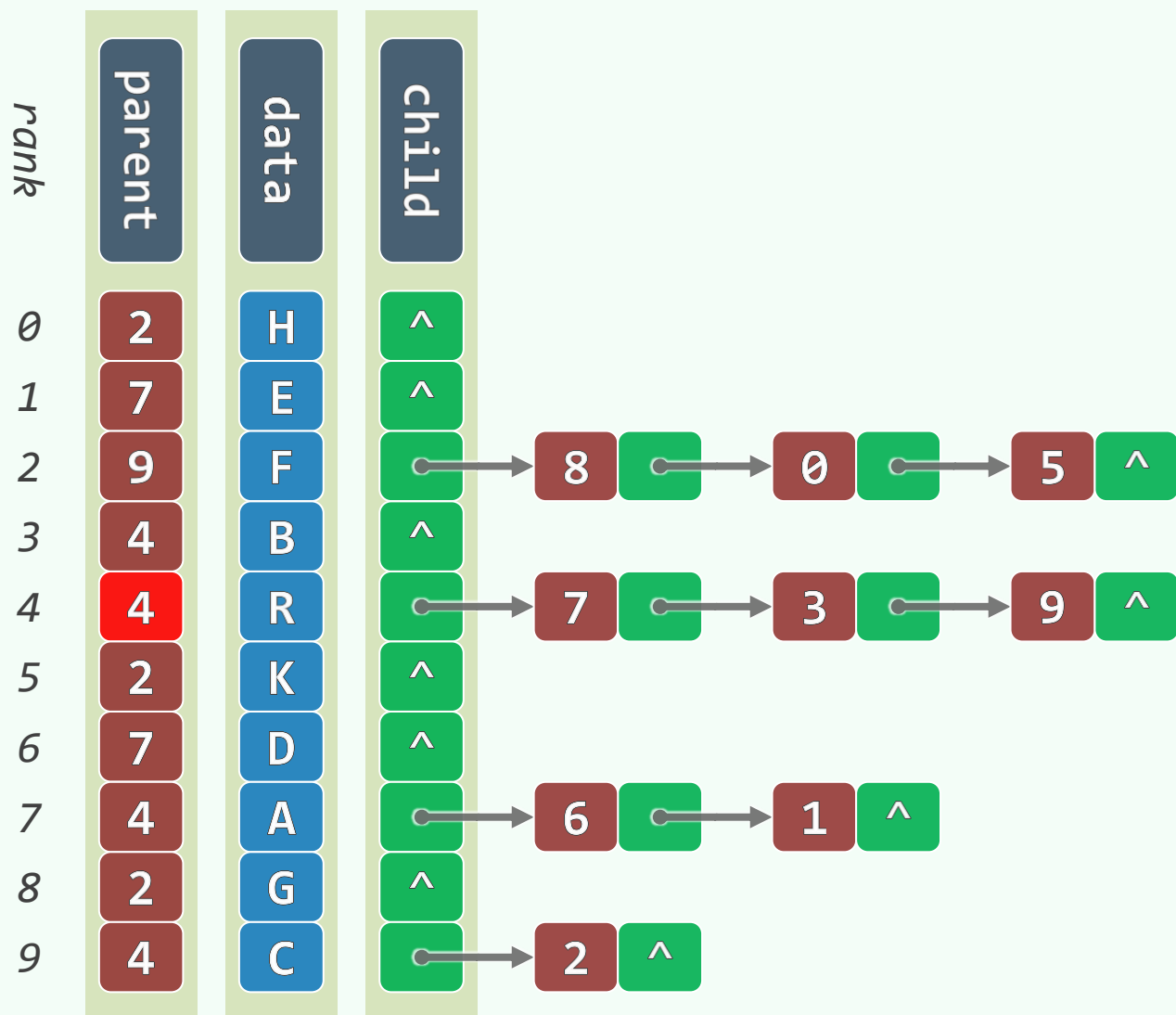
❖ 树根：R ~ $\text{parent}(4) = 4$



孩子节点



父节点 + 孩子节点



二叉树

二叉树：性质与转换

当地平线消失

躯体保持水平 大地保持水平

但别的一切 都垂直

真的嘛？您早已知道了我的名字吗？

对，您的名字叫“我的兄弟”。

咱们俩个人一样的年纪，况且又是同窗，以后不必论叔侄，只论弟兄朋友就是了

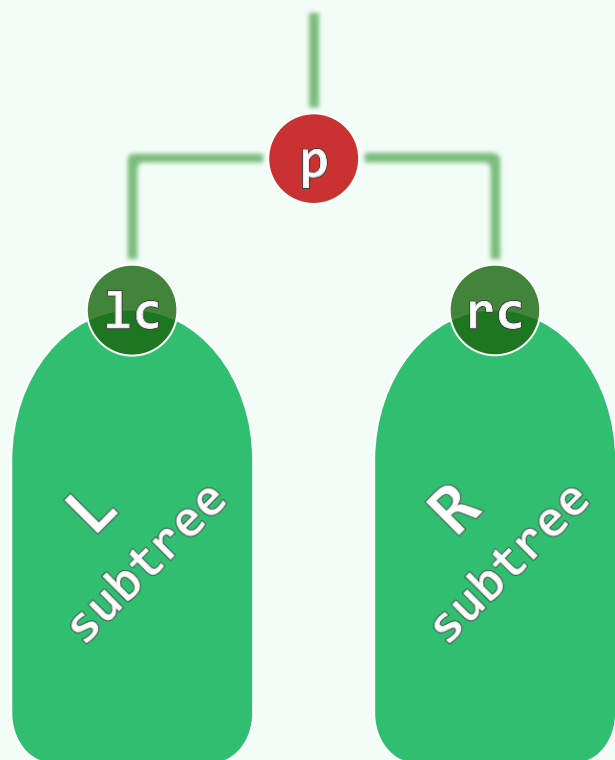
邓俊辉

deng@tsinghua.edu.cn

二叉树

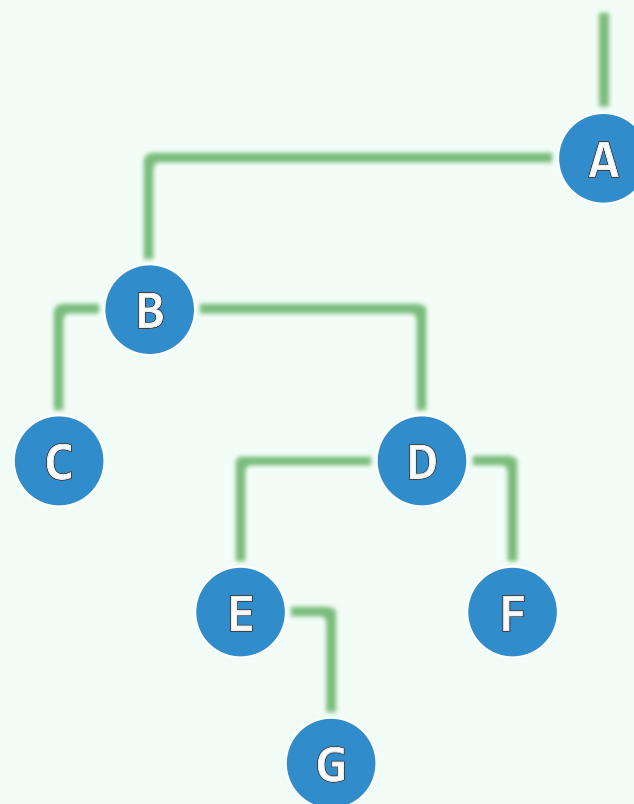
❖ Binary Tree: 节点度数均不超过2

❖ p称lc、rc为**左孩子/子树**、**右孩子/子树**



❖ lc、rc称p为**左父亲**、**右父亲**

❖ 推而广之: **左祖先**、**右祖先**



基数：设度数为0、1和2的节点，各有 n_0 、 n_1 和 n_2 个

❖ 边数 $e = n - 1 = n_1 + 2 \cdot n_2$

1、2度节点，各贡献1、2条边

❖ 叶节点数 $n_0 = n_2 + 1$

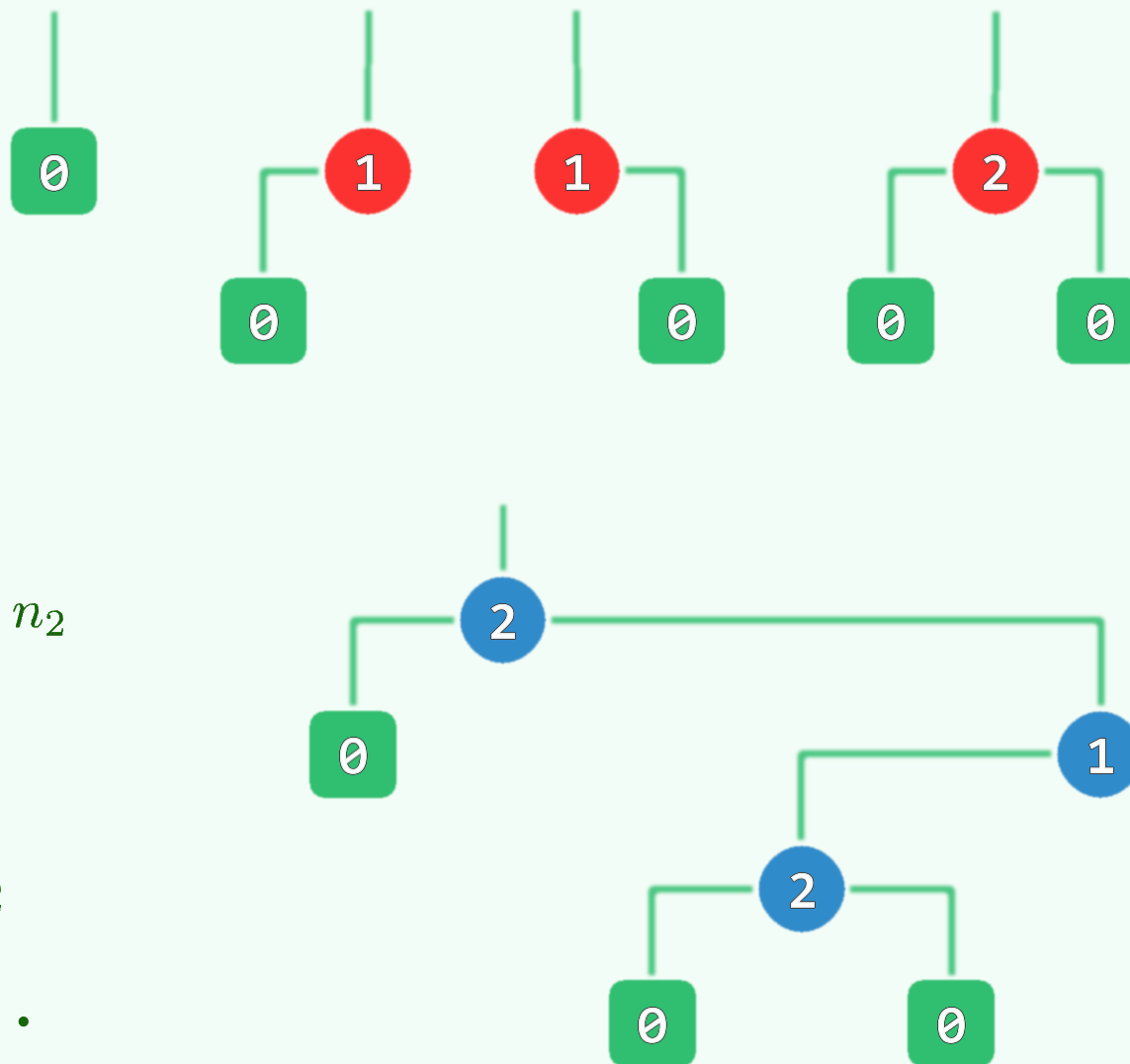
与 n_1 无关

❖ 节点总数 $n = n_0 + n_1 + n_2 = 1 + n_1 + 2 \cdot n_2$

❖ 特别地，当 $n_1 = 0$ 时为**真二叉树**，有

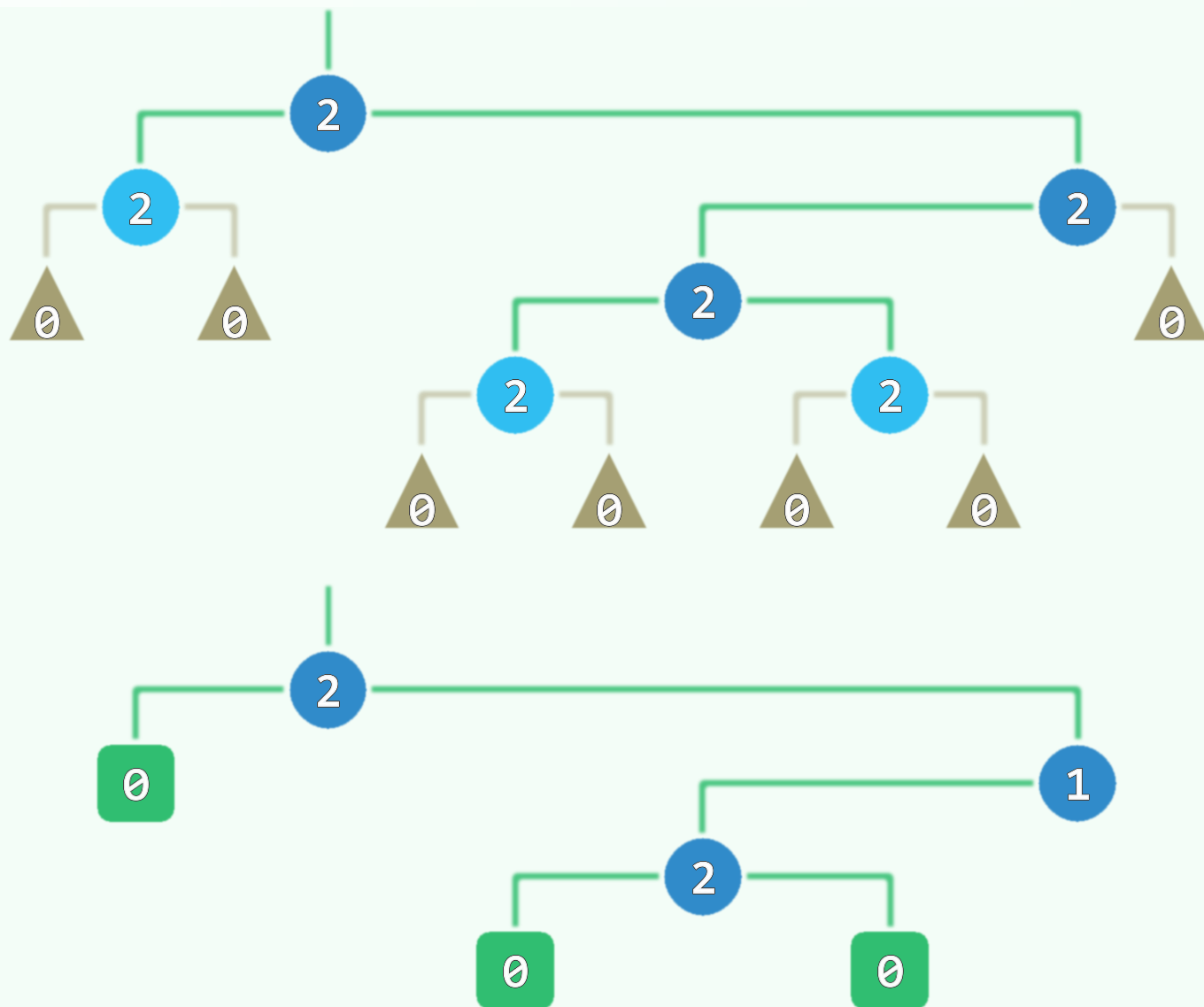
$$e = 2 \cdot n_2 \quad \text{和} \quad n_0 = 1 + n_2 = (n + 1)/2$$

此时，节点度数均为**偶数**，不含单分支节点...



真二叉树

- ❖ 通过引入 $n_1 + 2 \cdot n_0$ 个外部节点
可使原有节点度数统一为2
- ❖ 如此，每棵二叉树都可转化为
真二叉树 (proper binary tree)
- ❖ 如此转换之后
全树自身的复杂度并未实质增加
- ❖ 红黑树之类的结构经如此转换之后
可以简化描述、理解、实现和分析



满树

❖ 深度为 k 的节点，至多 2^k 个

❖ 节点总数 n 与高度 h 必满足

$$h + 1 \leq n \leq 2^{h+1} - 1$$

❖ 特殊情况

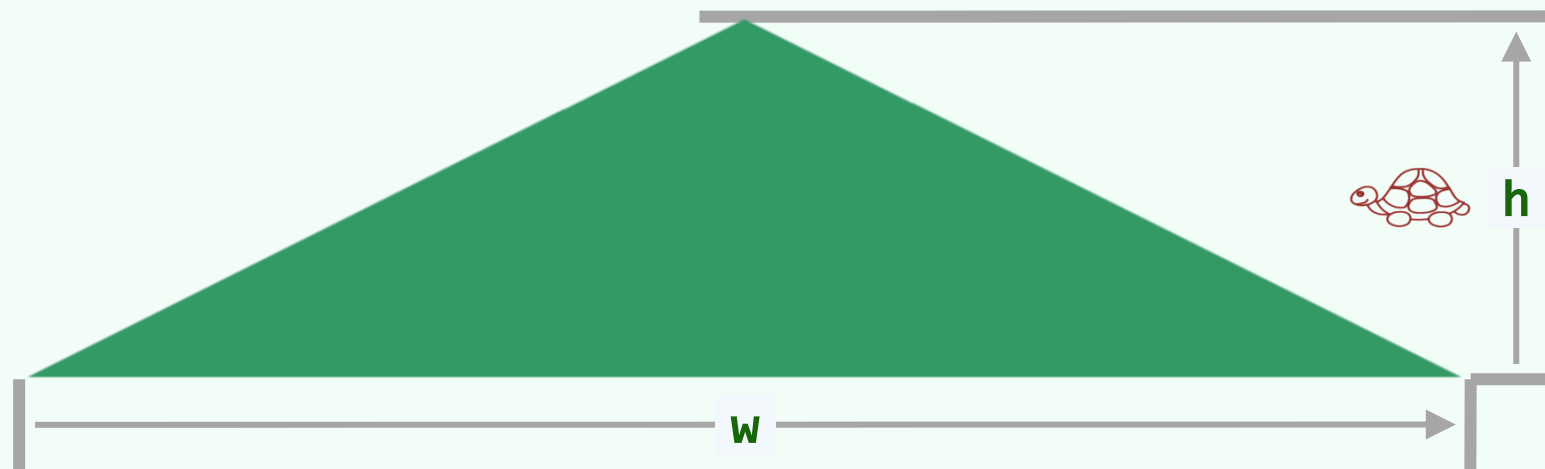
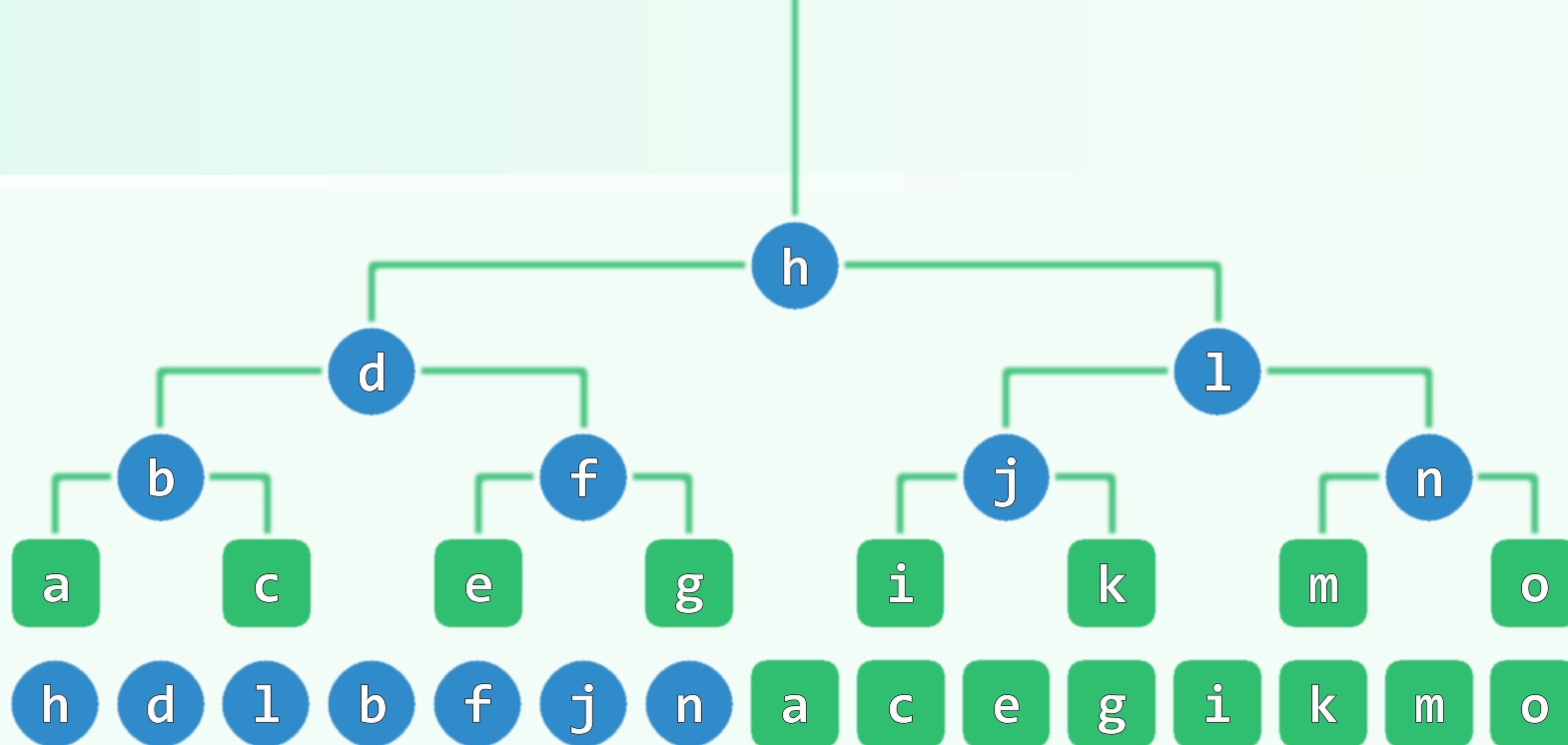
- $n = h + 1$

退化为一条单链

- $n + 1 = 2^{h+1}$

满二叉树

full binary tree

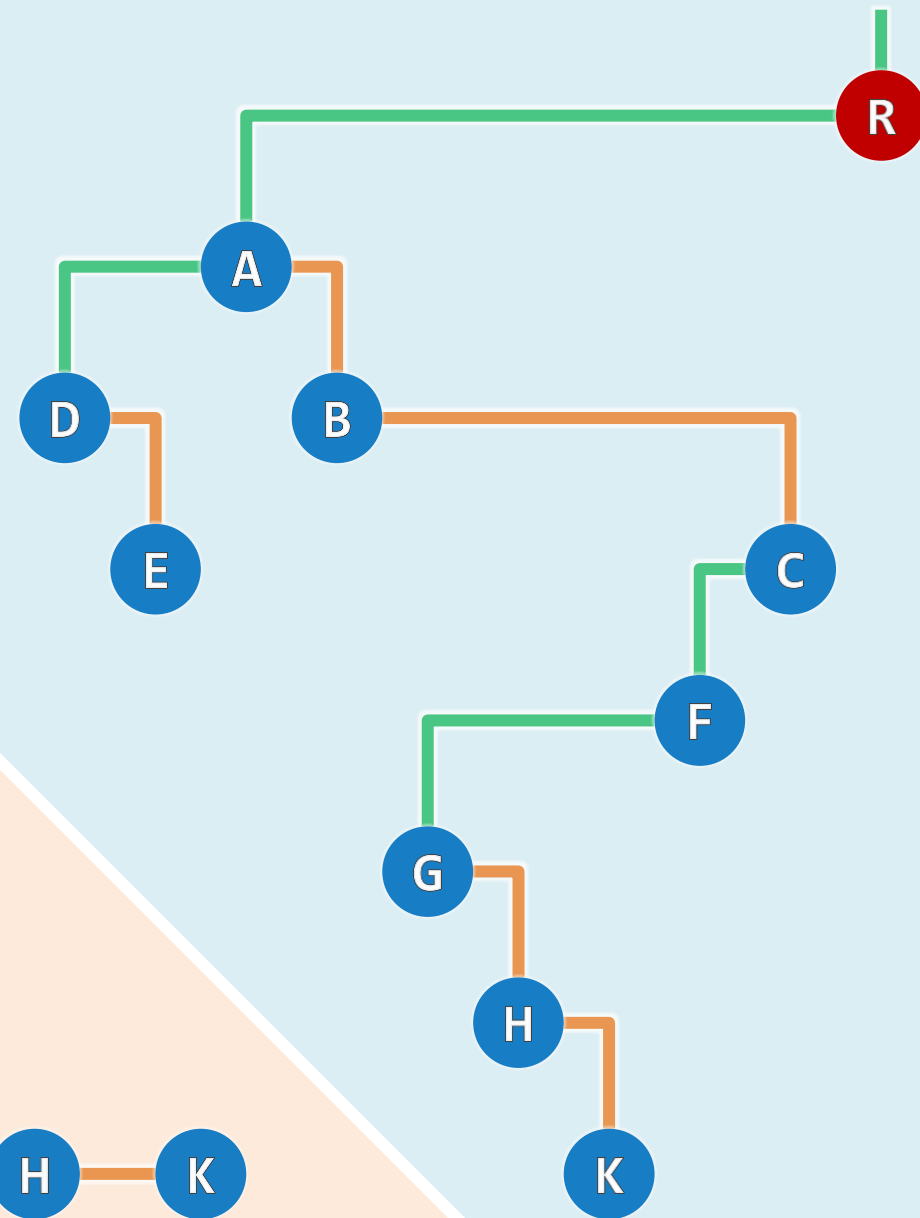
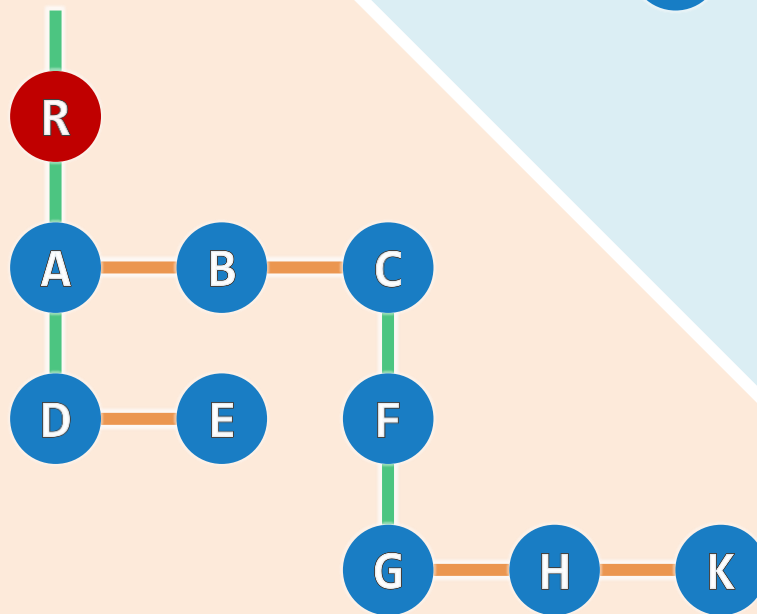
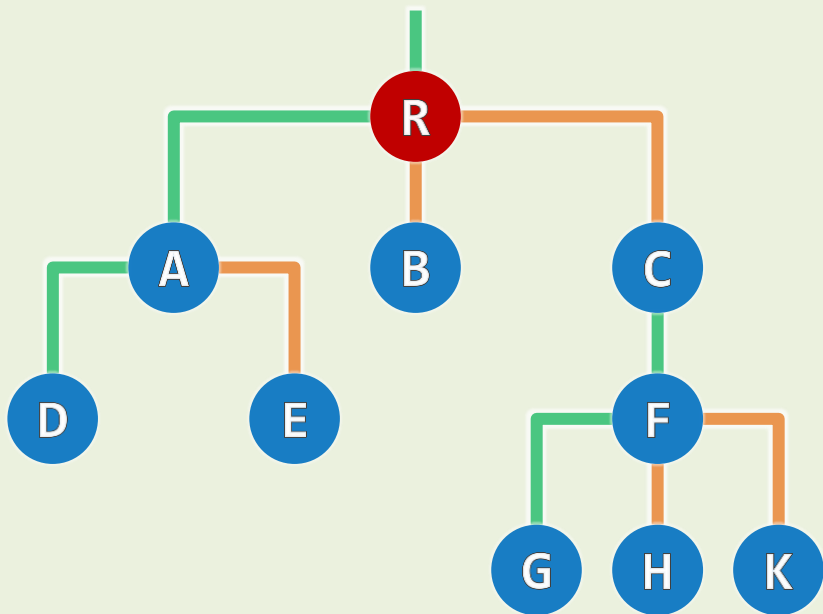


描述多叉树：“长子-弟弟”表示法

❖ 有根且有序的多叉树，均可转化并表示为二叉树

❖ 长子 ~ 左孩子 `firstChild()` ~ `lc()`

弟弟 ~ 右孩子 `nextSibling()` ~ `rc()`



二叉树

二叉树：实现

Two roads diverged in a yellow wood
And sorry I could not travel both

Anyone who loves his father or mother more than me is not
worthy of me; anyone who loves his son or daughter more
than me is not worthy of me.

邓俊辉

deng@tsinghua.edu.cn

BinNode模板类

```
template <typename T> using BNP = BinNode<T>*; //Binary Node Position
```

```
template <typename T> struct BinNode {
```

```
    BNP<T> parent, lc, rc; //父亲、孩子
```

```
    T data; int height; RBColor color; //数据、(黑)高度/np1、颜色
```

```
    Rank updateHeight(); void updateHeightAbove(); //更新规模、高度
```

```
    BNP<T> insertLc( const T & ); //插入左孩子
```

```
    BNP<T> insertRc( const T & ); //插入右孩子
```

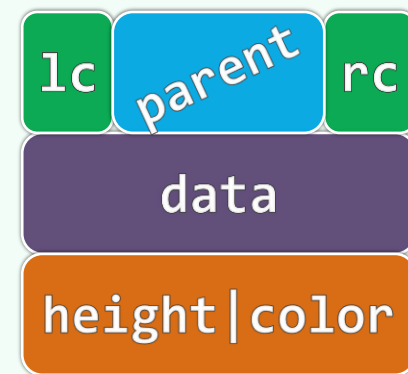
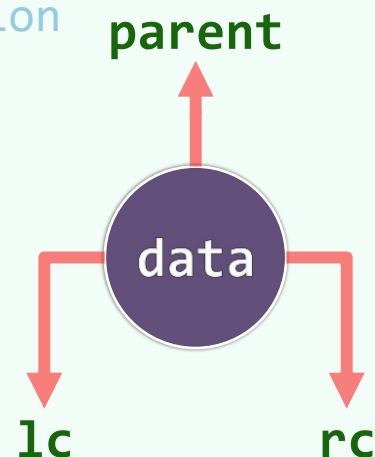
```
    void attachLc( BNP<T> ); //插入左子树
```

```
    void attachRc( BNP<T> ); //插入右子树
```

```
    BNP<T> succ(); // (中序遍历意义下) 当前节点的直接后继
```

```
    template <typename V> void traverse( V& ); //遍历
```

```
}; //BinNode
```



节点的插入与接入

```
template <typename T>
```

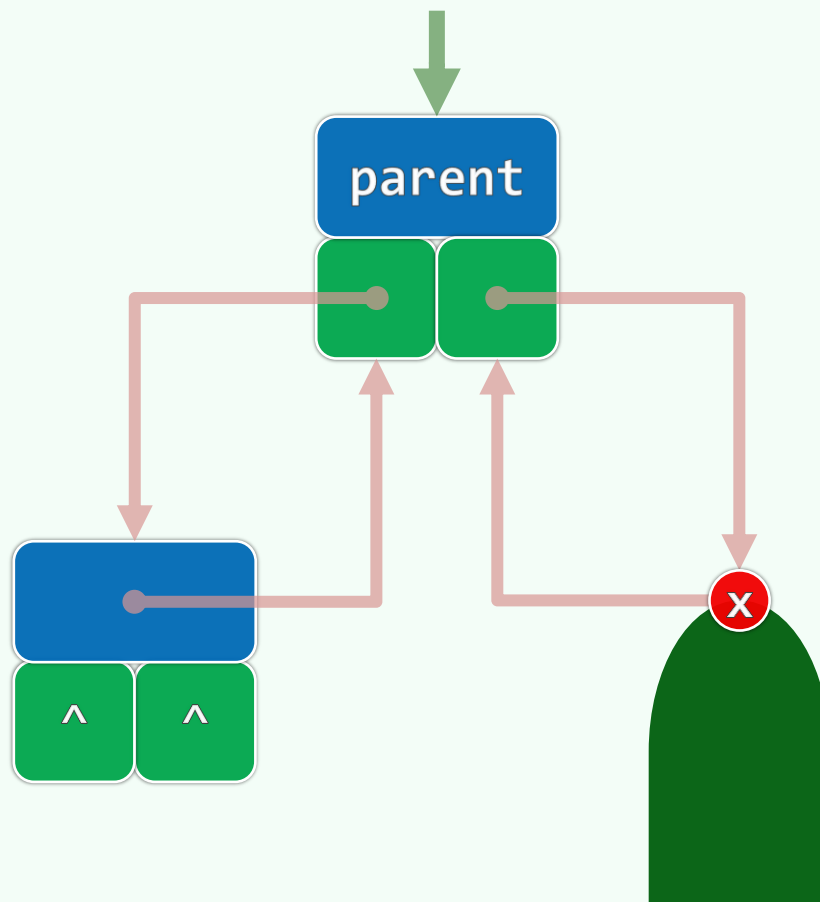
```
BNP<T> BinNode<T>::insertLc( const T & e )
```

```
{ return lc = new BinNode<T>( e, this ); }
```

```
template <typename T>
```

```
void BinNode<T>::attachRc( BNP<T> x )
```

```
{ rc = x; if ( x ) x->parent = this; }
```



高度的更新

```
constexpr int H_NULL = -1; //空的常规二叉树，高度为-1

#define Height(x) ( (x) ? (x)->height : H_NULL ) //简化描述，安全地处置边界情况

template <typename T> int BinNode<T>::updateHeight() //O(1)
{ return height = 1 + max( Height(lc), Height(rc) ); }

template <typename T> void BinNode<T>::updateHeightAbove() { //O(n = depth(x))
    for ( BNP<T> x = this; x; x = x->parent ) //可优化
        x->updateHeight();
} //updateHeightAbove
```

BinTree模板类

```
template <typename T> class BinTree {  
  
protected: Rank _size; BNP<T> _root;  
  
public: BNP<T> insert( T const& ); //插入根节点  
  
        BNP<T> insert( T const&, BNP<T> ); //插入左孩子  
  
        BNP<T> insert( BNP<T>, T const& ); //插入右孩子  
  
        BNP<T> attach( BinTree<T>, BNP<T> ); //接入左子树  
  
        BNP<T> attach( BNP<T>, BinTree<T> ); //接入右子树  
  
        BinTree<T> secede( BNP<T> ); //子树分离  
  
        Rank remove( BNP<T> ); //子树删除：需借助后序遍历，稍后再做讲解  
  
} //BinTree
```

节点的插入

BNP<T> BinTree<T>::insert(BNP<T> x, const T & e); //作为右孩子

BNP<T> BinTree<T>::insert(const T & e, BNP<T> x) { //作为左孩子

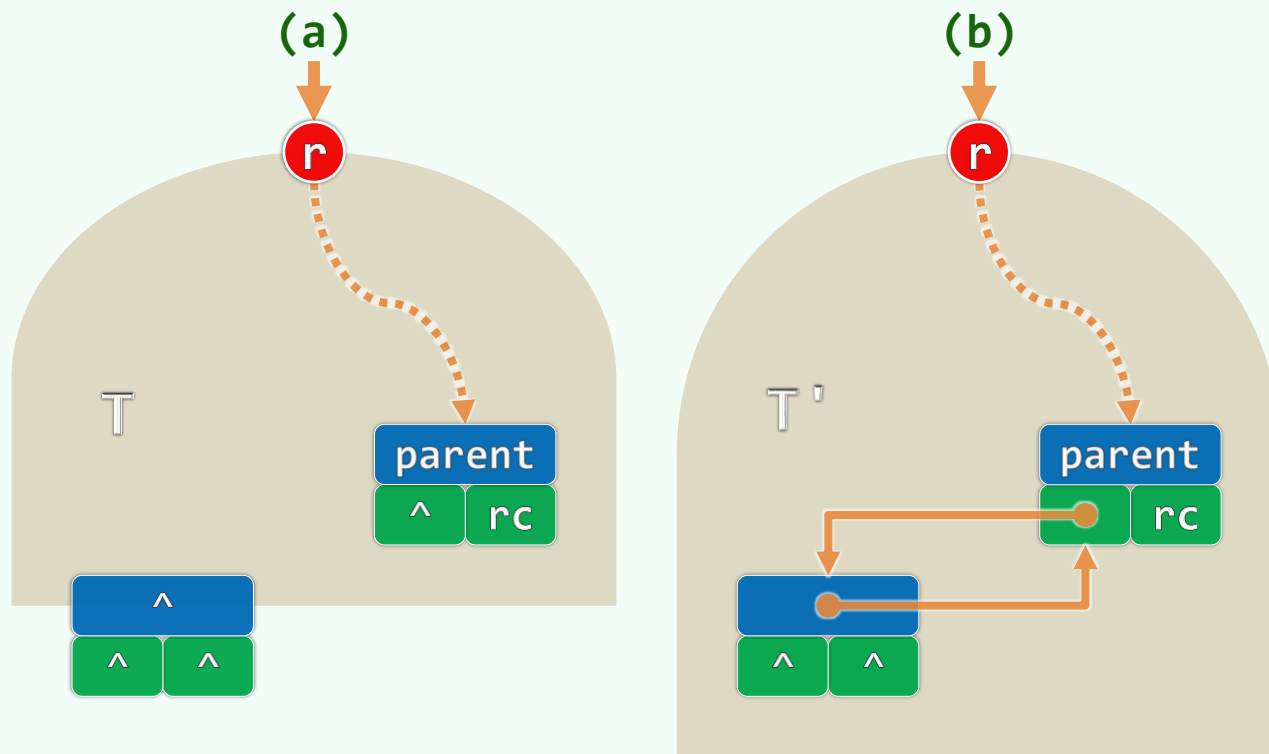
_size++;

x->insertLc(e);

x->updateHeightAbove();

return x->lc;

} //insert



子树的接入

BNP<T> BinTree<T>::attach(BinTree<T> S, BNP<T> x); //接入左子树

BNP<T> BinTree<T>::attach(BNP<T> x, BinTree<T> S) { //接入右子树

if (! (x->rc = S._root))

return NULL; //空树无需费事

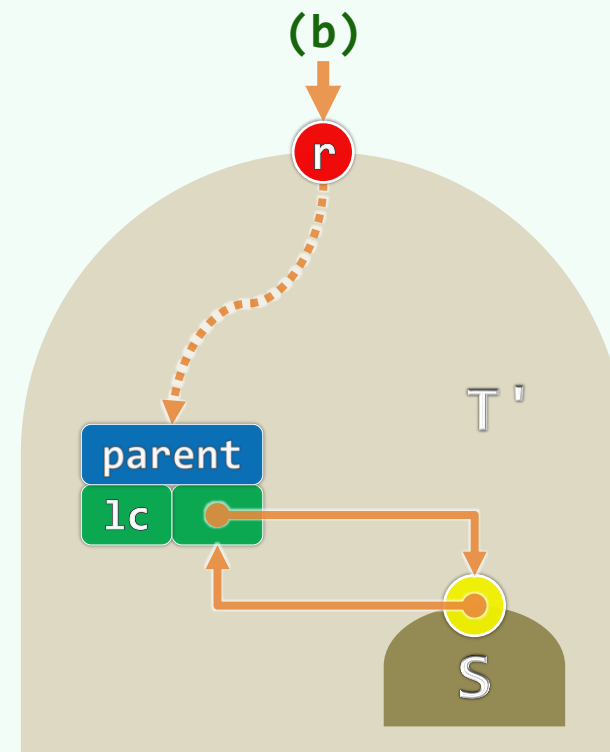
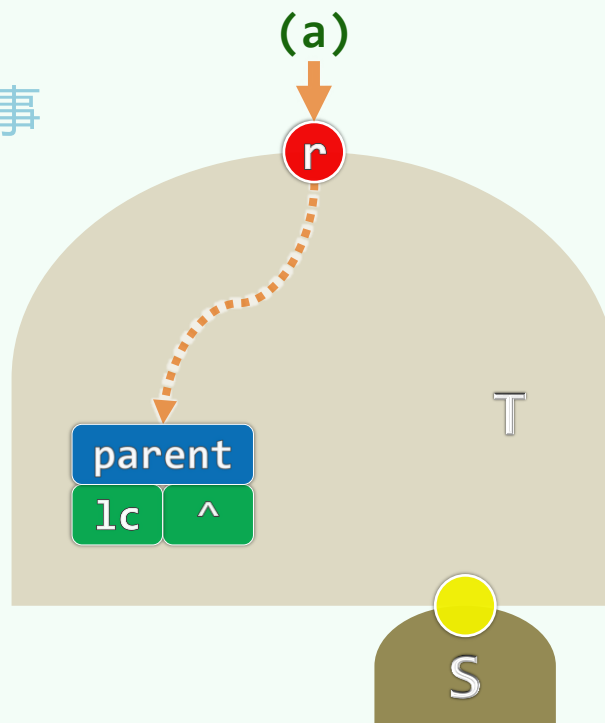
x->rc->parent = x;

_size += S._size;

x->updateHeightAbove();

S._root = NULL; S._size = 0;

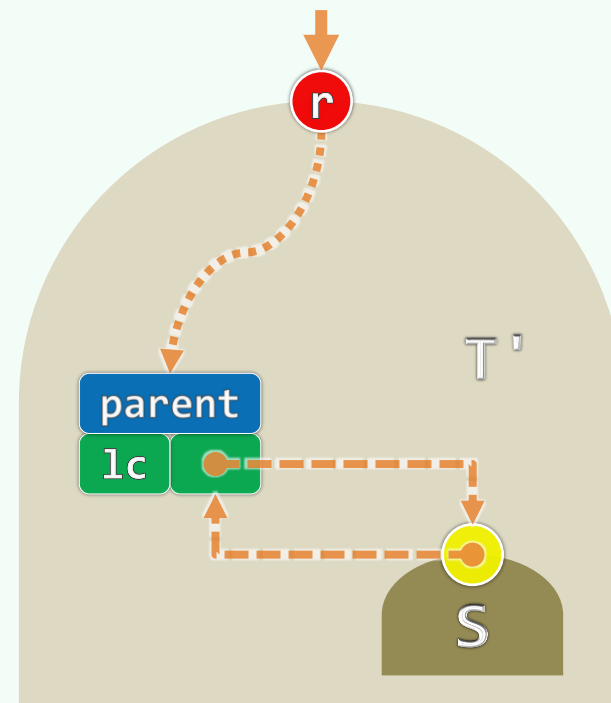
return x;



} //attach

子树的分离

```
template <typename T> BinTree<T> BinTree<T>::secede( BNP<T> x ) {  
  
    FromParentTo( x ) = NULL;  
  
    if ( x->parent ) x->parent->updateHeightAbove();  
  
    BinTree<T> S; //创建空树  
  
    S._root = x; x->parent = NULL; //新树以x为根  
  
    S._size = x->size(); _size -= S._size; //更新规模  
  
    return S; //返回封装后的子树  
  
} //secede
```



二叉树

遍历：应用

一桩事情的真相与奥妙，通常并不藏在最深的地方，有时就在表面。只不过，一般人视若无睹。要想成为一个好的算命先生，首先就必须学会观察...

这三家的人，在族谱之中寻查自己的谱系，却寻不着，因此算为不洁，不准供祭司的职任

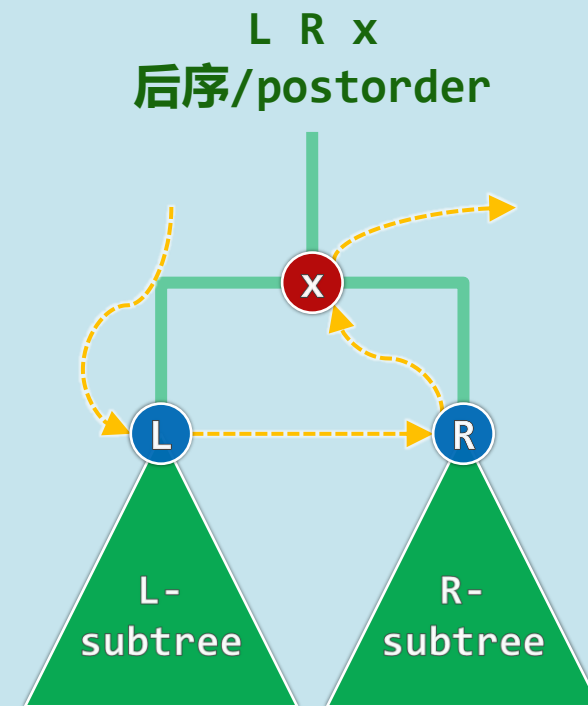
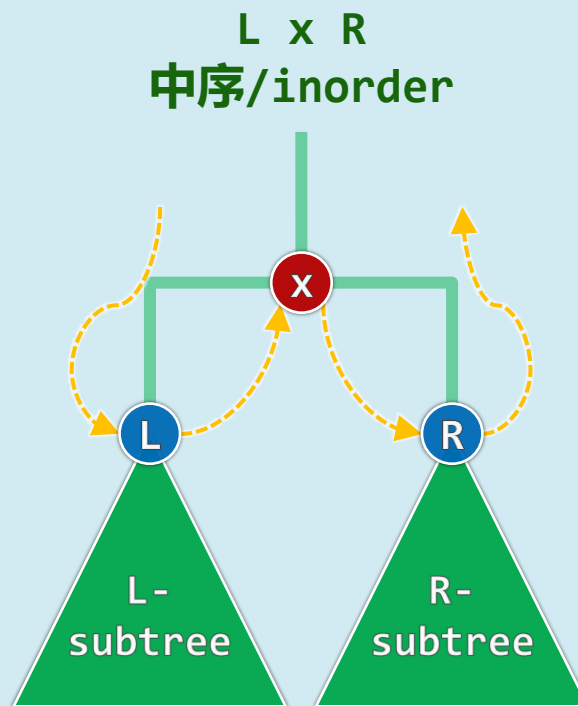
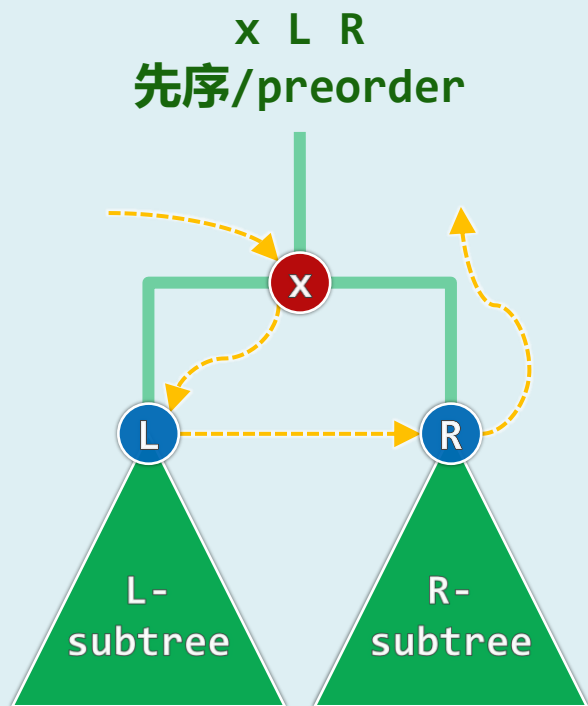
邓俊辉

deng@tsinghua.edu.cn

遍历

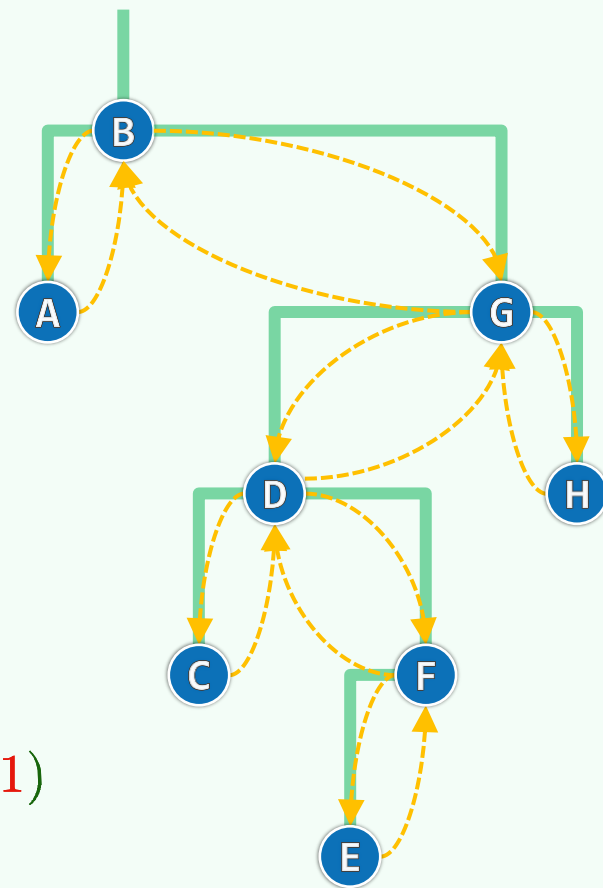
❖ 按照**某种次序**访问树中各节点，每个节点被访问**恰好一次**

❖ 结果 ~ 过程 ~ 次序 ~ 策略



递归实现

```
❖ template <typename T, typename V> void traverse( BNP<T> x, V& visit ) {  
    if ( ! x ) return;  
    visit( x->data ); // pre(x) = { B, A, G, D, C, F, E, H }  
    traverse( x->lc, visit );  
    visit( x->data ); // in(x) = { A, B, C, D, E, F, G, H }  
    traverse( x->rc, visit );  
    visit( x->data ); // post(x) = { A, C, E, F, D, H, G, B }  
} //traverse
```



❖ 假定visit()只需**常数**时间，则有： $T(n) = T(m) + T(n - m - 1) + O(1)$

对照Akra-Bazzi定理，属于 $0 < p < 1$ 的情形，解得： $T(n) = \Theta(n)$

先序遍历实例：统计规模

❖ 先序输出文件树的结构

```
c:\> tree.com c:\windows
```

❖ 统计子树规模，亦即当前节点的后代总数

```
template <typename T> Rank BinNode<T>::size() {
```

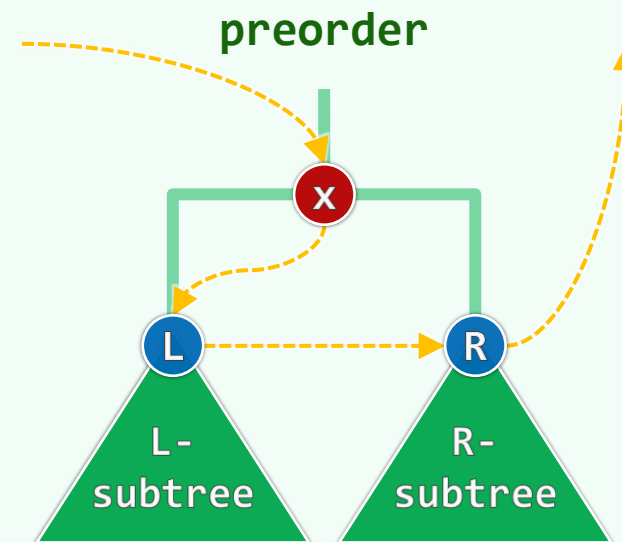
```
    Rank s = 1; //先计入本身
```

```
    if (lc) s += lc->size(); //递归计入左子树
```

```
    if (rc) s += rc->size(); //递归计入右子树
```

```
    return s; //也可改由中序、后序遍历得到
```

```
} //O(n=size): 懒惰策略，现用现算；也可改为勤奋策略，始终动态维护
```



先序遍历实例：深复制：基本

```
template <typename T> //p ~ Parent; t ~ source Tree
```

```
BNP<T> clone( BNP<T> p, BNP<T> t ) { //深复制
```

```
    if ( !t ) return NULL;
```

```
    BNP<T> x = new BinNode<T>( t->data, p, NULL, NULL, t->height, t->color );
```

```
    x->lchild = clone( x, t->lchild );
```

```
    x->rchild = clone( x, t->rchild );
```

```
    return x; //由t复制出x
```

```
} //clone
```

先序遍历实例：深复制：扩展

```
template <typename T> BinTree<T>::BinTree( BNP<T> t ) //复制子树
```

```
{ _root = clone( NULL, t ); _size = t->size(); }
```

```
template <typename T> BinTree<T>::BinTree( const BinTree<T>& t ) //复制全树
```

```
{ _root = clone<T>( NULL, t._root ); _size = t.size(); }
```

```
template <typename T> BinTree<T>& BinTree<T>::operator=( const BinTree<T>& t ) {
```

```
    if ( _root != t._root )
```

```
        { remove( _root ); _root = clone<T>( NULL, t._root ); _size = t.size(); }
```

```
    return *this;
```

```
} //通过重载 “operator=” 实现赋值
```

中序遍历实例：输出二叉树

❖ `template <typename T>`

```
void printBinTree( BNP<T> x, int depth ) {
```

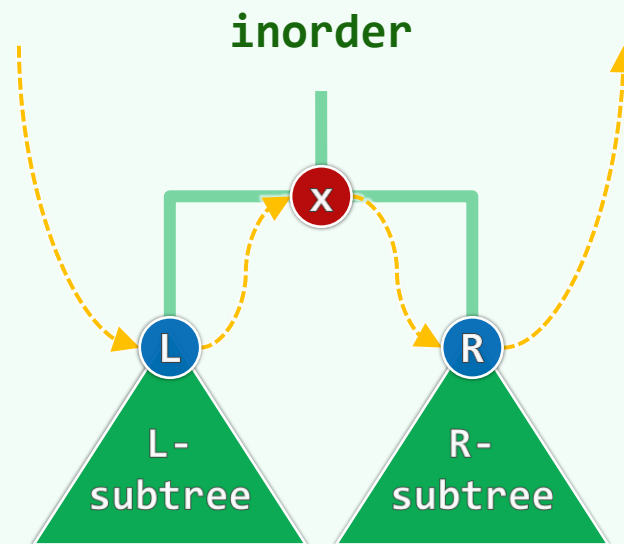
```
    if ( !x ) return;
```

```
    printBinTree( x->rc, depth + 1 ); //右子树 (在上)
```

```
    print( x ); //根节点居中
```

```
    printBinTree( x->lc, depth + 1 ); //左子树 (在下)
```

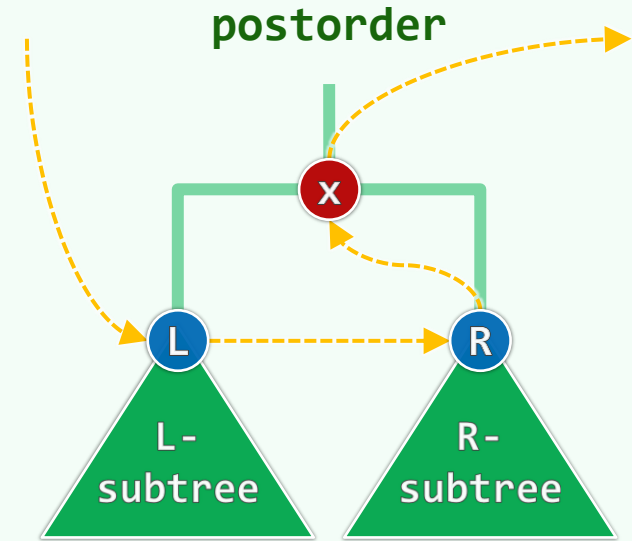
```
} //printBinTree
```



❖ 中序序列`in(x)`：全然不顾纵向的**深度**，只是依照横向的**齿序**逐个枚举

后序遍历实例：子树的删除与析构

```
template <typename T> Rank BinTree<T>::remove( BNP<T> x ) {  
    if ( !x ) return 0; FromParentTo( x ) = NULL;  
    if ( x->parent ) x->parent->updateHeightAbove();  
    Rank n = clean( x ); _size -= n; return n;  
} //remove  
  
template <typename T> Rank clean( BNP<T> x ) {  
    if ( !x ) return 0;  
    Rank n = 1 + clean( x->lc ) + clean( x->rc );  
    delete x; return n;  
} //clean
```

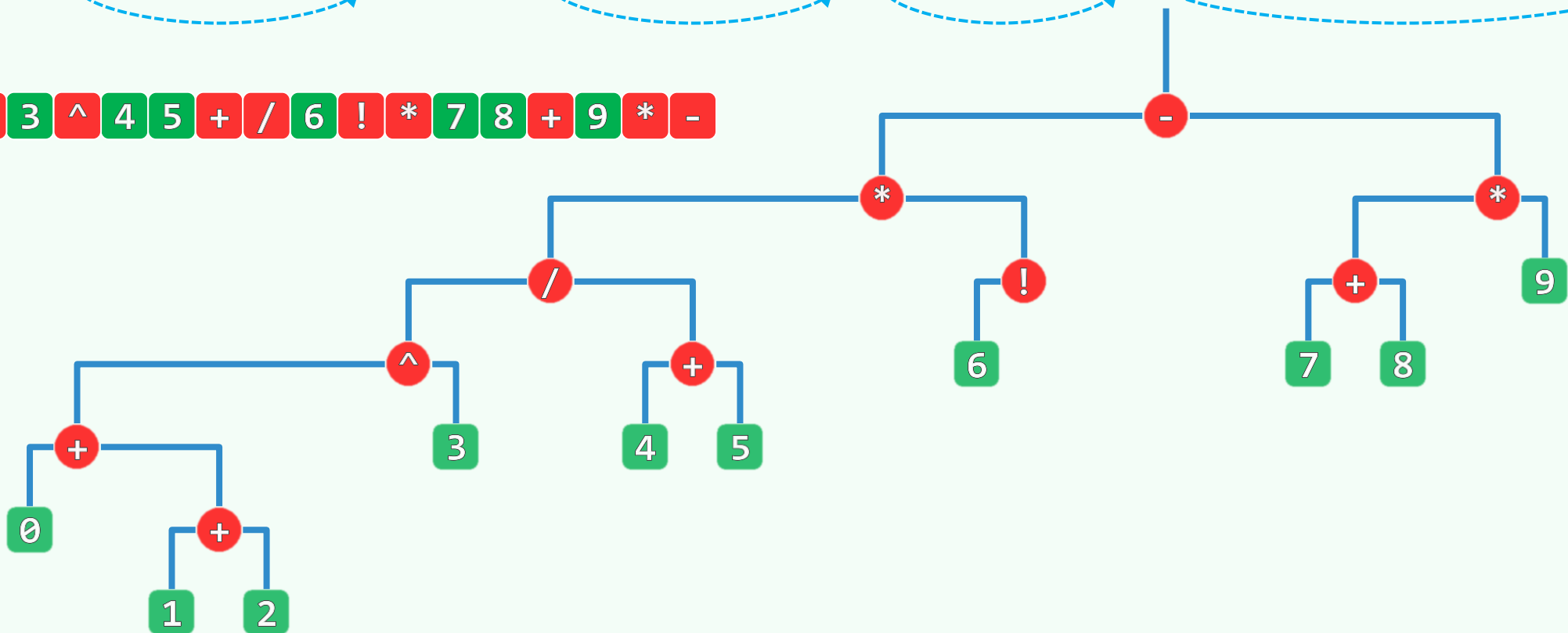


后序遍历实例：表达式树：后序遍历 = RPN

(0 + (1 + 2)) ^ 3 / (4 + 5) * 6 ! - (7 + 8) * 9

(((((0 + (1 + 2)) ^ 3) / (4 + 5)) * (6 !)) - ((7 + 8) * 9))

0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -



二叉树

遍历：迭代式先序

山中只见藤缠树，世上哪见树缠藤
青藤若是不缠树，枉过一春又一春

凡是过往，皆为序章

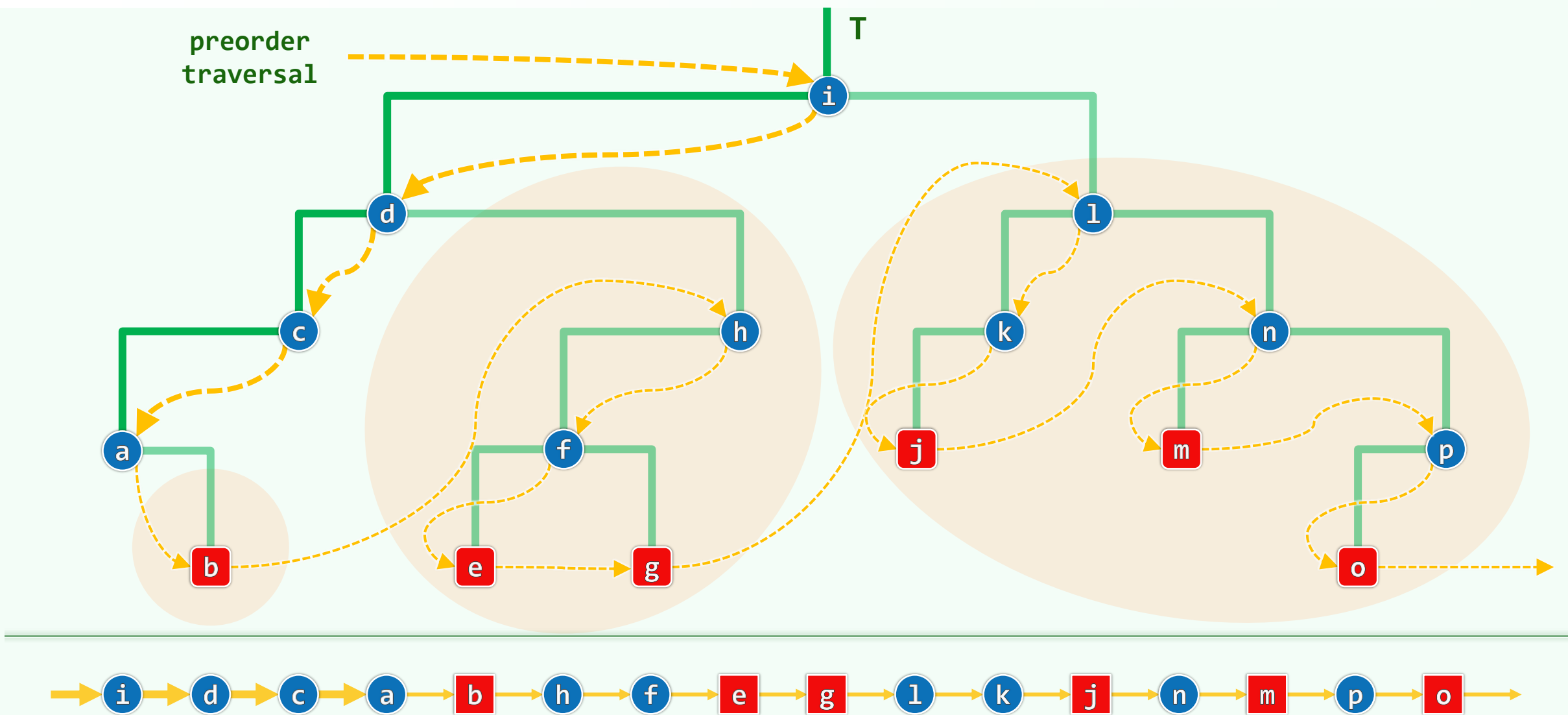
我是如來最小之弟

一桩事情的真相与奥妙，通常并不藏在最深的地方，有时就在表面。只不过，一般人视若无睹。要想成为一个好的算命先生，首先就必须学会观察...

邓俊辉

deng@tsinghua.edu.cn

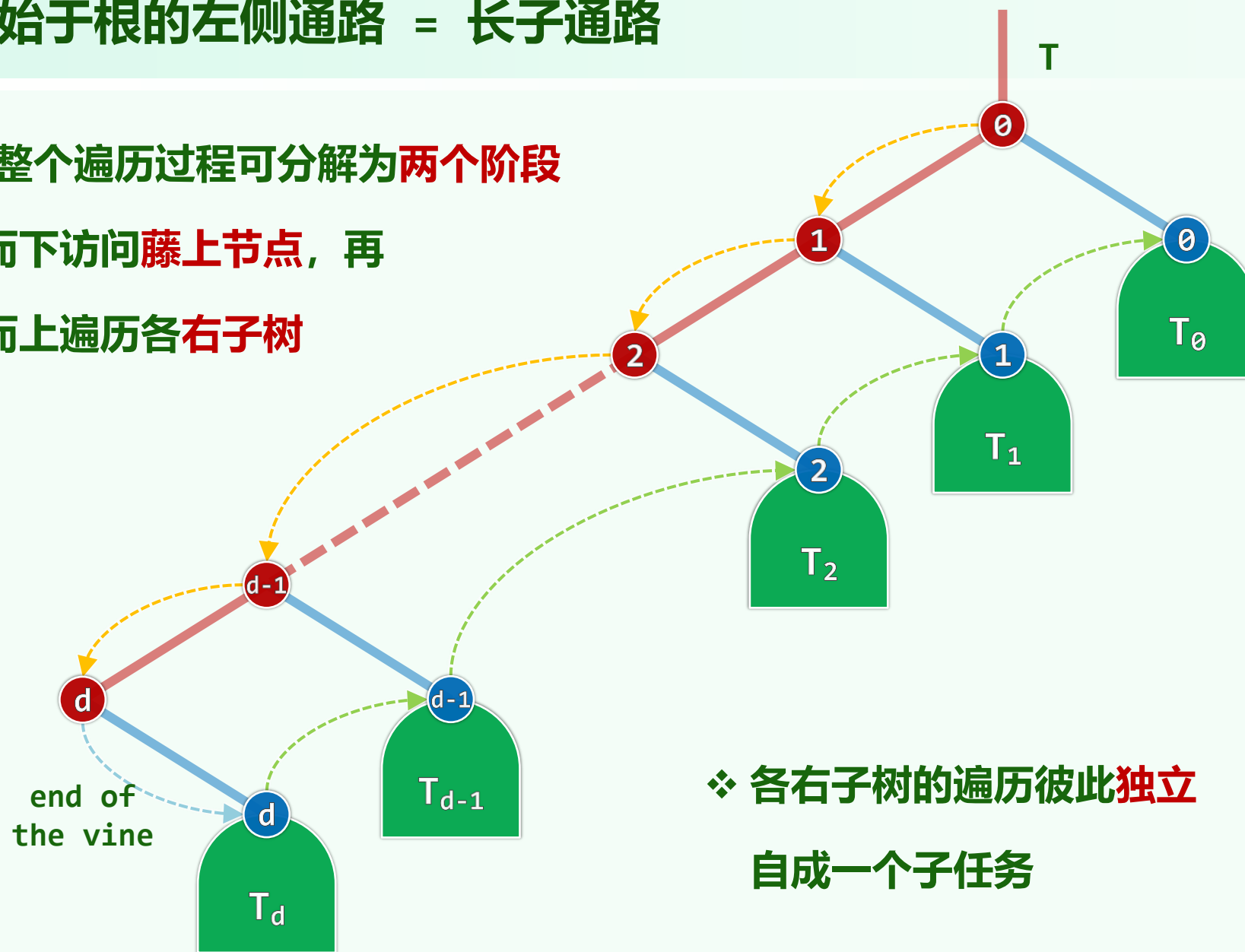
不依赖递归，如何实现先序遍历？效率如何？



藤 = 起始于根的左侧通路 = 长子通路

❖ 沿着藤，整个遍历过程可分解为两个阶段

- 自上而下访问藤上节点，再
- 自下而上遍历各右子树



❖ 各右子树的遍历彼此独立
自成一个子任务

$\text{visit}(v_0)$
 $\text{visit}(v_1)$
 $\dots$
 $\text{visit}(v_d)$
 $\text{preorder}(T_d)$
 $\dots$
 $\text{preorder}(T_1)$
 $\text{preorder}(T_0)$

算法

```
❖ template <typename T, typename V> void preorder1( BNP<T> x, V& visit ) {  
    Stack<BNP<T>> S; S.push( x ); //从当前节点出发  
    while ( !S.empty() ) //以右子树为单位, 依次访问每一条藤  
        for ( x = S.pop(); x; x = x->lc ) //自x出发, 沿着藤依次  
            { visit( x->data ); S.push( x->rc ); } //访问各节点, 并令右孩子 (或空) 入栈  
} //preorder1
```

❖ while循环 $\mathcal{O}(n)$, for循环 $\mathcal{O}(n)$, 岂不是... $\mathcal{O}(n^2)$?

其实, 总共不过 $\mathcal{O}(n)$; 摊还至每个节点, 不过 $\mathcal{O}(1)$!

为此, 只需数数栈操作的累计次数...aggregate...

性能

```
❖ template <typename T, typename V> void preorder1( BNP<T> x, V& visit ) {  
    Stack<BNP<T>> S; S.push( x ); //从当前节点出发  
    while ( !S.empty() ) //以右子树为单位, 依次访问每一条藤  
        for ( x = S.pop(); x; x = x->lc ) //自x出发, 沿着藤依次  
            { visit( x->data ); S.push( x->rc ); } //访问各节点, 并令右孩子 (或空) 入栈  
} //preorder1
```

❖ 唯有右孩子才能 (也必须) 进栈, #push = #rc = $n+1$

#pop = #push = $n+1$; #empty = #pop + 1 = $n+2$

❖ 空间 = 显式的栈S: 虽渐近地仍是 $O(h)$, 但每个元素仅4个字节, 常系数已降至极限

二叉树

遍历：迭代式中序

貴省女子固多，不過擇其緊要者錄之。下邊二廚則
又次之。餘者庸常之輩，則無冊可錄矣

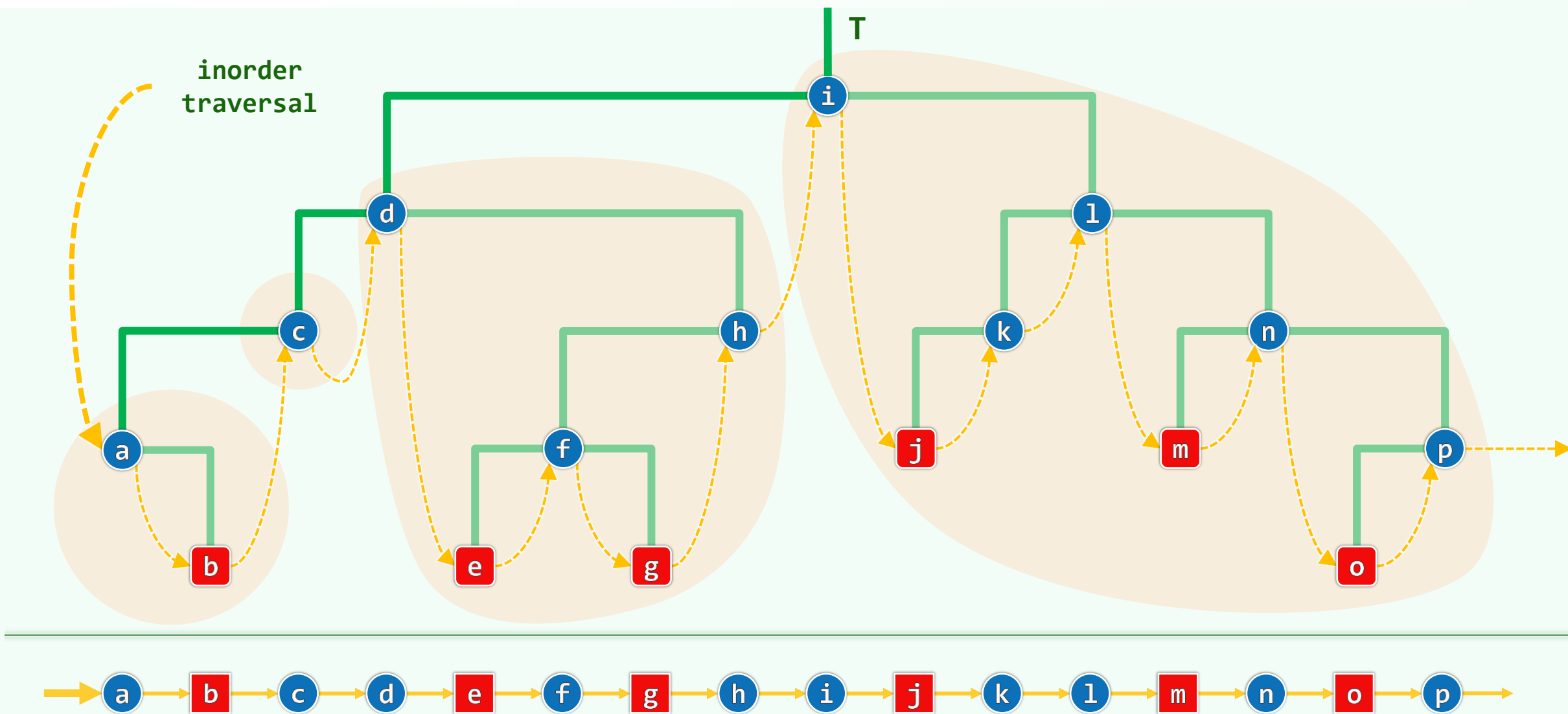
Although we've come to the end of the road
Still I can't let you go

他从哪条路来，必从哪条路回去，必不得来到这城

邓俊辉

deng@tsinghua.edu.cn

不依赖递归，如何实现中序遍历？效率如何？



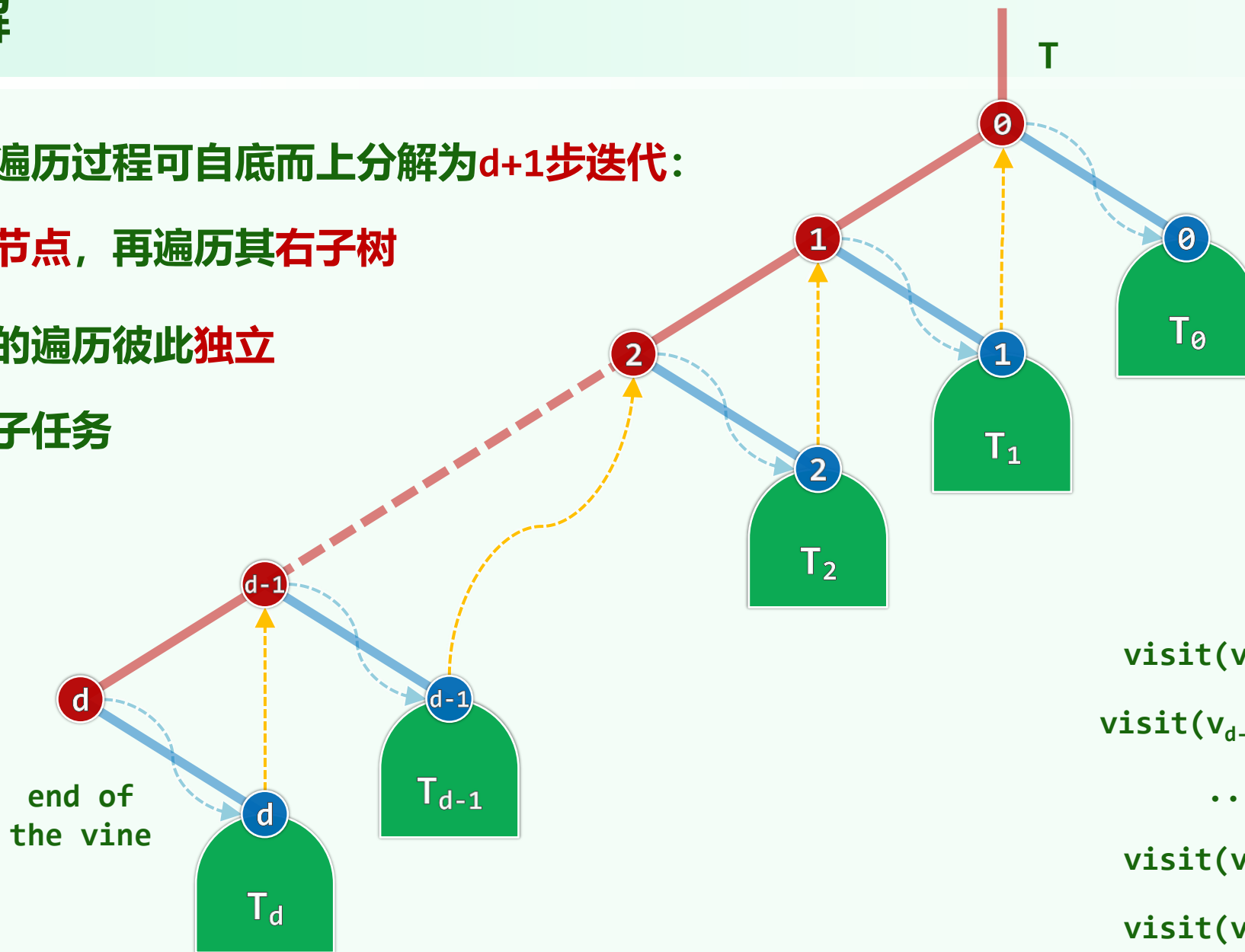
沿藤分解

❖ 沿着藤，遍历过程可自底而上分解为 $d+1$ 步迭代：

访问藤上节点，再遍历其右子树

❖ 各右子树的遍历彼此独立

自成一个子任务



```
visit( $v_d$ ), inorder( $T_d$ )  
visit( $v_{d-1}$ ), inorder( $T_{d-1}$ )  
.....  
visit( $v_1$ ), inorder( $T_1$ )  
visit( $v_0$ ), inorder( $T_0$ )
```


算法

```
template <typename T, typename V> void inorder1( BNP<T> x, V& visit ) {  
  
    Stack<BNP<T>> S; //辅助栈  
  
    while ( x || !S.empty() ) { //只要x非NULL, 或S仍非空  
  
        for ( ; x; x = x->lc ) S.push( x ); //藤上节点便自顶而下入栈  
  
        visit( (x = S.pop())->data ); //取出并访问末节点, 再  
  
        x = x->rc; //转向右子树  
  
    } //while  
  
} //inorder1
```

正确性：数学归纳

❖ 假设算法可正确地遍历所有规模不超过 n 的二叉树

❖ 考查任一规模为 $n+1$ 的二叉树，其根记作 x

将藤之终点记作 t ， t 之右孩子记作 y

❖ 整个算法分为五个阶段：

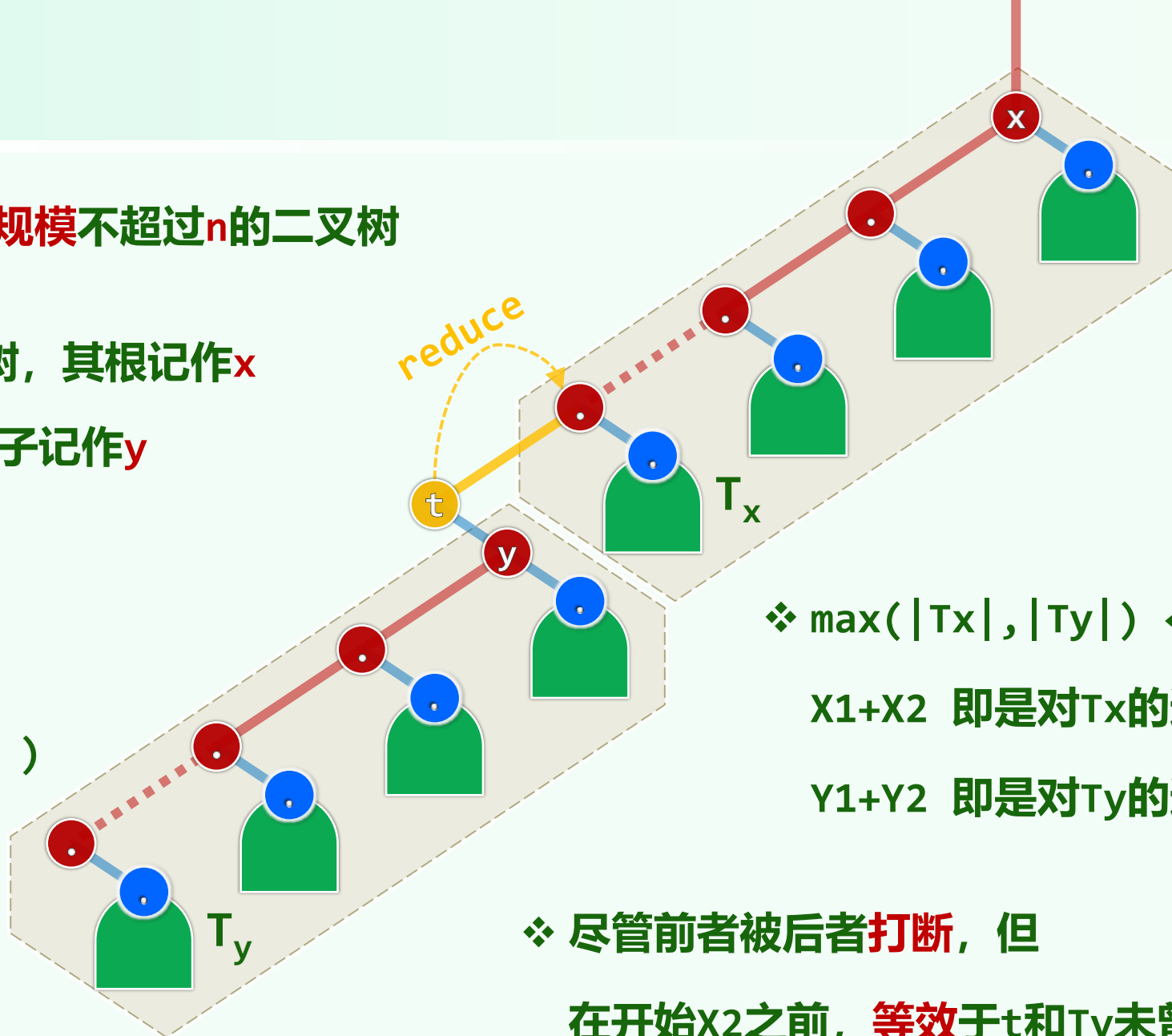
X1: for(x)..push(x)

T: visit($t = S.pop()$)

Y1: for(y)..push(y)

Y2: inorder(T_y)

X2: inorder(T_x)



❖ $\max(|T_x|, |T_y|) \leq n$ ，故

X1+X2 即是对 T_x 的遍历

Y1+Y2 即是对 T_y 的遍历

❖ 尽管前者被后者打断，但

在开始X2之前，等效于 t 和 T_y 未曾存在过

性能

- ❖ 空间：与先序遍历一样，最大递归深度有**常数**倍的增加，可吞吐的树的最大规模有**指数**量级的增加
- ❖ 时间：依然不是 $O(n^2)$ ；均摊分析可知，总体不过 $O(n)$...
...accounting...事先，为每个节点派发**3**枚金币
- ❖ 时间消耗的主体部分仍是**栈**操作，每一次都有节点来**支付**报酬
 - **push**(x)：不会有节点重复入栈，故可以就由x支付
 - $x = \text{pop}(x)$ ：也不会有节点重复出栈，故亦可由x支付
 - **empty**()：仅出现在while的逻辑判断中

除了最后一次，每次执行都对应于某棵**空**的右子树

虽然空树没钱，但它的父亲却有...子债...父还

05-C4

二叉树

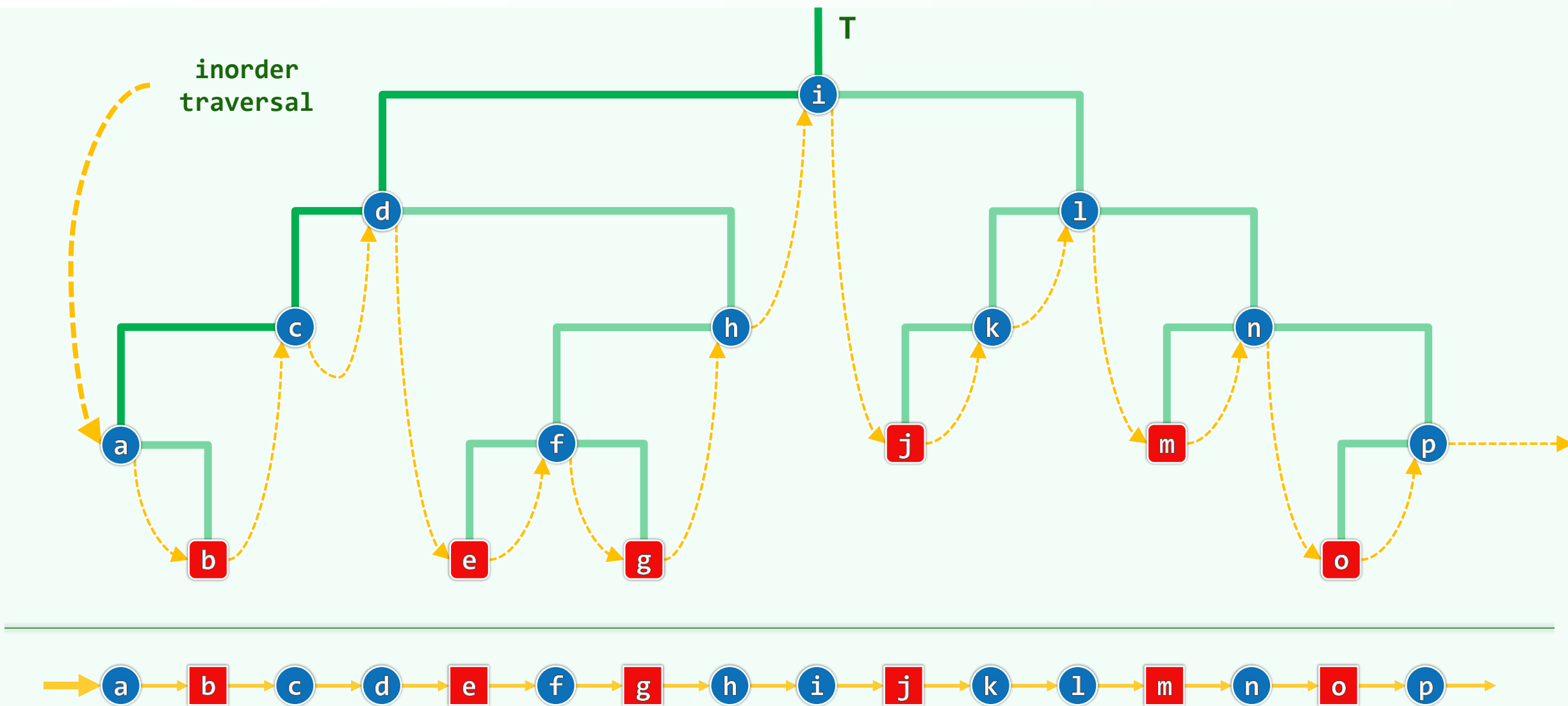
遍历：中序后继

邓俊辉

deng@tsinghua.edu.cn

No two leaves are exactly alike. - Leibniz

简明遍历: `for ( x = first(x); x; x = x->succ() )`



直接后继：右子树中藤之终点

```
❖ template <typename T> BNP<T> BinNode<T>::succ()  
    { return rc ? eov(rc) : lla(this); }
```

无非两种情况，取决于是否有**右子树**

```
❖ template <typename T> //End Of Vine
```

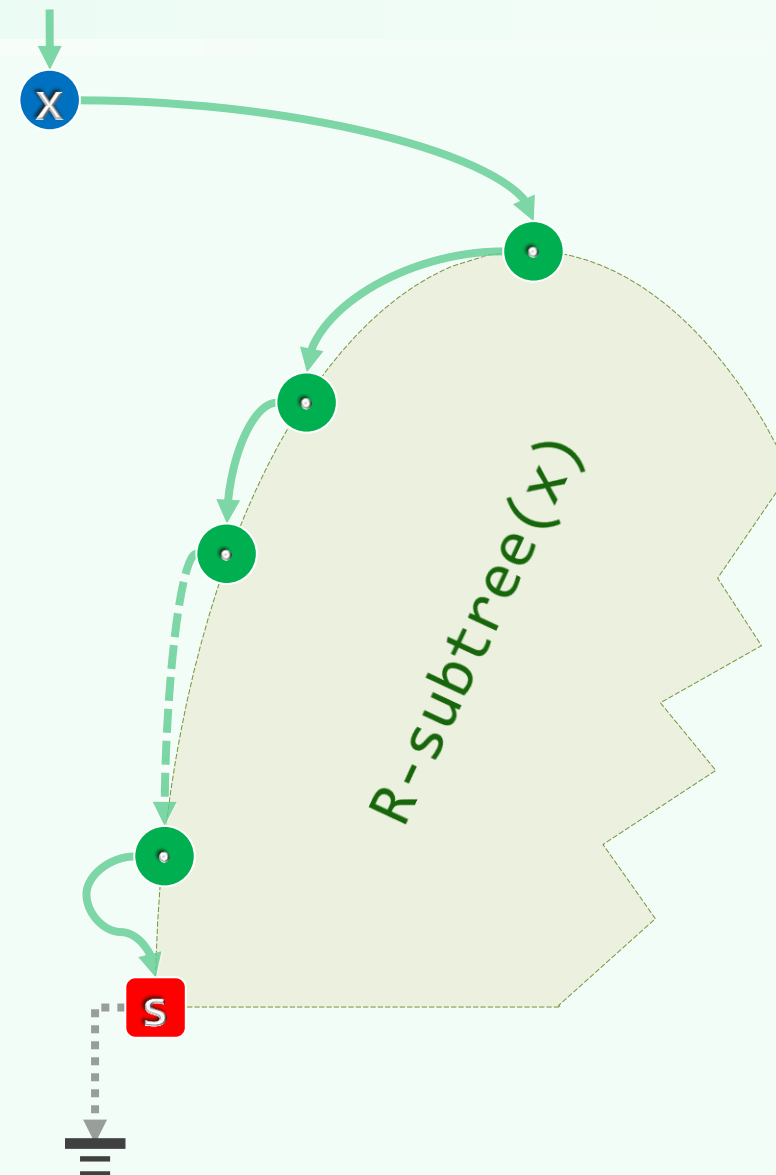
```
BNP<T> eov( BNP<T> x ) {
```

```
    if ( x ) //从x出发
```

```
        for ( ; x->lc; x = x->lc ); //顺藤下行
```

```
    return x; //直至终点
```

```
} //eov
```



直接后继：最低的左祖先

❖ 没有右子树时，逆向思维：哪个节点以x为前驱？

对称地，搜索方向由下行转为上行

❖ `template <typename T> //Lowest Left Ancestor`

```
BNP<T> llb( BNP<T> x ) {
```

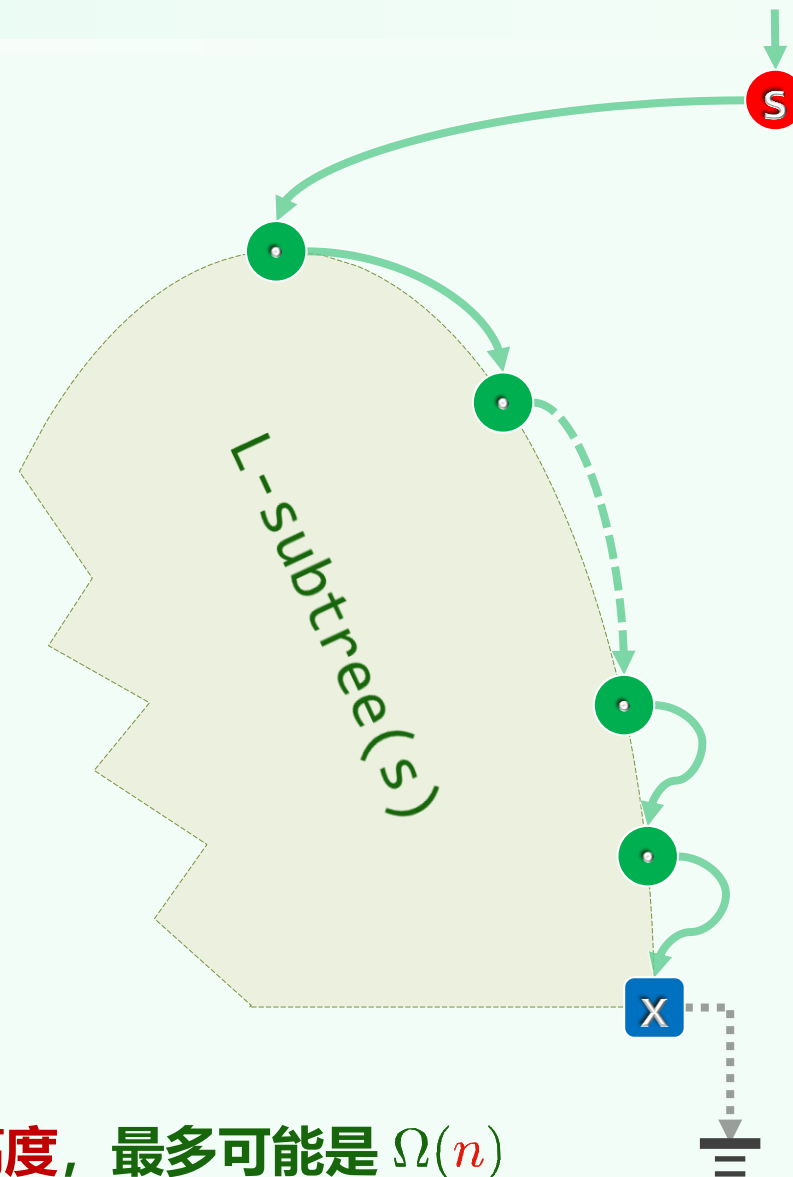
```
    while ( x->parent && x == x->parent->rc )
```

```
        x = x->parent; //右父亲的...右父亲的
```

```
    return x->parent; //左父亲 (可能是NULL)
```

```
} //llb
```

❖ 运行时间分别为 $\text{height}(x)$ 与 $\text{depth}(x)$ ：均不超过全树的高度，最多可能是 $\Omega(n)$



规范简约的遍历：算法

```
❖ template <typename T, typename V> void inorder2( BNP<T> x, V& visit ) {  
    for ( x = first(x); x; x = x->succ() ) //依次  
        visit( x->data ); //枚举所有节点  
} //inorder2
```

❖ 任何遍历都可套用这个**范式**，只要写好各自的first()和succ()

就中序序列而言，首节点就是EOV: auto first = [](auto x) { return eov(x); };

❖ 最坏情况下，**单次**succ()就需 $\Omega(n)$ 时间，难道总体是 $\Omega(n^2)$ ？

形式上的极致简约，是以计算**效率**的牺牲为代价？

❖ 我们来统计一下，所有的eov()和succ()中，x**改变位置**（被**赋值**）的次数，记作 $T(n)$

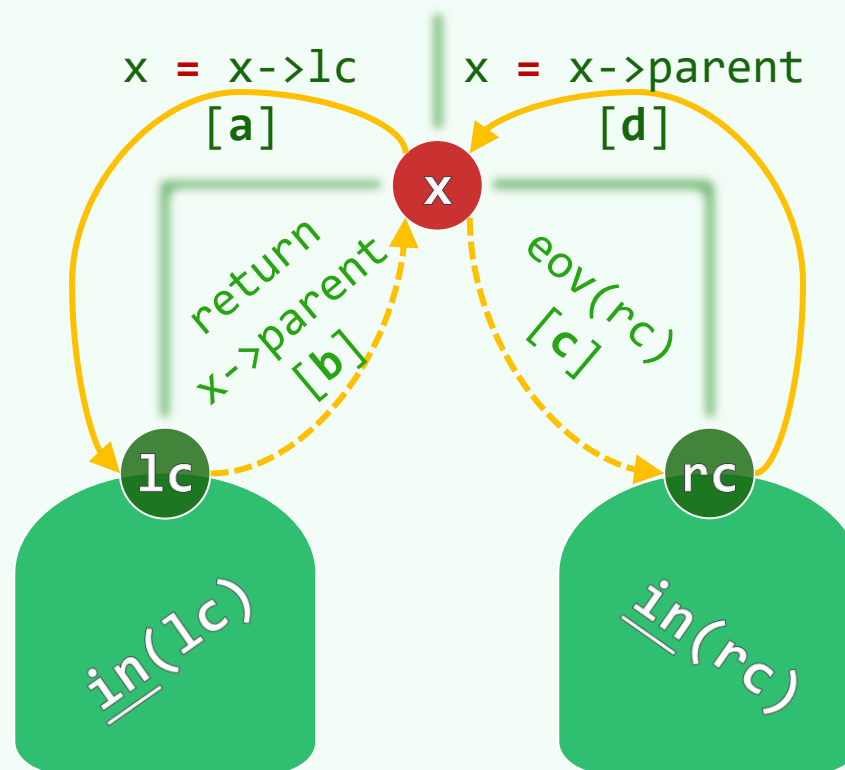
规范简约的遍历：性能

❖ 验证: $T(1) = 0$; 以下导出其递推方程

❖ 立足于树根 x , 整个遍历过程可以等效地分为七个阶段:

- ✓ **[a]** 经 $\text{first}(x)$ 的第一次赋值转至 lc
 - (等效于递归地) 完成 $\text{inorder}(lc)$
- ✗ **[b]** 经 $\text{l1a}(x \rightarrow \text{pred})$ 的最后一步返回 x
- ✗ $\text{visit}(x)$ 访问 x
- ✗ **[c]** (为查找 $\text{eov}(rc)$ 而) 直接转入 rc
 - (等效于递归地) 完成 $\text{inorder}(rc)$
- ✓ **[d]** 经最后一次 $\text{l1a}()$ 的最后一次赋值返回 x

❖ 故有: $T(n) = T(m) + T(n - m - 1) + 2 = 1 \cdot n - 1 = \mathcal{O}(n)$



二叉树

遍历：迭代式后序

很久以来
我就渴望升起
长长的，象绿色植物
去缠绕黄昏的光线

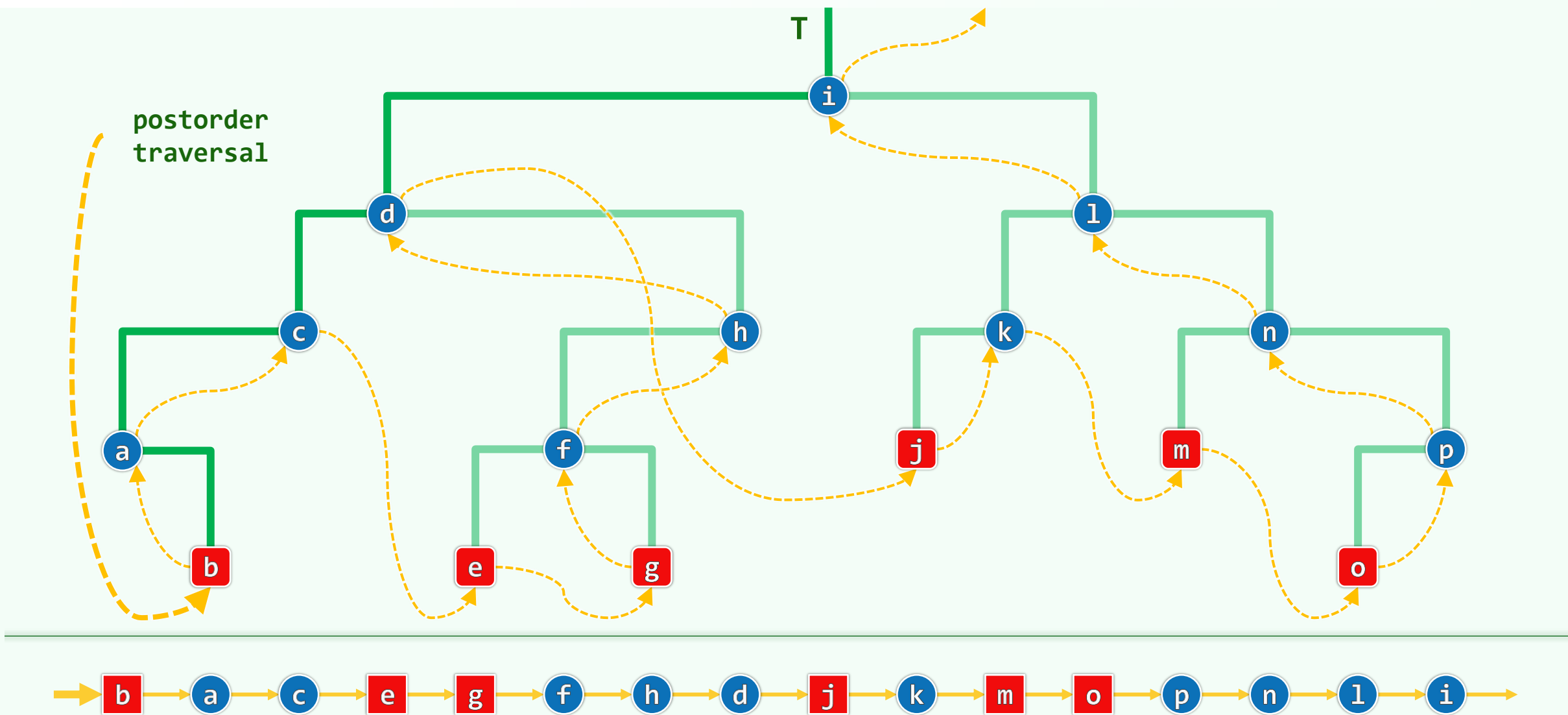
大宗百世不遷，小宗五世則遷

此各司中皆貯的是普天之下所有的女子過去未來的簿冊，
爾凡眼塵軀，未便先知的

邓俊辉

deng@tsinghua.edu.cn

不依赖递归，如何实现后序遍历？效率如何？



沿藤分解

❖ 思路不变，还是
从根出发，梳理出一条藤

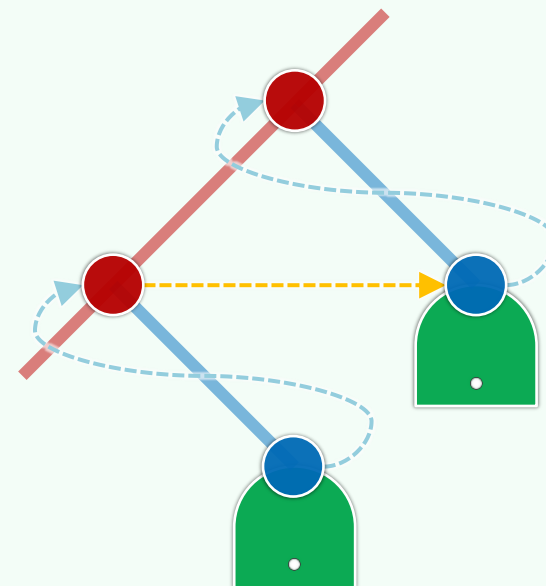
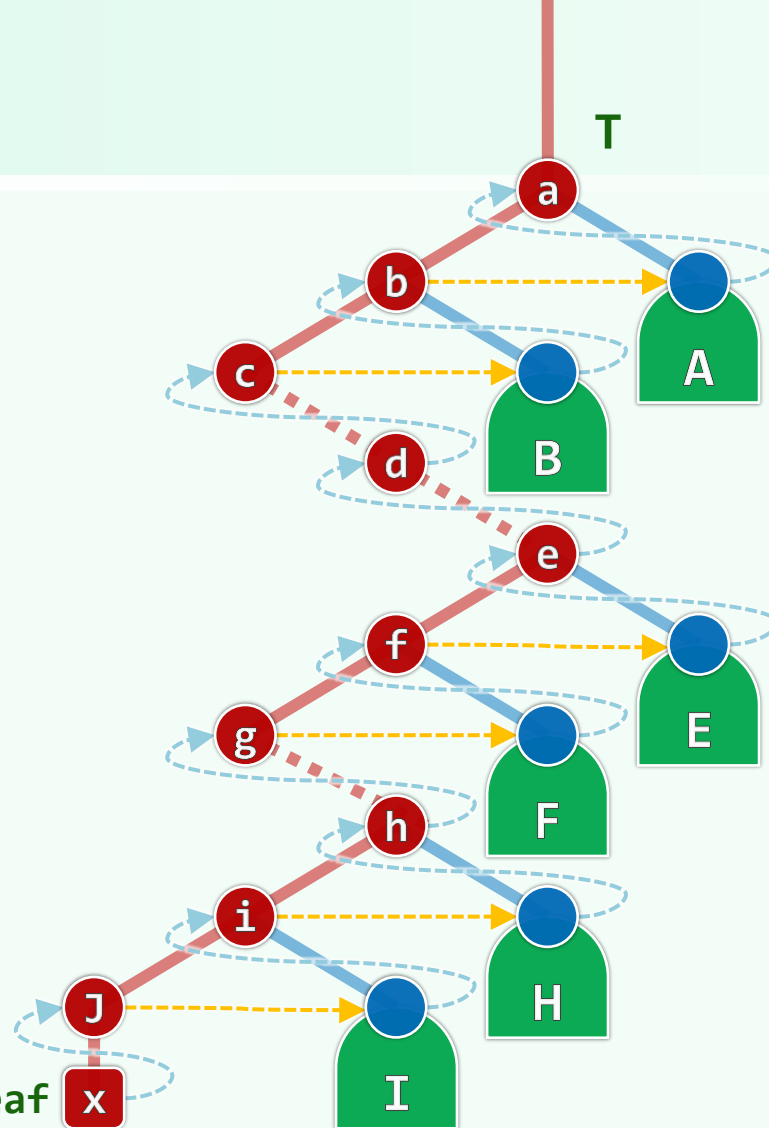
❖ 稍显复杂，原理相同

- 尽可能向左
- 实不得已才向右
- 直至全树中

最靠左的叶节点

亦即遍历的起点

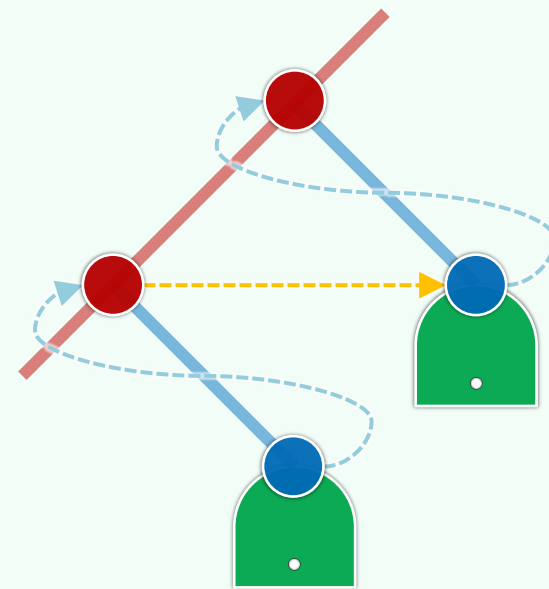
leftmost leaf



❖ 如此，遍历序列可分解为： $\underline{post}(T) = x; j, \underline{post}(J); \dots; c, \underline{post}(C); b, \underline{post}(B); a, \underline{post}(A)$

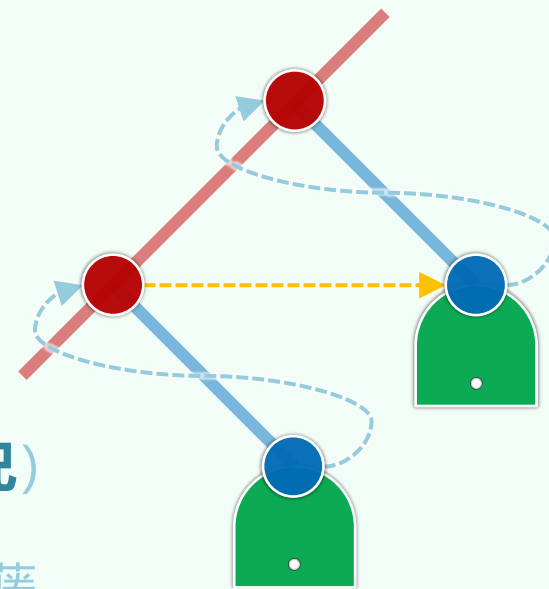
序曲

```
template <typename T>
void gotoLeftmostLeaf( Stack <BNP<T>>& S ) {
    for ( BNP<T> x = S.top(); x; ) //自顶而下
        if ( x->rc ) S.push( x->rc ); //右孩子优先入栈
        if ( x->lc ) //若有左孩子, 则优先左转
            { S.push( x->lc ); x = x->lc; }
        else x = x->rc; //实不得已, 才向右
    } //for
} //gotoLeftmostLeaf
```



全曲

```
template <typename T, typename V>
void postorder1( BNP<T> x, V& visit ) {
    Stack<BNP<T>> S;  if ( x ) S.push( x ); //辅助栈
    while ( ! S.empty() ) { //x始终为当前节点
        if ( S.top() != x->parent ) //若栈顶非x之父（而为右兄）
            gotoLeftmostLeaf( S ); //则在右兄子树中理出对应的藤
        x = S.pop(); visit( x->data ); //弹出并访问栈顶（即前一节点之后继）
    } //while
} //postorder1
```



05-C6

二叉树

遍历：层次

真君曰：“昔呂洞賓居廬山而成仙，鬼谷子居雲夢而得道，今或無此吉地麼？”璞曰：“有，但當遍歷耳。”

我们注定是扎根于前半生的，即使后半生充满了强烈的和令人感动的经历

邓俊辉

deng@tsinghua.edu.cn

算法实现

```
template <typename T, typename VST>

void BinNode<T>::levelorder( VST & visit ) { //二叉树层次遍历

    Queue< BNP<T> > Q; Q.enqueue( this ); //引入辅助队列，根节点入队

    while ( ! Q.empty() ) { //在队列再次变空之前，反复迭代

        BNP<T> x = Q.dequeue(); visit( x->data ); //取出队首节点并随即访问

        if ( x->lc ) Q.enqueue( x->lc ); //左孩子入队

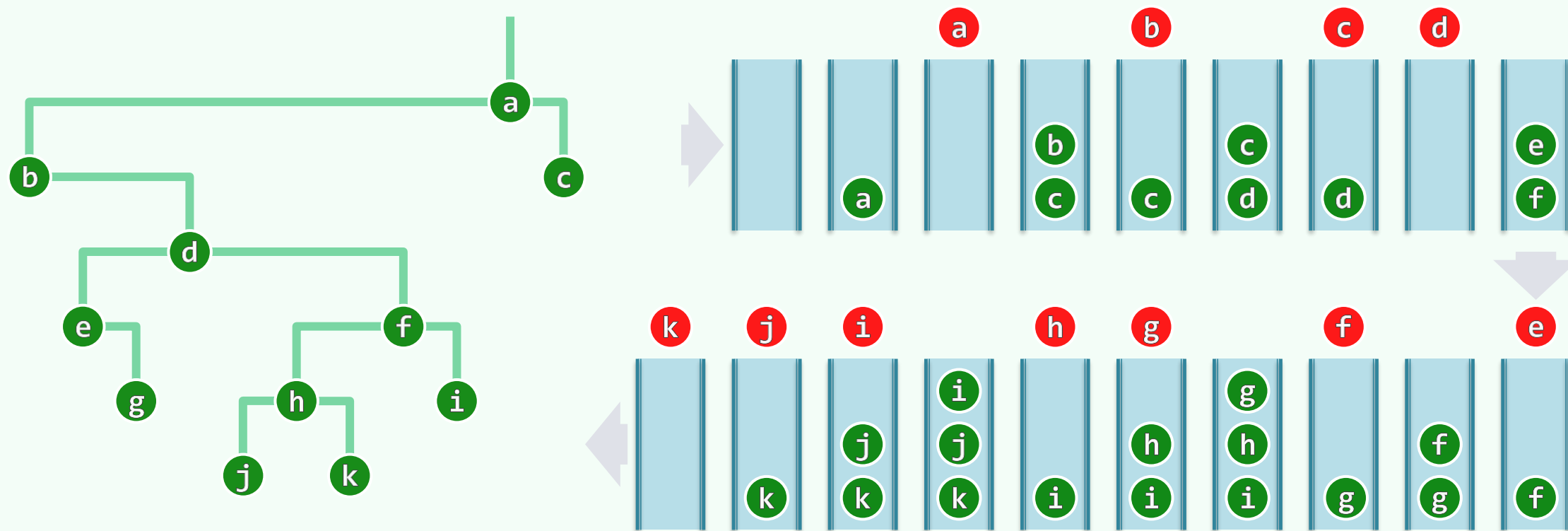
        if ( x->rc ) Q.enqueue( x->rc ); //右孩子入队

    } //while

} //levelorder
```


实例与性能

- ❖ 每个节点或早或迟都会入队、出队并接受访问，各自消耗 $\mathcal{O}(1)$ 时间，故总共 $\mathcal{O}(n)$
- ❖ 辅助队列的规模，始终不超过 $\lceil n/2 \rceil$ ，空间复杂度 $\mathcal{O}(n)$



完全二叉树

❖ 辅助队列的规模真能达到 $\lceil n/2 \rceil$ ，而且
最晚入队的两个节点中有一个是左孩子 //定义

❖ $2^h \leq n < 2^{h+1}$, $h = \lfloor \log_2 n \rfloor = \mathcal{O}(\log n)$

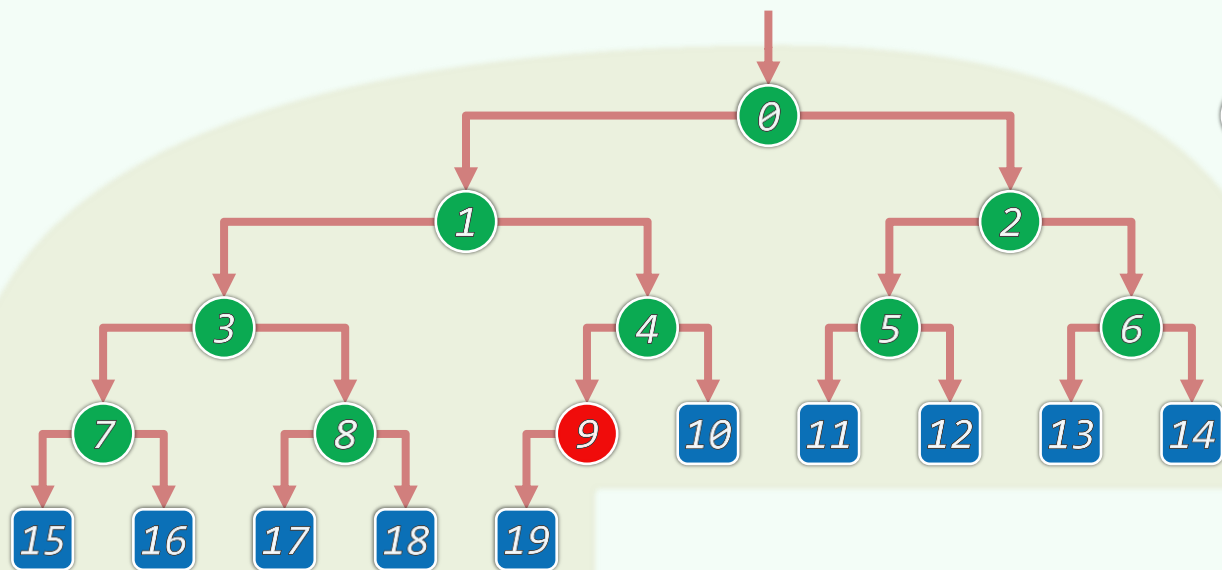
同规模的二叉树中，高度最小

❖ 将所有节点按层次遍历次序存入向量

- 长度为 $\lfloor n/2 \rfloor$ 的前缀都是内部节点

- 长度为 $\lceil n/2 \rceil$ 的后缀都是叶节点

❖ 更重要的是，在父与子、兄与弟之间
依据秩，有着简明的对应关系



0

1 2

3 4 5 6

7 8 9 10 11 12 13 14

15 16 17 18 19

$$\text{rank}(\text{parent}(k)) = \lfloor (k - 1) / 2 \rfloor$$

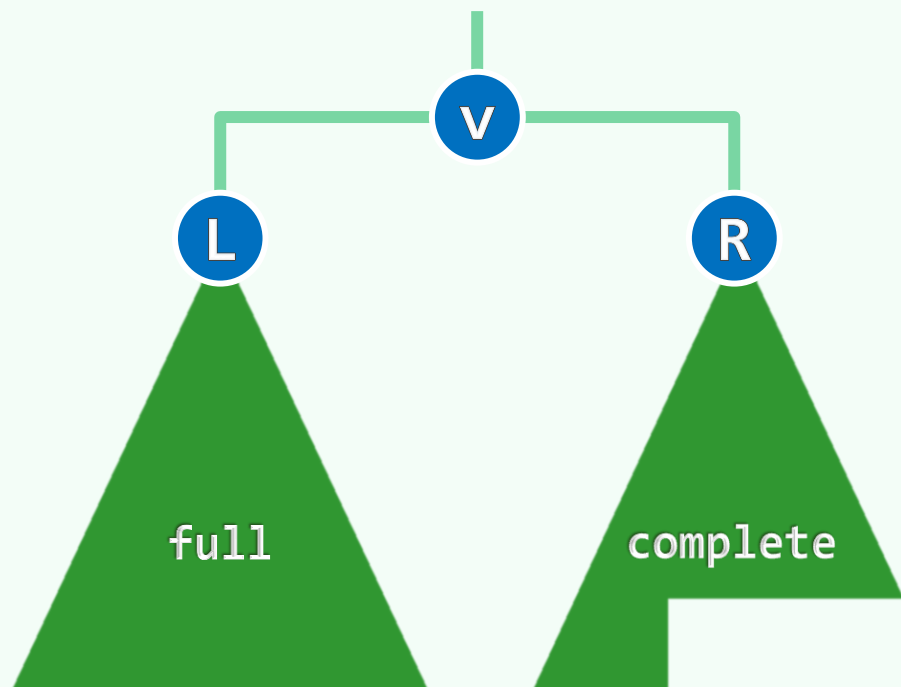
$$\text{rank}(\text{lc}(k)) = 2 \cdot k + 1$$

$$\text{rank}(\text{rc}(k)) = 2 \cdot k + 2$$

满树

❖ full binary tree:

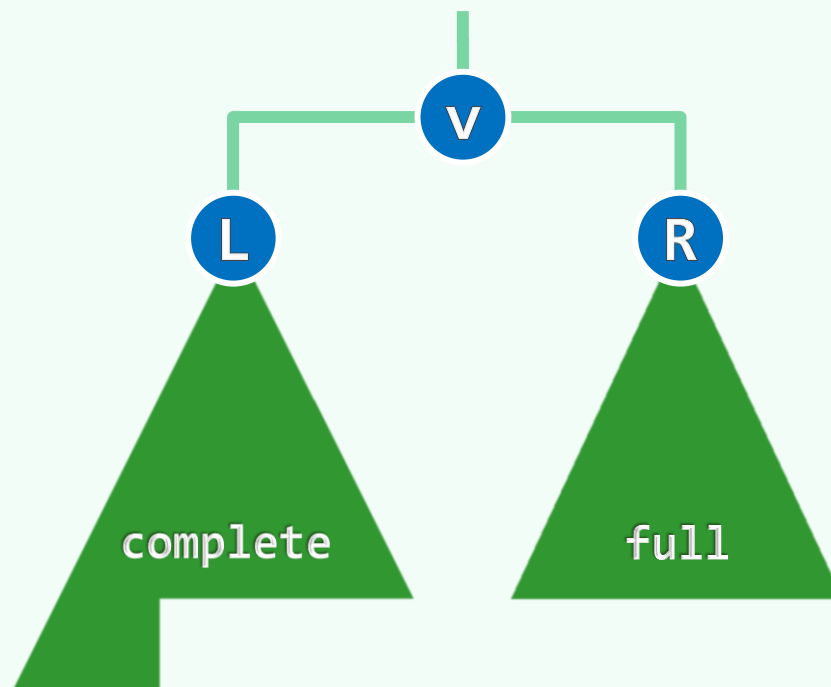
所有叶节点**深度一致**的完全二叉树



左满，右完全

❖ 在高度同为 h 的所有二叉树中

满树的**规模最大**: $n = 2^{h+1} - 1$



左完全，右满

05-C7

二叉树

遍历：重建

No matter where they take us,
We'll find our own way back.

邓俊辉

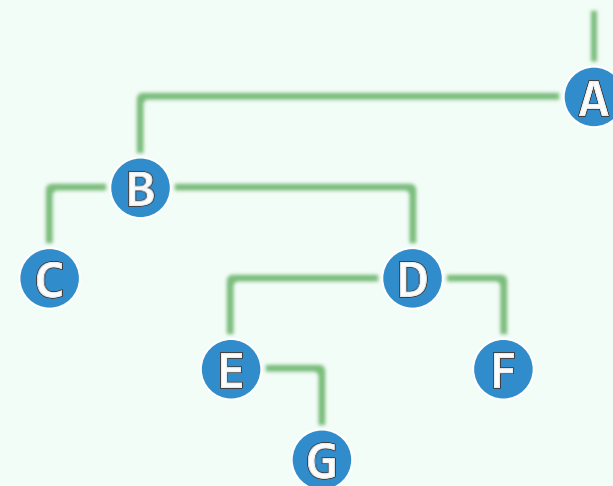
deng@tsinghua.edu.cn

重建

❖ rebuild: 由先序、中序、后序、层次等遍历序列，确定一棵二叉树的完整结构

❖ 以上任何序列，都不足以独自完成重建

<i>Level</i>	ab	ab	
<i>pre</i>	ab	ab	
<i>post</i>	ba	ba	
<i>in</i>		ab	ab



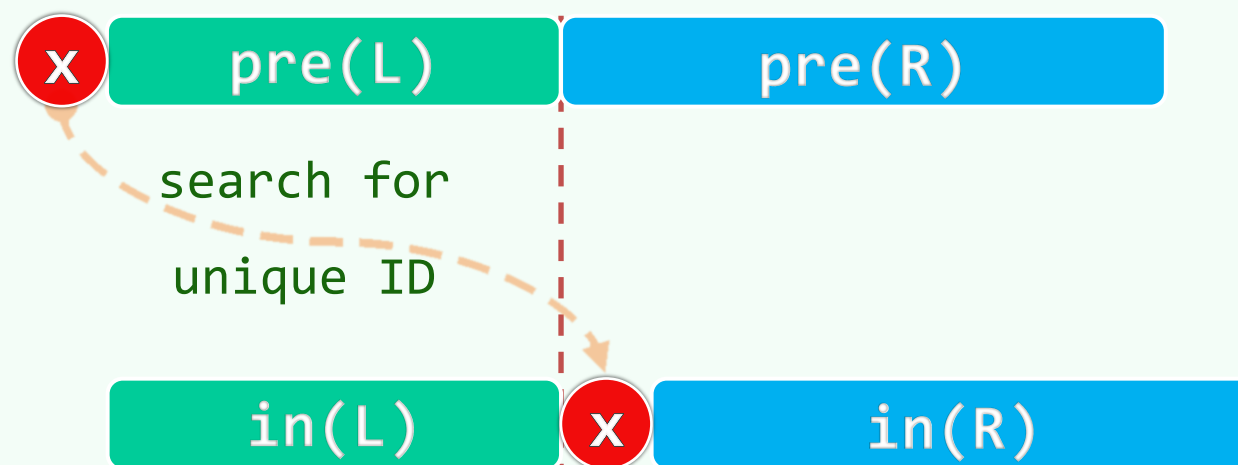
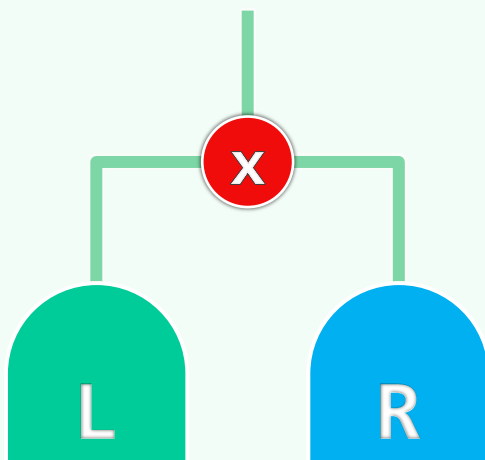
Level = ABCDEFG

pre = ABCDEGF

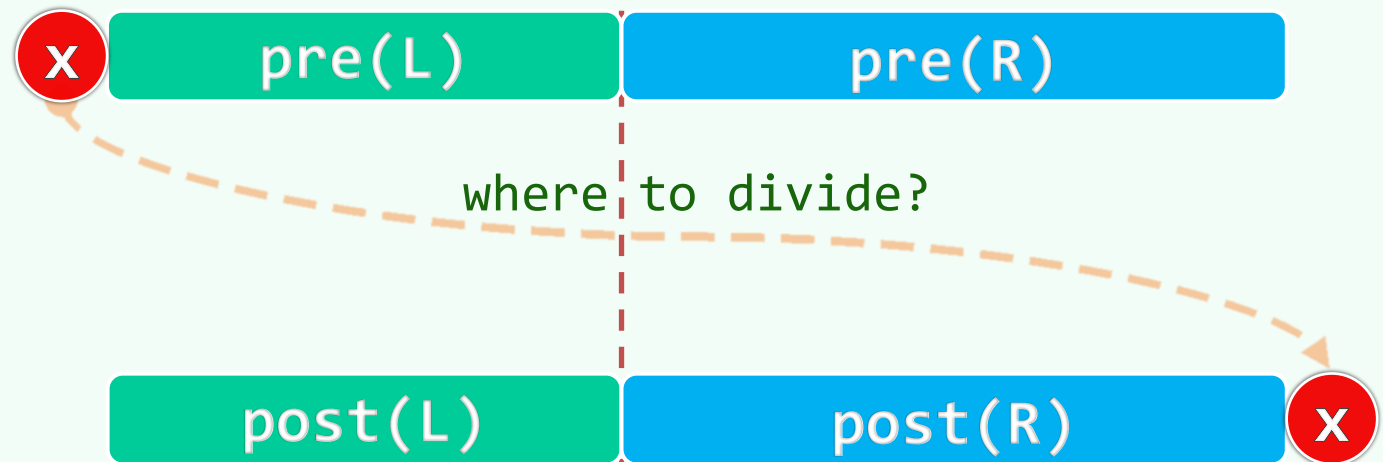
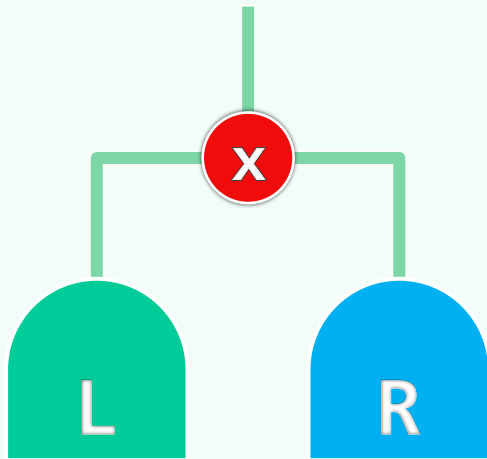
post = CGEFDBA

in = CBEGDFA

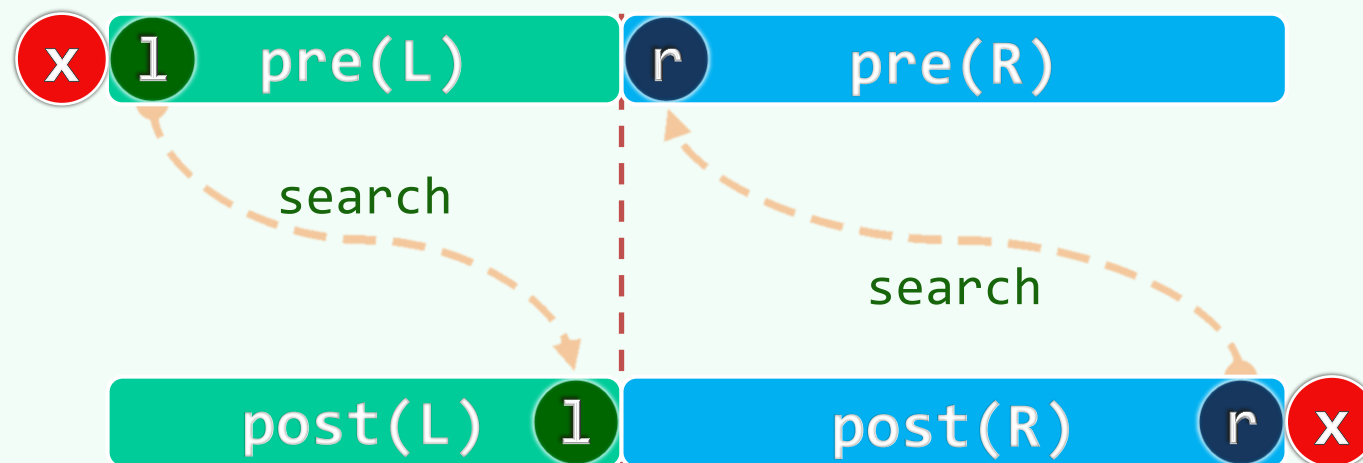
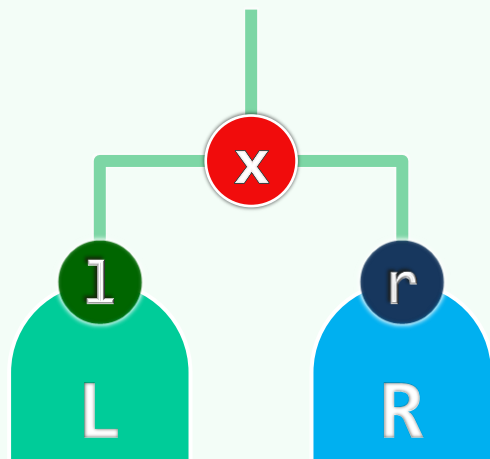
[先序 | 后序] + 中序



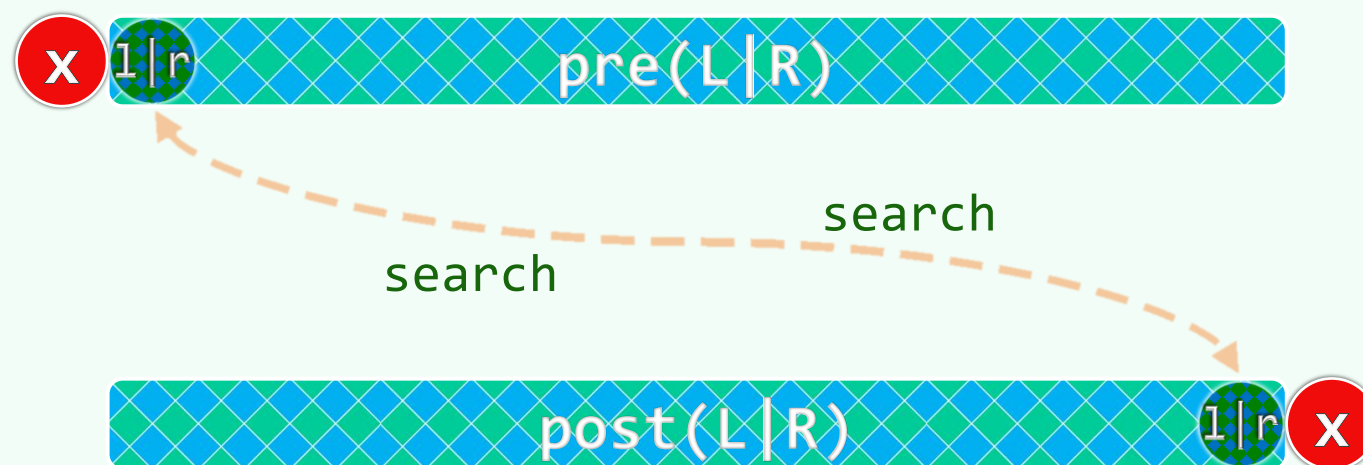
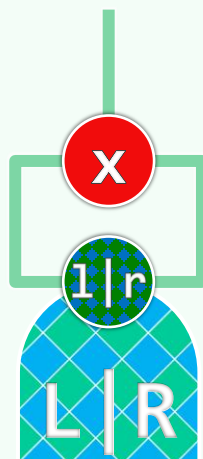
[先序 + 后序] ?



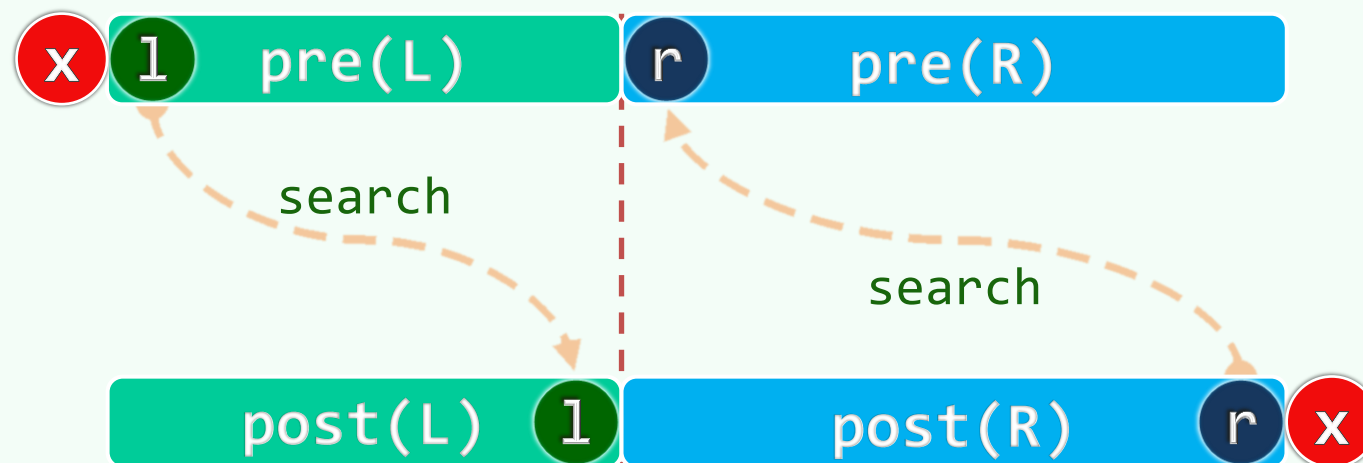
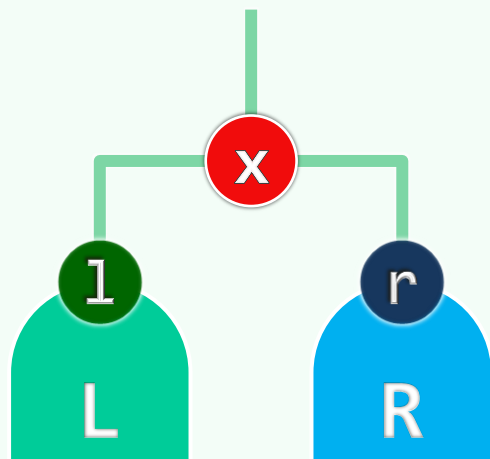
[先序 + 后序] !



[先序 + 后序] ? ?



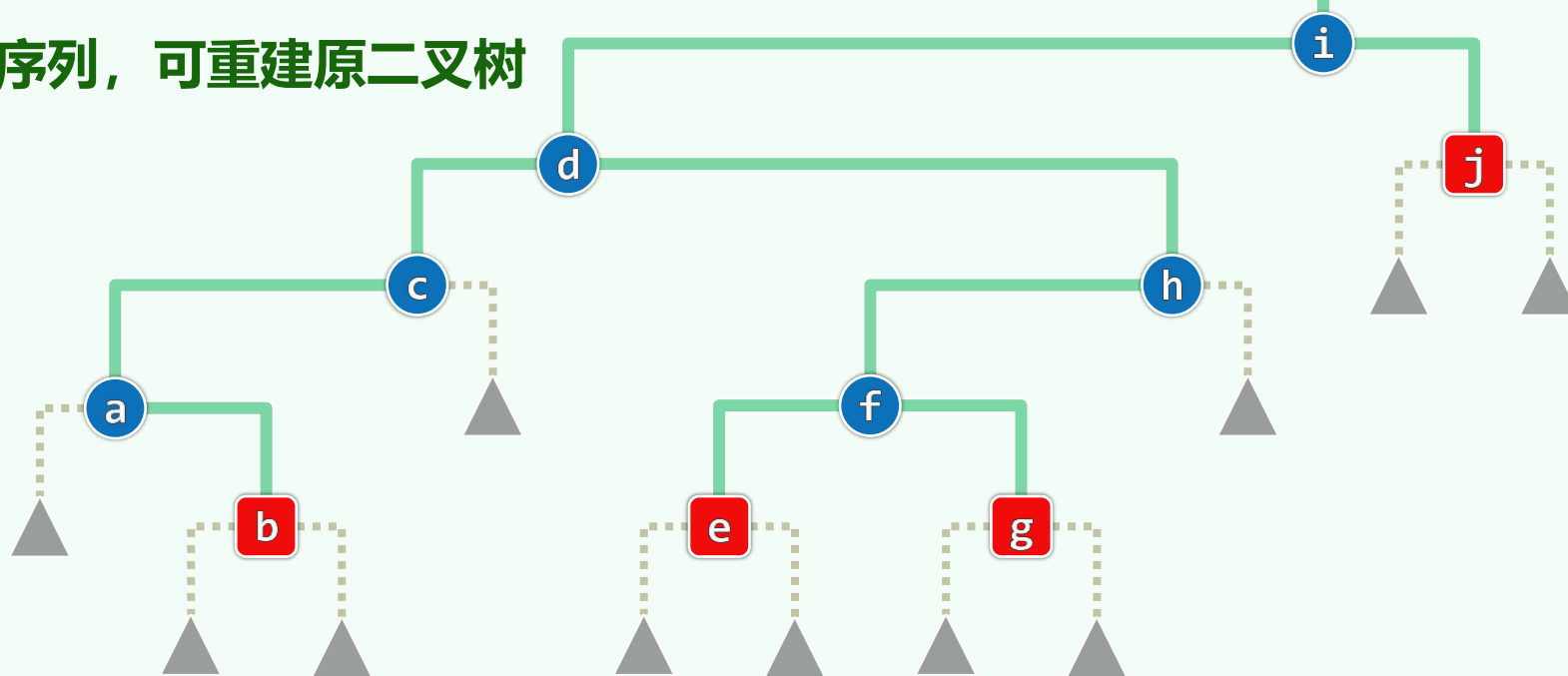
[先序 + 后序] x 真!



增强序列：遍历过程中遇到的NULL，也一并输出为“^”

❖ 归纳证明：由增强的先序遍历序列，可重建原二叉树

- 所有局部的 “ x ^ ^ ”
——对应于叶节点x
- 将其替换为 “ ^ ”
- ...



❖ 后序遍历，亦是如此

❖ 而中序遍历，则未必

比如， “ ^ a ^ b ^ ”

未必对应于子树a

preorder : i d c a ^ b ^ ^ ^ h f e ^ ^ g ^ ^ ^ j ^ ^

inorder : ^ a ^ b ^ c ^ d ^ e ^ f ^ g ^ h ^ i ^ j ^

postorder : ^ ^ ^ b a ^ c ^ ^ e ^ ^ g f ^ h d ^ ^ j i

二叉树

Huffman编码树：问题与算法

05-D1

句讀之不知，惑之不解，或師焉，或不焉，小學而大遺，吾未見其明也

两年的时间，在你看来，也许就是一眨眼的功夫，对不对？可对我来说，它实在长得没边。我用不着为两年后的事情操心

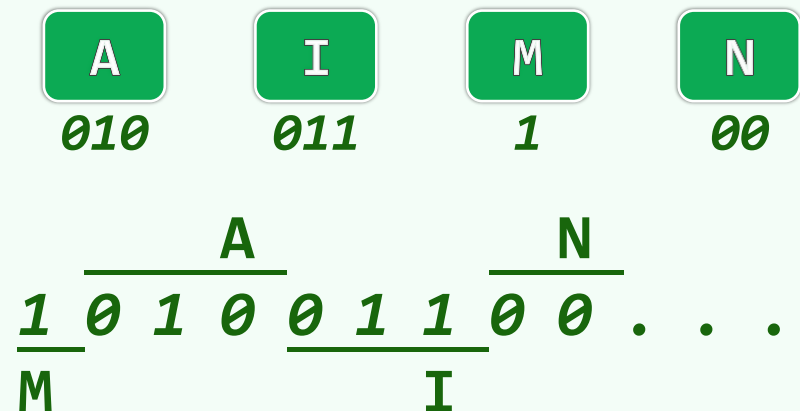
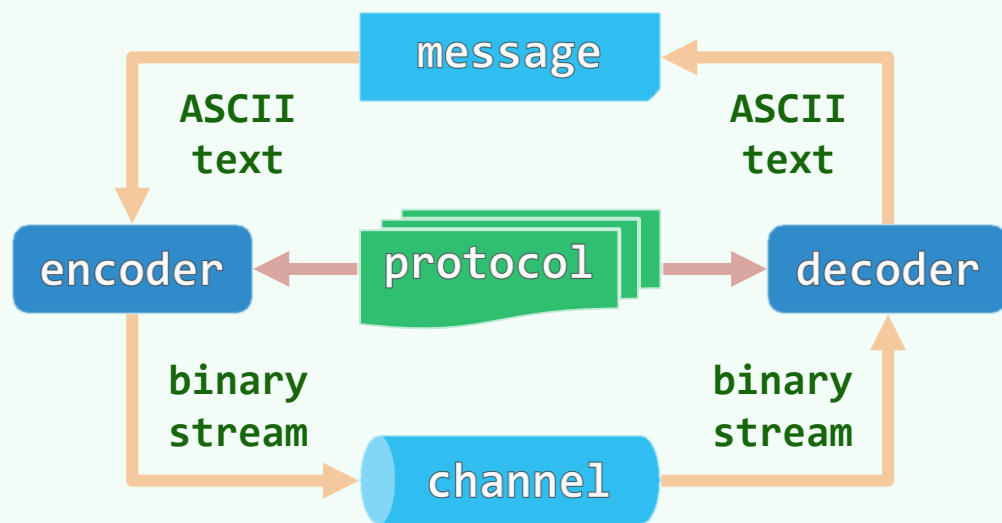
邓俊辉

deng@tsinghua.edu.cn

数字通讯中的编码

❖ 组成数据文件的字符来自**字母表** Σ , $|\Sigma|=n$

字符被赋予**互异**的二进制串



❖ **句读难题 ~ 歧义:**

某字符的编码, 可能恰是另一字符编码的**前缀**

❖ 比如, 倘若将**I**的编码改作**001**, 那么

解码者应当把**001**译作**I**, 还是**NM**?

前缀无关编码 | Prefix-Free Coding

❖ 将字符分别作为一个叶节点

随意地组成一棵真二叉树

❖ 字符 x ~ 从根到 x 的通路 ~ x 的编码

$L|R \sim 0|1$

❖ 在每一条通路上，除终点外都是内部节点

如此，自然就保证了前缀无关

❖ PFC并不唯一

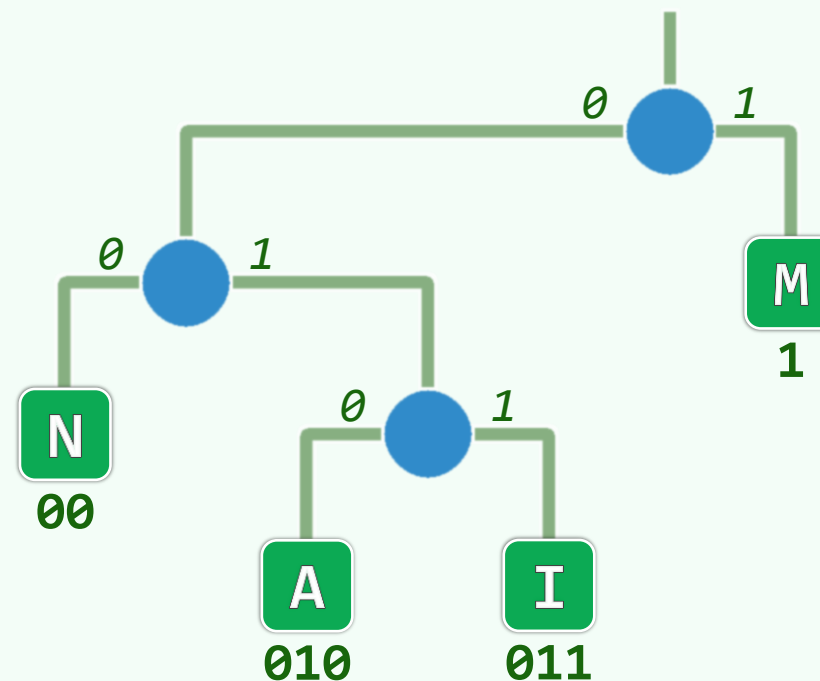
何者的编码效率最高、成本最低？

首先要定义好衡量与评估的尺度

❖ 编码长度 | Code Length

- 字符 ~ 对应叶节点的深度

- 整体 ~ 所有字符的编码长度总和



最优编码树 | Optimal Coding Tree

$$CL = 3+3+2+1 = 9$$

❖ CL更短 ~ 更优

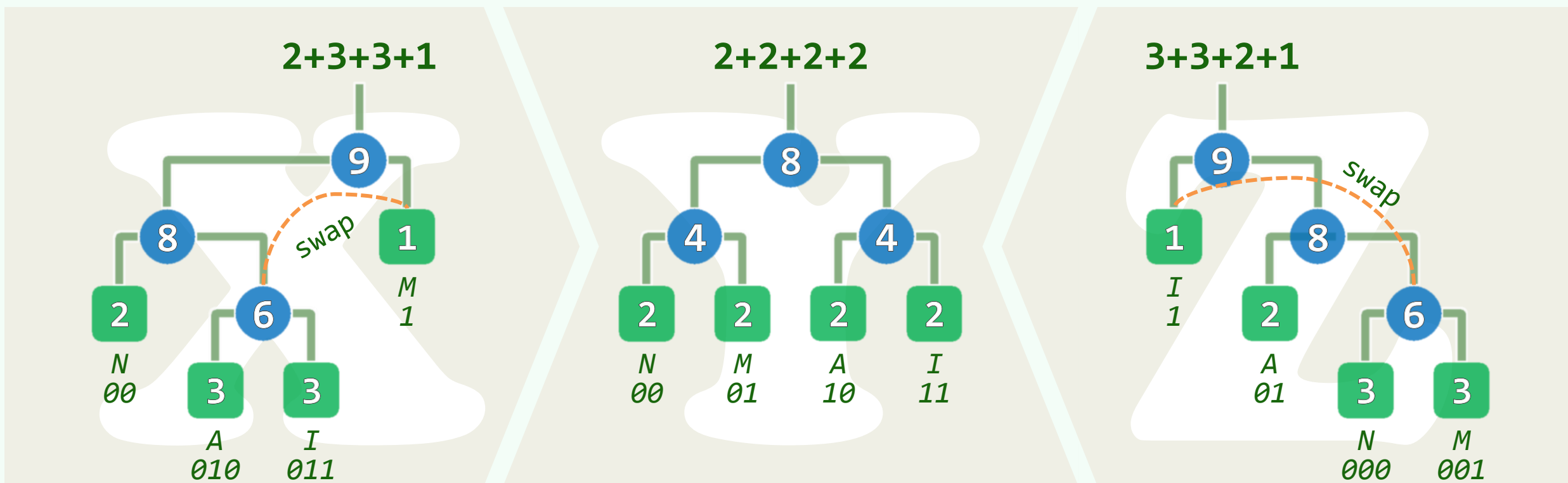
CL最短 ~ 最优

❖ OCT必然存在，虽未必唯一

❖ 叶子只能出现在**倒数两层**

否则，通过节点交换，总能降低CL

❖ **真的完全树**即是OCT...



频数 | Frequency



❖ 在不同的文化、时代、专业领域中
字符的出现**概率**或**频度**不尽相同
甚至，往往相差极大...

❖ 已知各字符的**期望频数**
如何构造最优编码树？



带权编码长度 | Weighted Coding Length

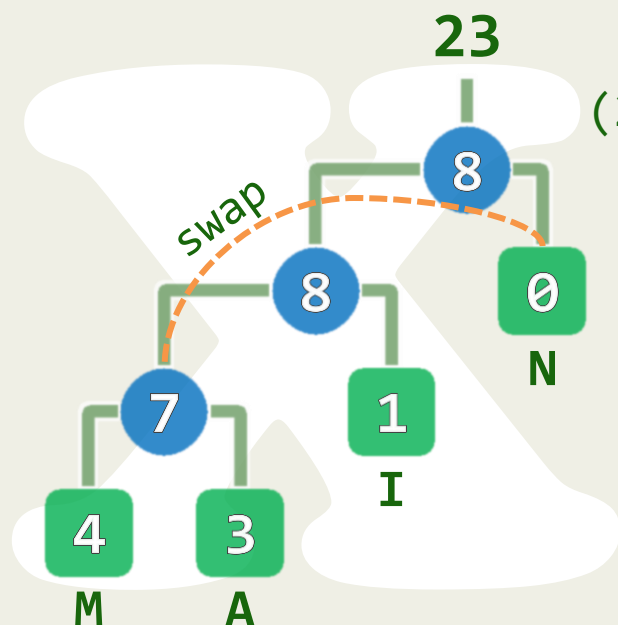
❖ 以字符的**频数**做加权，更符合实际

❖ 比如，对于MAMMAMIA

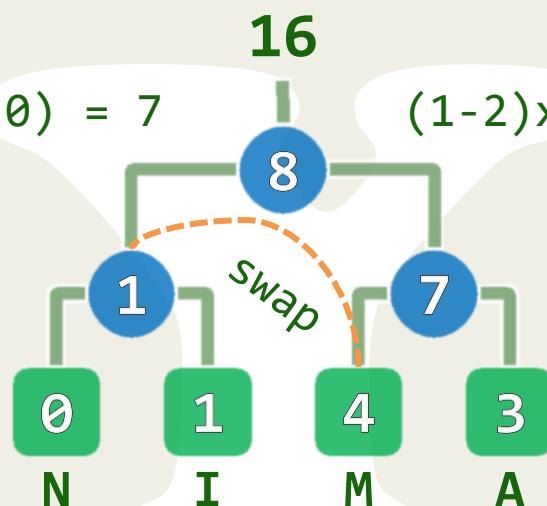
❖ 此时，完全树**未必**就是OWCT

A、I、M、N的**频数**分别为4、3、1、0

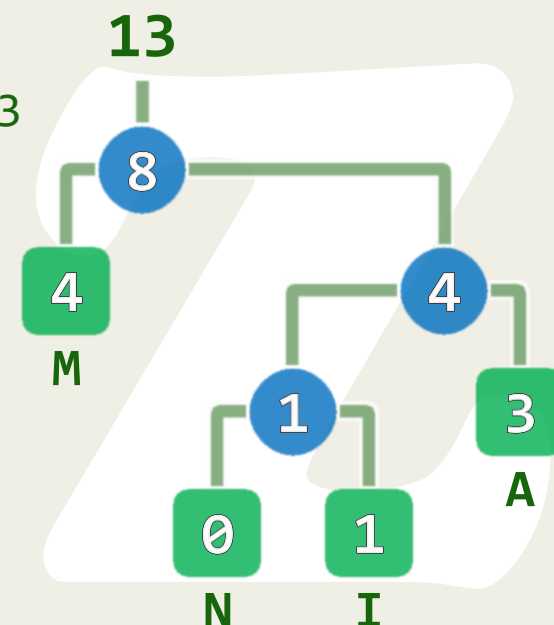
$$3 \times 3 + 2 \times 1 + 3 \times 4 + 1 \times 0$$



$$2 \times 3 + 2 \times 1 + 2 \times 4 + 2 \times 0$$



$$2 \times 3 + 3 \times 1 + 1 \times 4 + 3 \times 0$$



最优带权编码树 | Optimal Weighted Coding Tree

❖ 叶子节点的权重 = 字符的**频数**

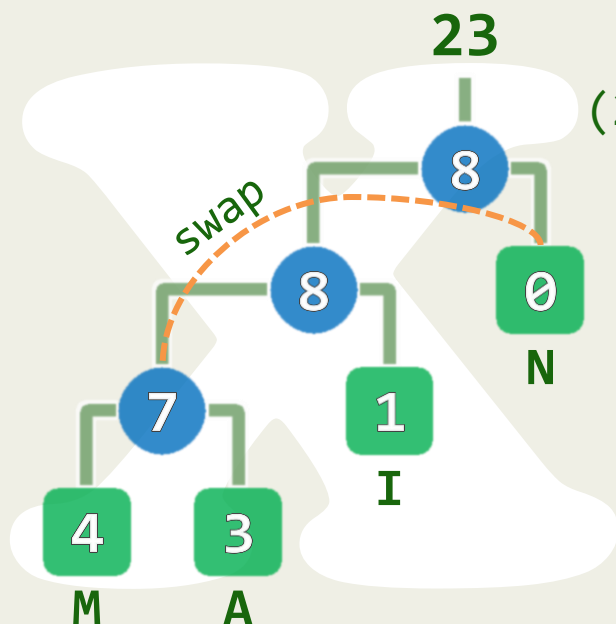
❖ 所谓OWCT, 就是WCL最小者

内部节点的权重 = 两个孩子的权重之和

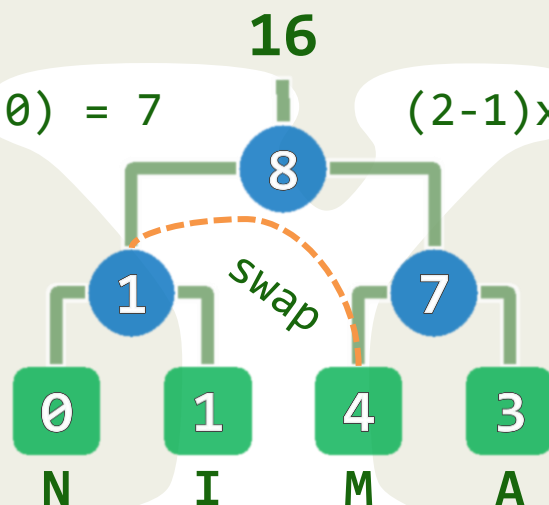
❖ 通过**适当**交换, 同样可以缩短WCL

❖ 字符的WCL**总和** = WCL = 内部节点的权重**总和** 频数**高** | **低**的字符, 尽可能放在**高** | **低**处

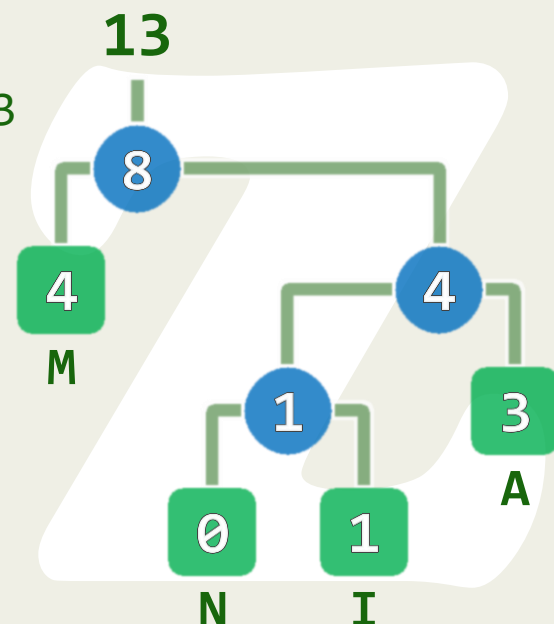
$$3 \times 3 + 2 \times 1 + 3 \times 4 + 1 \times 0$$



$$2 \times 3 + 2 \times 1 + 2 \times 4 + 2 \times 0$$



$$2 \times 3 + 3 \times 1 + 1 \times 4 + 3 \times 0$$



Huffman算法, 1952

❖ **贪心策略**: 自底而上地逐个**拼接**: 频数**小** | **大**的字符, **更早** | **晚**拼接

❖ **字符**各自成一棵树, **权重**即其频数; n 棵树组成**森林** F

while (F 中的树不止一棵)

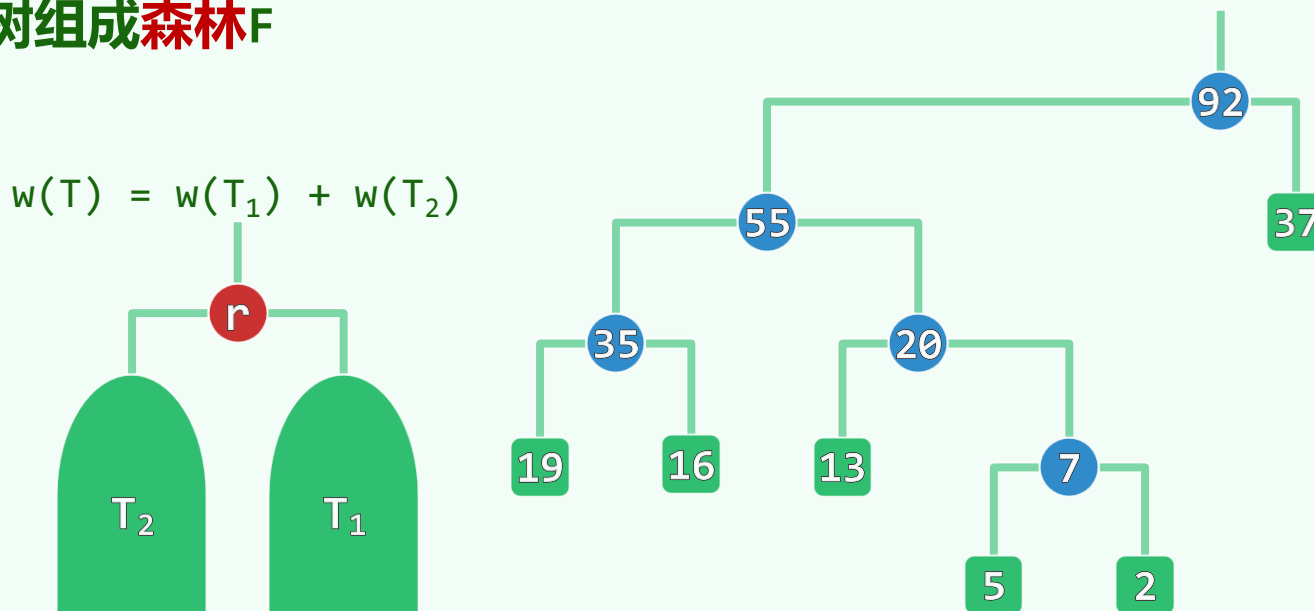
取出频率**最小**的两棵树: T_1 和 T_2

将它们**合并**成一棵新树 T

- 以 T_1 、 T_2 为左、右**子树**
- **权重**取作 T_1 与 T_2 权重之和

最终**唯一**剩下的那棵树, 便是一棵OWCT

❖ **贪心策略**通常并不能保证得到**最优解**, Huffman的算法是个**例外**吗?



二叉树

Huffman编码树：正确性

05-D2

幸福与荒谬是同一大地的两个产子，它们是不可分的

我生来就不像我所见过的任何一个人；我敢断言，我与世上的任何一个人
都迥然不同；虽说我不比别人好，但至少我与他们完全两样

邓俊辉

deng@tsinghua.edu.cn

不确定中的确定

❖ 当我们宣称Huffman树是OWCT时，是指**哪一棵**？

❖ OWCT通常都有**多棵**

- 比如，深度**相同**的子树**总是**可以互换
- 甚至，深度**不同**者，**或许**也能互换

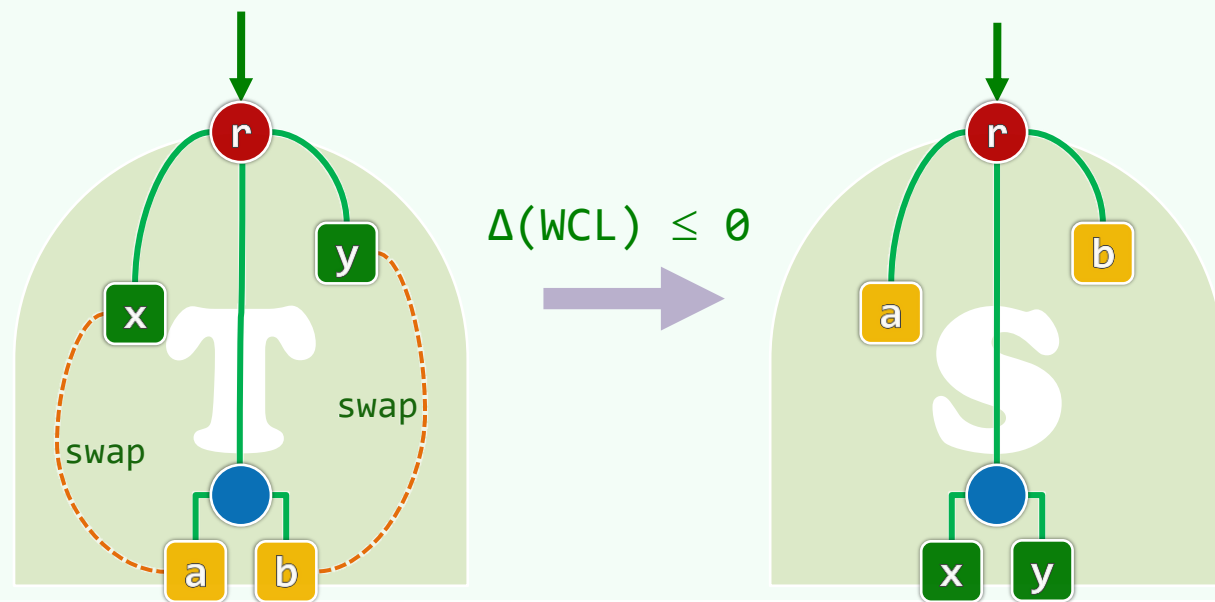
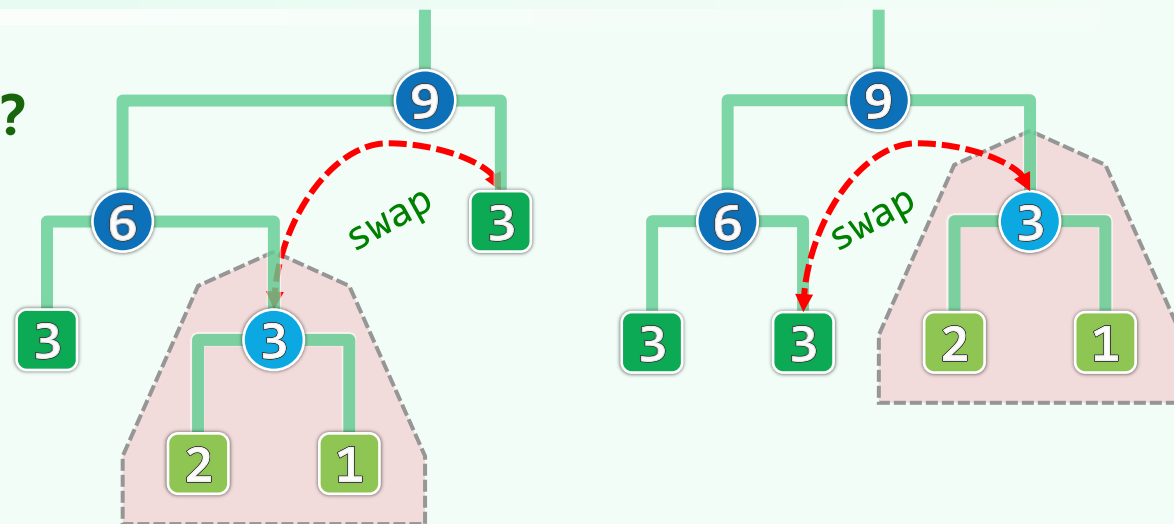
❖ 对某些**细节**，Huffman算法也未明确

❖ 约定：就以**我们**实现的版本为准

其中最重要地：**左**兄弟均不小于**右**兄弟

❖ 【最小字符成对】

频数**最低**的字符x和y，必在**某棵**OWCT中是兄弟



数学归纳： Huffman算法所生成的，必是一棵OWCT

❖ 假设 $|\Sigma| < n$ 时均成立；考查任何 $|\Sigma| = n$

❖ 引入字符 z : $w(z) = w(x) + w(y)$

并得到字符集 $\Pi = (\Sigma \setminus \{x, y\}) \cup \{z\}$

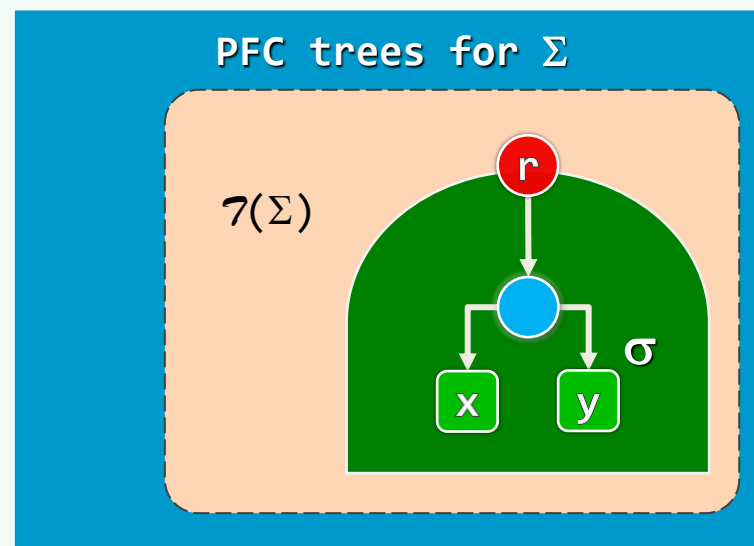
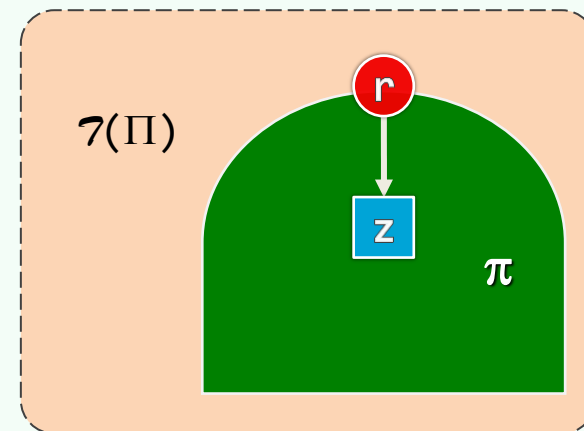
❖ $\mathcal{T}(\Sigma)$: Σ 所有满足 “x与y成对” 的编码树

$\mathcal{T}(\Pi)$: Π 的所有编码树

❖ 两个集合之间存在一一对应关系，而且

- 若 $\sigma \in \mathcal{T}(\Sigma)$ 对应于 $\pi \in \mathcal{T}(\Pi)$
- 则有 $WCL(\sigma) - WCL(\pi) \equiv CL(x) + CL(y)$

❖ 差距**固定**，最优必**对应**于最优



05-D3

二叉树

Huffman编码树：算法实现

树形建筑也出现了，看上去规模与地球上的差不多，只是挂在树上的建筑叶子更为密集

邓俊辉

deng@tsinghua.edu.cn

超字符 + 树

❖ struct HuffChar { //Huffman字符

char **ch**; uint32_t **weight**; //字符、频数

HuffChar(uint32_t w = 0, char c = '^') : weight(w), ch(c) {}; //构造

HuffChar(HuffChar const& hc) : weight(hc.weight), ch(hc.ch) {}; //复制

bool operator<(HuffChar const& hc) //比较器

{ return weight > hc.weight; } //故意大小颠倒

}; //HuffChar

❖ using HuffTree = BinTree< HuffChar >; //Huffman树: 由BinTree派生

#define Weight(T) ((T).root()->data.weight) //快捷方式: 根节点的频数, 即子树的权重

森林 ~ 优先级队列

- ❖ `using HuffForest = PQ_List< HuffTree >;` // Huffman森林: 暂用**列表**实现
- `using HuffForest = PQ_ComplHeap< HuffTree >;` // 日后可改用**完全二叉堆**实现
- `using HuffForest = PQ_LeftHeap< HuffTree >;` // 也可改用**左式堆**实现
- ❖

```
template <typename T> class PQ_List : public PQ<T>, public List<T> {  
    public: PQ_List( T* E = NULL, int n = 0 ) { while (0 < n--) insertFirst(E[n]); }  
        void insert( T e ) { insertLast( e ); } //新元素直接插至队尾  
        T delMax() { return remove( List::getMax() ); } //删除最优先的元素  
        T& getMax() { return List::getMax()->data; } //取出最优先的元素  
}; //PQ_List
```
- ❖ 算法的**核心引擎**是`delMax()`: 但`List::getMax()`是**顺序查找**, 需**线性时间**, **拖累**了整个算法...

构造Huffman树：贪心合并

❖ HuffTree huffTree(HuffForest& F) {

while (1 < F.size()) { //每迭代一步，森林中都会减少一棵树

HuffTree T1 = F.delMax(), T2 = F.delMax(), S; //取出权重**最小**的两棵树，合并成S

S.insert(HuffChar(Weight(T2) + Weight(T1))); //权重取作子树权重之**和**

S.attach(T2, S.root()); S.attach(S.root(), T1); //T2权重**不小于**T1

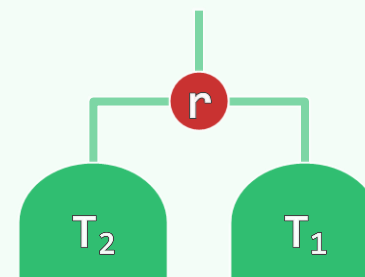
F.insert(S); //再**插回**森林

} //森林中最终唯一所剩的那棵树，即Huffman编码树

return F.delMax(); //且层次序列必然单调**非增**

} //huffTree

$$w(T) = w(T_1) + w(T_2)$$



❖ 日后将PQ_List替换为PQ_ComplHeap或PQ_LeftHeap，复杂度便会从 $O(n^2)$ 直接降到 $O(n \log n)$ ！

编码表 ~ 散列表

```
❖ #include "HashTable.h" //用HashTable (第09章) 实现

    using HuffTable = Hashtable< char, char* >; //Huffman编码表

❖ HuffTable* huffCodeTable( HuffTree tree ) { //将各字符编码, 统一存入

    Hashtable table; //以散列表实现的编码表

    Stack<char> theseus; theseus.push('x');

    huffCT( theseus, table, tree.root() );

    return table;

}; //huffCodeTable
```

导出编码表 ~ 递归遍历

```
void huffCT( Stack<char> theseus, Hashtable& table, BNP<HuffChar> v ) {  
    if ( IsLeaf( v ) ) { //每遇到一个叶节点, 便导出对应的编码串, 并添加到编码表中  
        int n = theseus.size() - 1; Stack<char> sueseht;  
        for ( Rank k = 0; k < n; k++ ) sueseht.push( theseus.pop() );  
        char* code = new char[n + 1]; code[n] = '\0';  
        for ( Rank k = 0; k < n; k++ ) theseus.push( code[k] = sueseht.pop() );  
        table.put( v->data.ch, code );  
    } else { //否则, 继续递归地遍历  
        if ( v->lc ) { theseus.push('0'); huffCT( theseus, table, v->lc ); } //0左  
        if ( v->rc ) { theseus.push('1'); huffCT( theseus, table, v->rc ); } //1右  
    } //else  
    theseus.pop();  
} //huffCT
```

二叉树

Huffman编码树：便捷的改进

05-D4

拿中国的情形来说，我们所依靠的不过是小米加步枪，但历史最后将证明，这小米加步枪比蒋介石的飞机加坦克还要强些

邓俊辉

deng@tsinghua.edu.cn

向量 + 列表 + 优先级队列

❖ 方案1: - 初始化时, 通过排序得到一个**非升序向量** $//O(n\log n)$

$O(n^2)$ - 每次 (从**后端**) 取出频率最低的两个节点 $//O(1)$

- 将合并得到的新树插入向量, 并保持有序 $//O(n)$

❖ 方案2: - 初始化时, 通过排序得到一个**非降序列表** $//O(n\log n)$

$O(n^2)$ - 每次 (从**前端**) 取出频率最低的两个节点 $//O(1)$

- 将合并得到的新树插入列表, 并保持有序 $//O(n)$

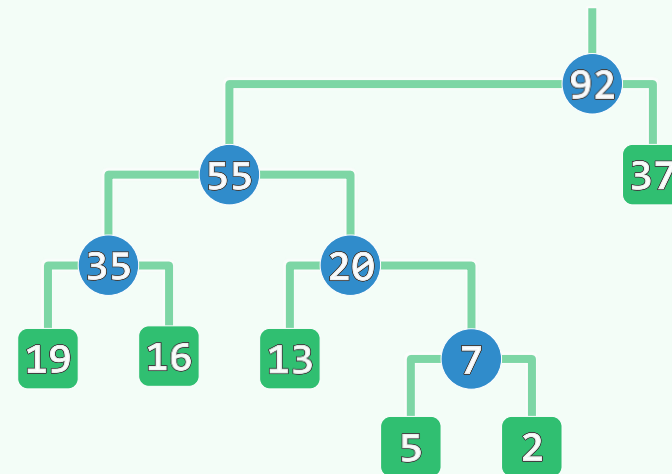
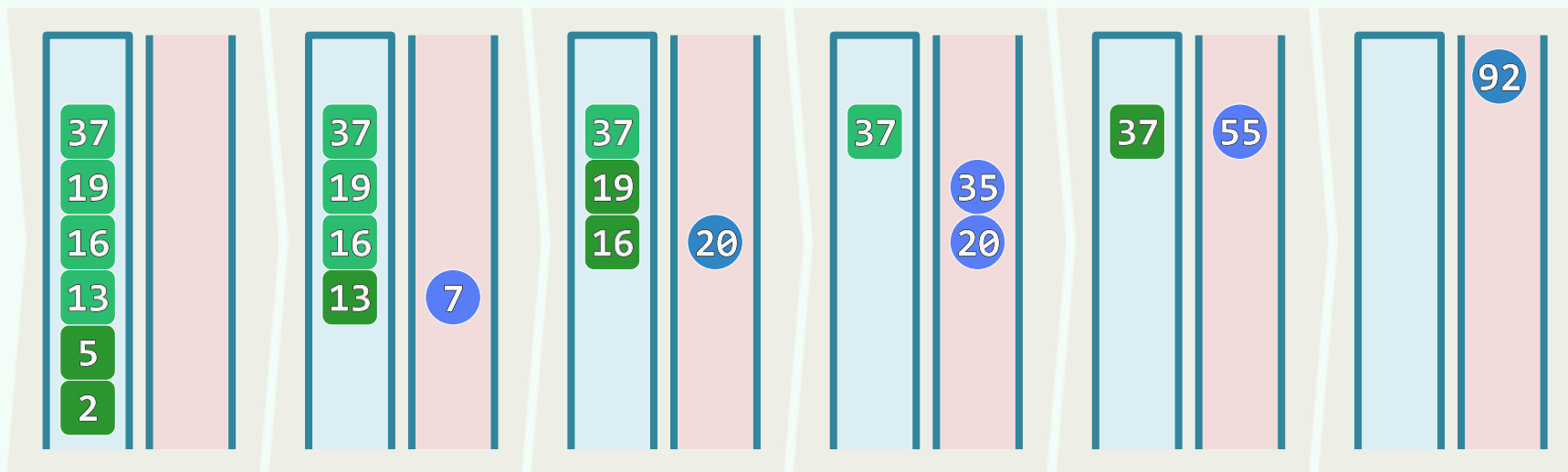
❖ 方案3: - 初始化时, 将所有树组织为一个**优先级队列** (第12章) $//O(n)$

$O(n\log n)$ - 取出频率最低的两个节点, 合并得到的新树插入队列 $//O(\log n) + O(\log n)$

预排序 x (栈 + 队列)

❖ 方案4: - 所有字符按频率非升序入栈 $O(n \log n)$

$O(n \log n)$ - 维护另一 (有序) 队列... $O(n)$



二叉树

下界与最优：比较树

要是你达到一个界线，你不能越过它，那你就会倒霉；但是你越过了它，也许你会更倒霉

就好比假设有四样东西，我们正在寻找其中的一个。不管它在哪里，如果我们一开始就知道这样东西在哪里的话，那么剩下的就不是什么问题了。又或者我们可以首先找到其余的三样东西，那么剩下的显然就是第四个了

此时你的思想进入我的思想
带有同样的行动和同样的面貌
使得我把二者构成同一个决定

邓俊辉

deng@tsinghua.edu.cn

下界 ~ 最优

❖ 我们已经多次看到

- 同一问题的不同算法，复杂度可能相差**悬殊**
- 通过敏锐观察与精巧设计，往往能使算法性能有所**改进**

❖ 什么是改进？就什么标准而言？**最坏**情况下的性能

❖ 针对任何一个具体的计算问题，算法的改进**不可能**持续获得

- 算法改进的**尽头**，称作该问题的复杂度**下界** | lower bound
- 性能达到下界的算法，即**最坏情况下最优的算法** | worst-case optimal algorithm

苹果甄别

❖ 从4个外观相同的苹果中

找出唯一的重量不同者

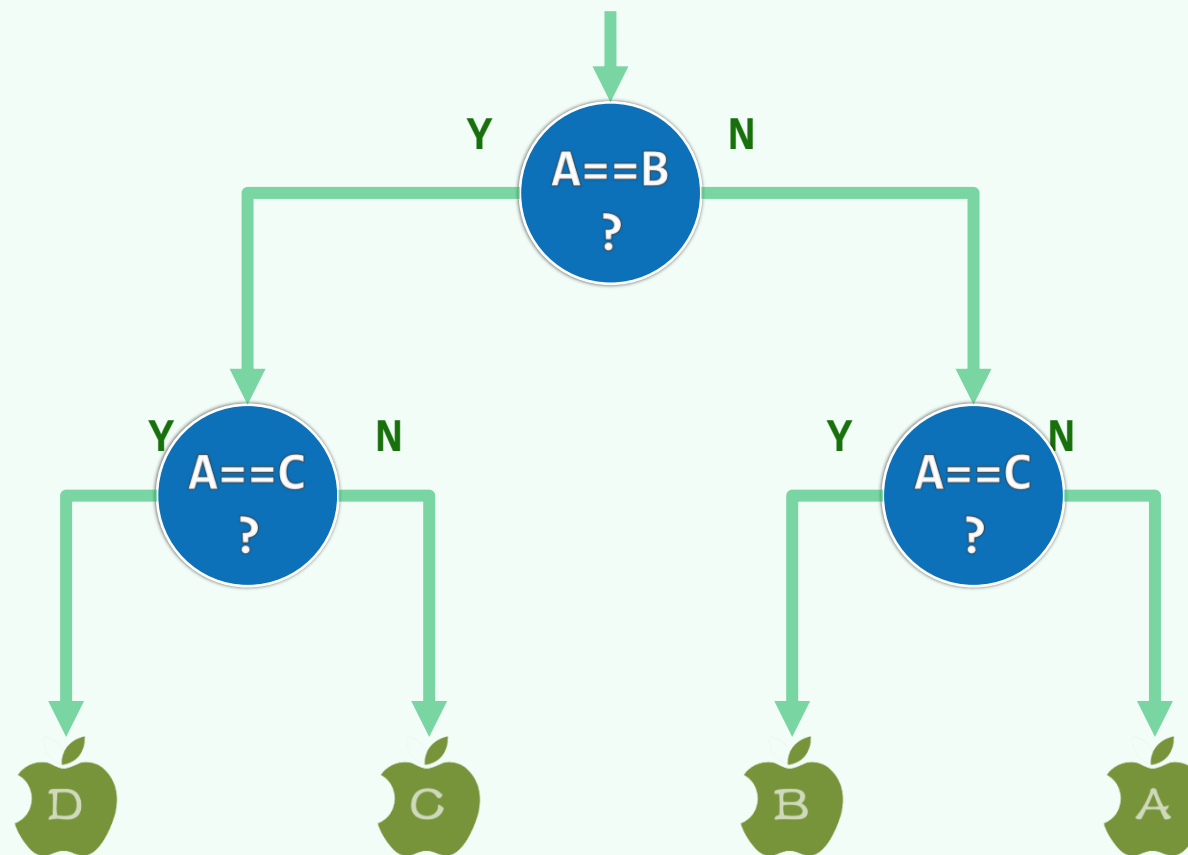
❖ 借助天平，只需2次称量

❖ 称量次数可否更少？

2次已是下界——1次无论如何不够

❖ 对于更一般的问题

如何确定下界呢...



比较树

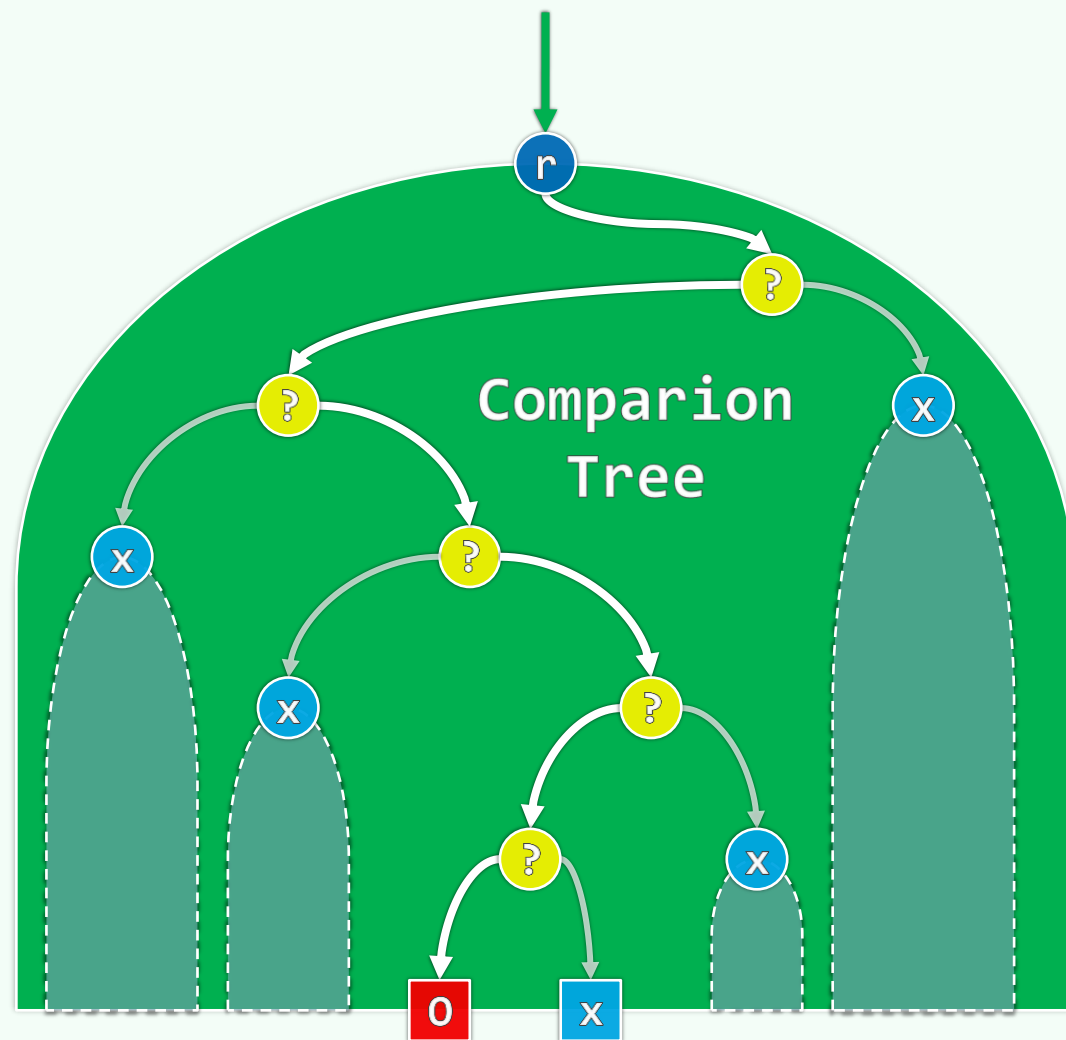
❖ 基于**比较**的算法 | comparison-based algorithm

❖ 每个CBA都对应于一棵**比较树** | comparison tree

- 运行的**输出** ~ 树中的**叶节点**
- 运行的**过程** ~ 从根到叶节点的**路径**
- 消耗的**时间** ~ 路径长度
- **最坏**情况下消耗的时间 ~ 树的**高度**

❖ n 种输出 ~ **至少** n 个叶节点

下界 ~ 树高 = $\Omega(\log_2 n)$



实例

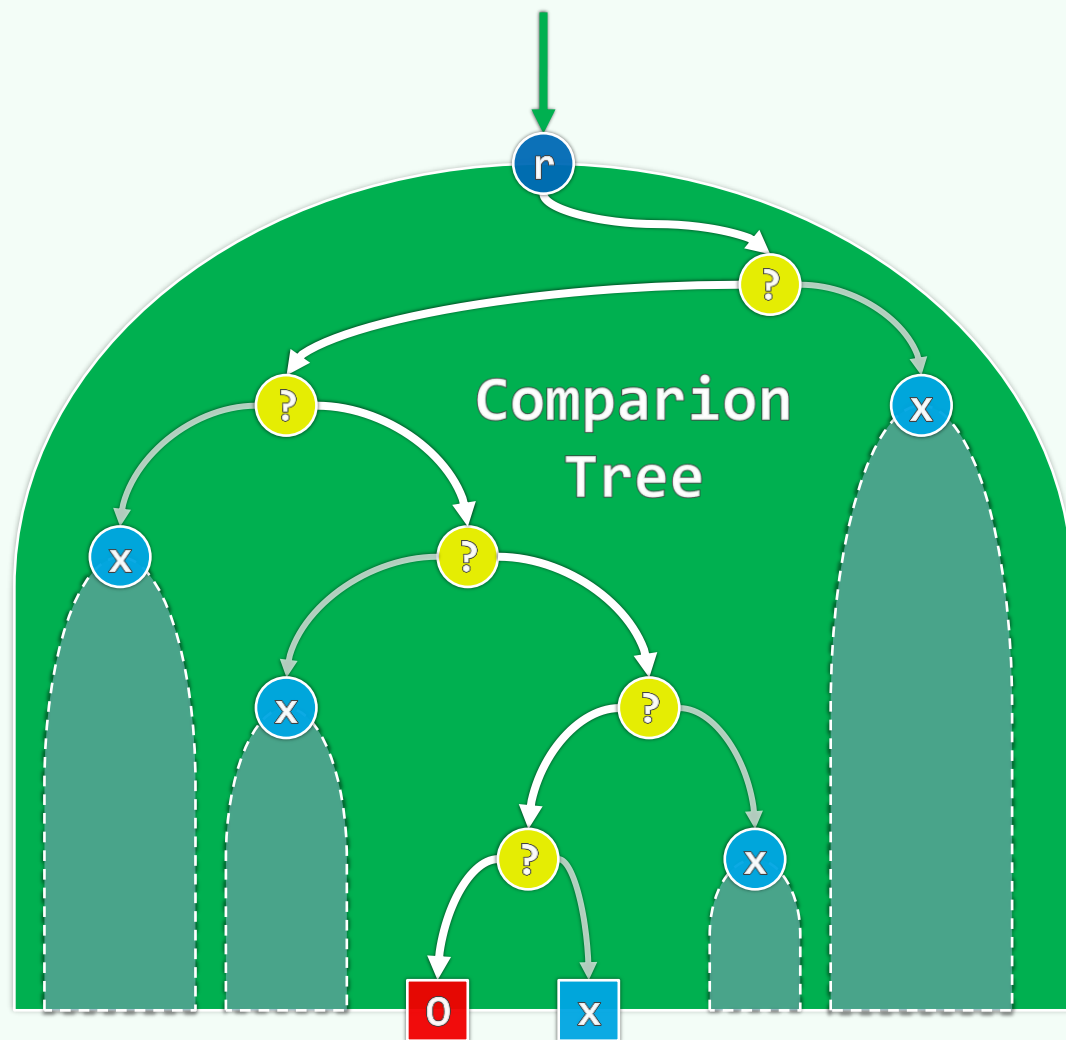
❖ 苹果甄别

- 共有4种输出
- 下界 = $\log_2 4 = 2$ 次称量

❖ n个元素的排序

- 共有 $n!$ 种输入
- 下界 = $\Omega(\log_2(n!)) = \Omega(n \log n)$
- 归并排序属于WCOA

❖ 每个问题的下界，都只能这样确定吗...



二叉树

下界与最优：线性归约

言有易，言無難

不怕不识货，就怕货比货

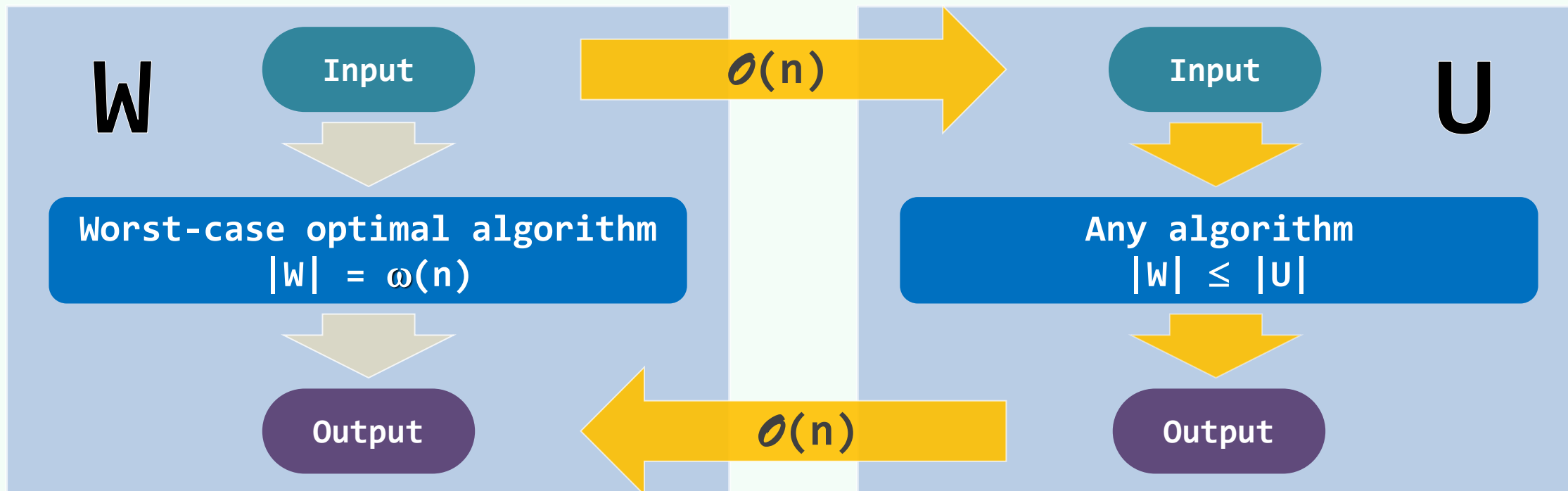
但是随着种种事实的增多，人们也学会了对它们做出分类，把它们简约为更普遍的事实...
简约为更广泛的关系并把它们纳入更简单的表达方式...使人能掌握其更大的数量，哪怕
自己所拥有的仍然只不过是同样的脑力而自己所运用的仍然只不过是同等的注意力

邓俊辉

deng@tsinghua.edu.cn

线性归约 (Linear-Time Reduction)

❖ 只要 $W \leq_N U$, U 的每个算法, 也对应于 W 的一个算法: 输入转化 + U 算法 + 输出转化 // 曲径通幽



linear-time reduction

NP-complete/Polynomial-time reduction

P-SPACE complete/ Polynomial-time many-one reduction

$\text{SORTING} \leq_N \text{HUFFMAN}$

❖ SORTING的每个输入 // n 个实数

可以直接转换为 (理解为) // $O(1)$

HUFFMAN的一个输入 // n 个字符的频数

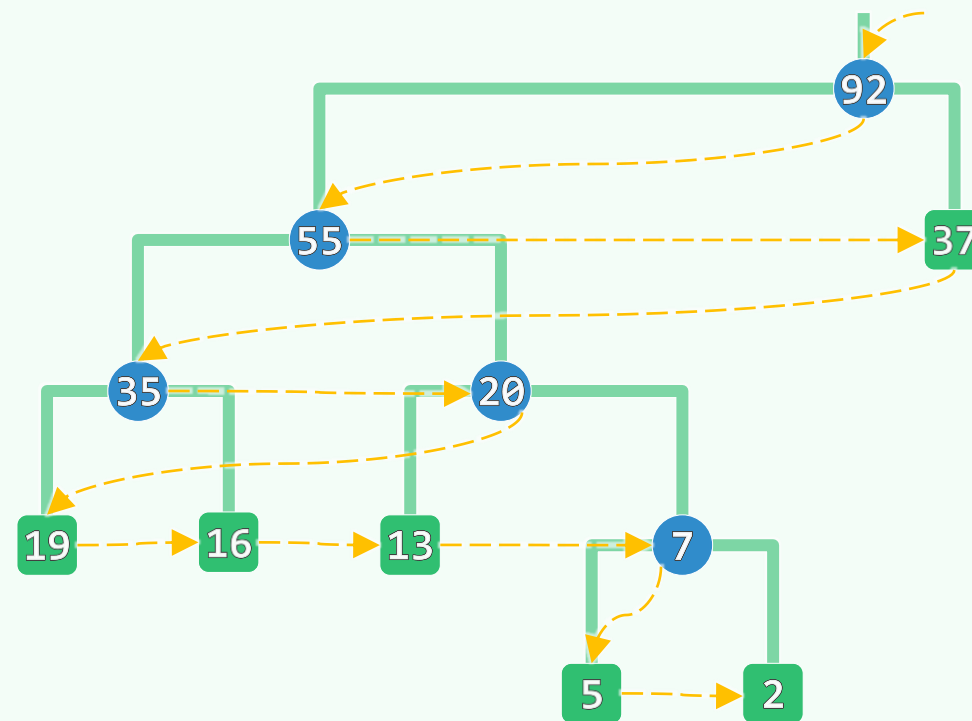
❖ HUFFMAN的输入 // 编码树

经层次遍历, 必然得到 // 高高低低, $O(n)$

有序序列 // 包含所有内部节点和叶节点

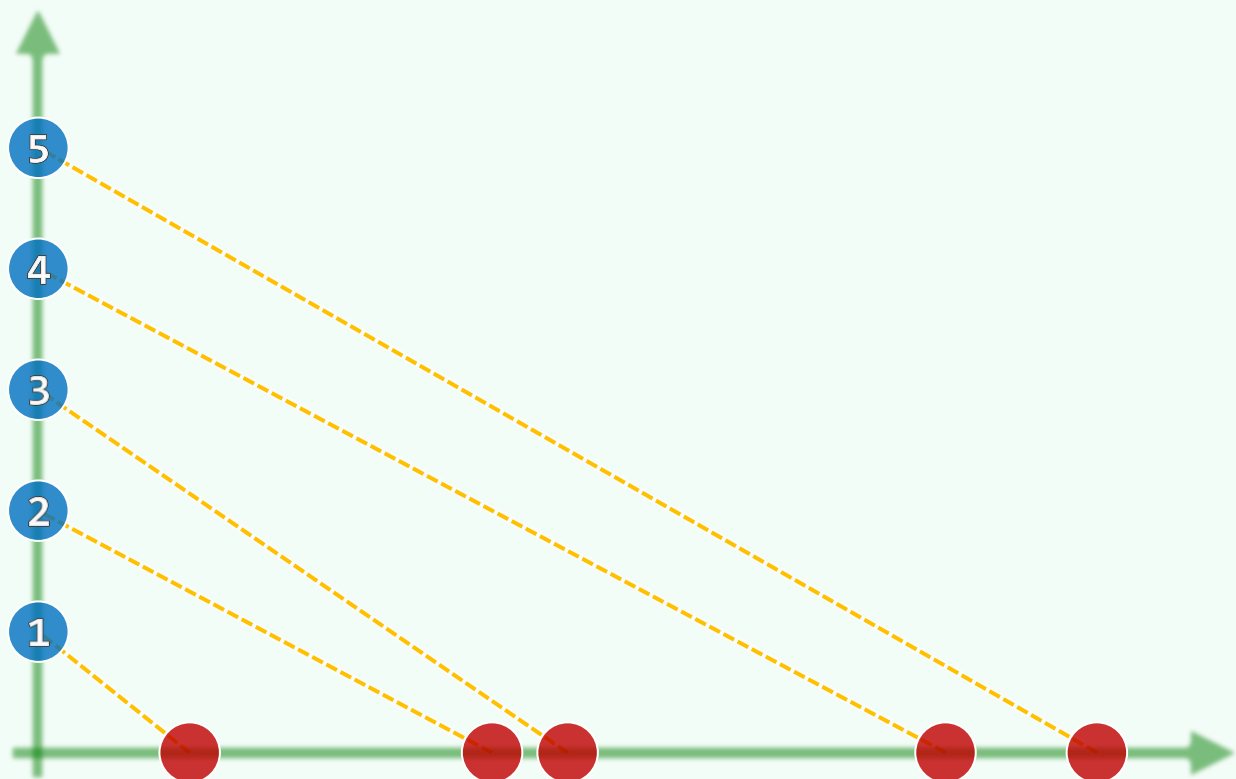
❖ 故有: $\text{SORTING} \leq_N \text{HUFFMAN}$

$$\Omega(n \log n) = |\text{SORTING}| \leq |\text{HUFFMAN}|$$



$\text{SORTING} \leq_N \text{RBM}$

- ❖ 【Red-Blue Matching】任给平面上互不重合的**红点**、**蓝点**各 **n** 个，试用 **n** 条**无交的**线段联接**异色点**
- ❖ SORTING的每个输入 $//n$ 个实数
可以直接转换为（理解为） $//O(1)$
 x 正半轴上的 **n** 个**红点** $//O(n)$ 预处理
- ❖ 再加入 **y** 正半轴上**有序**的 **n** 个**蓝点**
即得到RBM的一个输入
- ❖ RBM对应的输出
只需一趟**遍历**，便可转化为 $//O(n)$
SORTING的输出



$$SD \leq_N DIAM$$

❖ 【Set Disjointness】任给整数集合A和B，判断是否有公共元素

已知: $\Omega(n \log n) = |SD|$

❖ 【DIAM】在平面上任给一组点，找出相距最远的点对

直径 | diameter

❖ 对于SD的每个输入，将A | B中的整数

分别映射到单位圆的上 | 下半圆弧 // 比如CCW

于是...

❖ SD的输出为true | false，当且仅当

DIAM的输出为2——点集中存在对跖点 | antipodals

